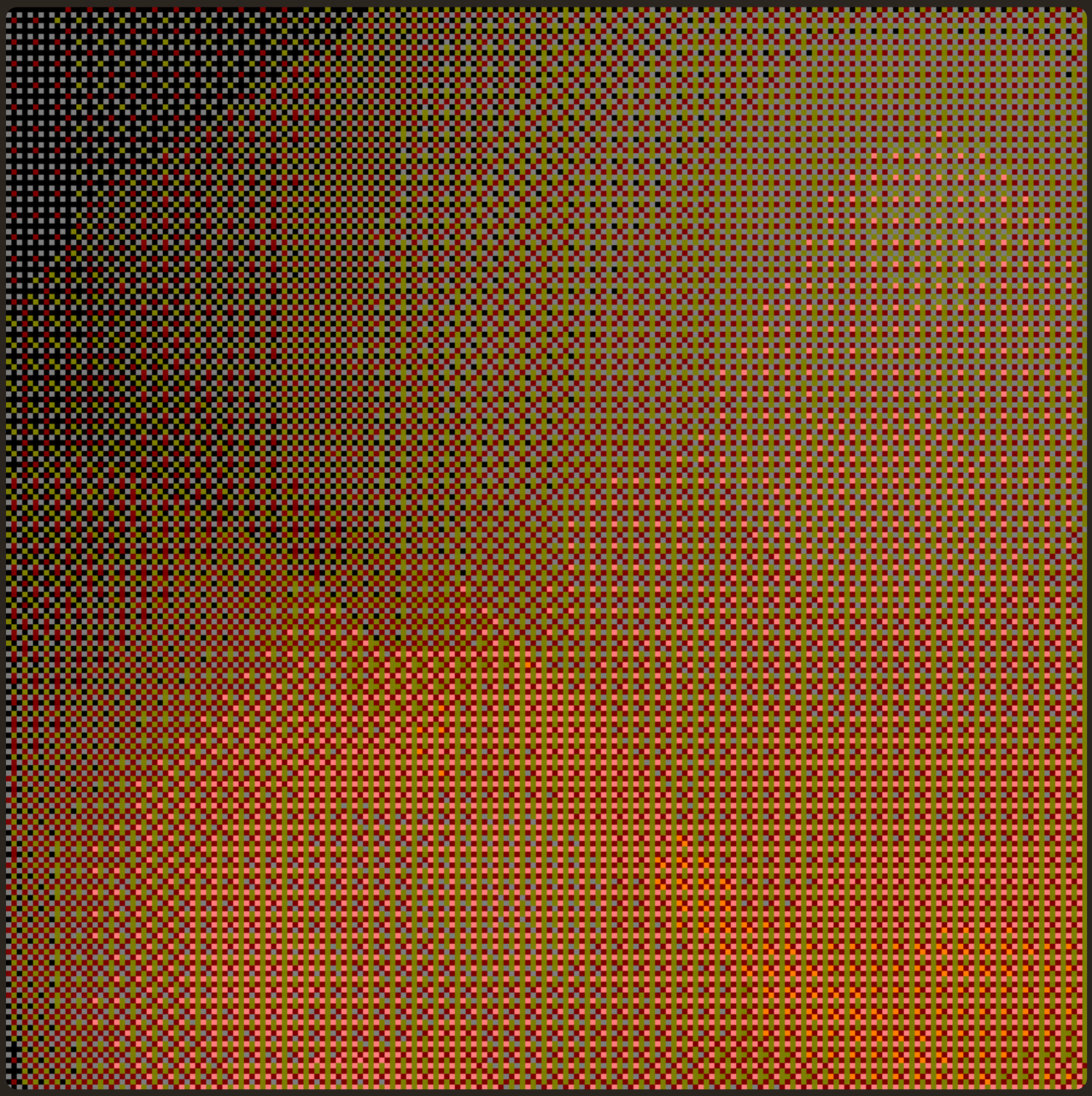
ZKSECURITY

Audit of OpenVM 2.0

AXIOM • JUNE 9TH, 2026 • TECHNICAL REPORT



§01

Introduction

On March 27th, 2026, zkSecurity started a security audit of OpenVM 2.0, spanning multiple components in different audit phases. The engagement was split into four different phases.

- **Phase 1** focused on the SWIRL proof system, the `stark-backend` native verifier, and the recursive verifier. The audit lasted four weeks, with two consultants, starting March 27th, 2026. We reviewed the `stark-backend` repository at commit `66dee8ab` and the `openvm` repository at commit `c89d052d`.
- **Phase 2** focused on the continuations aggregation pipeline and the new deferral framework. The audit lasted two weeks with two consultants, starting May 18th, 2026. We reviewed the openvm repository at commit `f01c912`.
- **Phase 3** focused on the `static-verifier` crate (part of the OpenVM v2 implementation), which implements a STARK verifier using Halo2 with KZG polynomial commitments, and is used to "compress" the final STARK proof from OpenVM into a small PlonK proof. The audit lasted two weeks with two consultants, starting May 18th, 2026.
- **Phase 4** focused on OpenVM 2.0.0-rc.1. The scope was the diff between the openvm repository's `develop-v2.0.0-beta` branch (commit `f01c912`) and its `develop-v2.0.0-rc.1` branch (commit `0110850`), together with the forked guest libraries that patch upstream hash crates to use OpenVM intrinsics. A Lean4 formal-verification supplement (the `openvm-fv` repository) accompanied the new SHA-2 and Keccak AIRs, providing extracted AIR-constraint theorems for those chips. The audit lasted two weeks with two consultants, starting May 18th, 2026.

Additional code reviewed

We additionally reviewed some small subsequent code changes:

- PR #2863 which fixes the proving SDK for deferral circuits in the case the deferral circuits do not form a complete binary tree, by padding with absent proofs up to the next power of two.
- PR #2900 which replaces `recursion_flag` with `recursion_depth`, which tracks the current depth of the main aggregation tree.
- PR #2903 which implements a minor change in the `EqNegBaseRandBus` bus interface.
- PR #2904 which fixes a small overcounting issue for interactions in the proof shape AIR.
- PR #2910 which refactors and improves the deferral proving interface in the SDK. Since this change is not small, we focused our review on the parts of the PRs that touch the in-scope files, as well as parts that could introduce soundness issues.

Lastly, after the phase 1 audit, the branches were rebased onto the new `v1.6.0` security release. Since the phase 1 commit hash was changed due to the rebase, we reviewed the diff between the phase 1 audit commits before and after the rebase. In particular, we ensured that:

- The diff between `stark-backend:develop-v2-before-v1.4` (2600062e) and `stark-backend:develop-v2` (be134de) only contains a version bump of plonky3.
- The diff between `openvm:develop-v2.0.0-beta-before-v1.6` (9709a5d) and `openvm:develop-v2.0.0-beta` (f01c912) is a subset of the diff between versions `v1.5.0` and `v1.6.0`. Note that we did not review the content of the version diff itself, but we made sure that no change was introduced during the rebase that could affect the validity of the phase 1 audit.

§02

Phase 1: STARK Backend and Recursive Verifier

This audit covered two repositories at specific commits:

- `stark-backend` (branch `develop-v2`, commit `66dee8ab`): a new implementation of the SWIRL proof system. Almost all code is new. A small number of files marked `[DIFF]` were reviewed only as a diff against the prior v1 codebase (tag `v1.3.0`); all other in-scope files were reviewed fresh without comparison to v1.
- `openvm` (branch `develop-v2.0.0-beta`, commit `c89d052d`): the diff from v1.5.0 to the new branch, excluding continuations, the deferral framework, the halo2 static verifier, and SDK updates.

The primary subjects of review were the SWIRL protocol and its soundness analysis, the native SWIRL verifier in `stark-backend`, and the new recursive verification circuit in `openvm/crates/recursion/`. The SWIRL proof system involves a new GKR-based LogUp argument, a batched constraint sumcheck, a stacked polynomial opening reduction, and the WHIR polynomial commitment scheme. The recursive verifier implements the full SWIRL verification protocol as a collection of 39 AIRs connected by buses, targeting proof composition for the OpenVM zkVM.

We include the full listing of audit files below. Files marked with a `[DIFF]` tag have been audited as a diff compared to v1.

stark-backend scope files

```
crates/stark-backend/Cargo.toml
crates/stark-backend/codec-derive/Cargo.toml
crates/stark-backend/codec-derive/src/lib.rs
crates/stark-backend/src/air_builders/symbolic/dag.rs [DIFF]
crates/stark-backend/src/air_builders/symbolic/mod.rs [DIFF]
crates/stark-backend/src/air_builders/symbolic/symbolic_expression.rs [DIFF]
crates/stark-backend/src/any_air.rs
crates/stark-backend/src/codec.rs
crates/stark-backend/src/config.rs
crates/stark-backend/src/duplex_sponge.rs
```

```
crates/stark-backend/src/engine.rs
crates/stark-backend/src/hasher.rs
crates/stark-backend/src/interaction/mod.rs [DIFF]
crates/stark-backend/src/keygen/mod.rs
crates/stark-backend/src/keygen/types.rs
crates/stark-backend/src/lib.rs
crates/stark-backend/src/poly_common.rs
crates/stark-backend/src/proof.rs
crates/stark-backend/src/soundness.rs
crates/stark-backend/src/transcript.rs
crates/stark-backend/src/verifier/batch_constraints.rs
crates/stark-backend/src/verifier/evaluator.rs
crates/stark-backend/src/verifier/fractional_sumcheck_gkr.rs
crates/stark-backend/src/verifier/mod.rs
crates/stark-backend/src/verifier/proof_shape.rs
crates/stark-backend/src/verifier/stacked_reduction.rs
crates/stark-backend/src/verifier/transcript_extractor.rs # test / Fiat-Shamir reference only
crates/stark-backend/src/verifier/whir.rs
crates/stark-sdk/Cargo.toml
crates/stark-sdk/src/config/baby_bear_bn254_poseidon2.rs
crates/stark-sdk/src/config/baby_bear_poseidon2.rs
crates/stark-sdk/src/config/log_up_params.rs [DIFF]
crates/stark-sdk/src/config/mod.rs
crates/stark-sdk/src/lib.rs
```

openvm scope files

```
Cargo.toml
crates/circuits/mod-builder/Cargo.toml
crates/circuits/primitives/Cargo.toml
crates/circuits/primitives/derive/src/lib.rs
crates/circuits/primitives/src/bitwise_op_lookup/mod.rs
crates/circuits/primitives/src/lib.rs
crates/circuits/primitives/src/range/mod.rs
crates/circuits/primitives/src/range_gate/mod.rs
crates/circuits/primitives/src/range_tuple/mod.rs
crates/circuits/primitives/src/var_range/mod.rs
crates/circuits/primitives/src/xor/lookup/mod.rs
crates/recursion/Cargo.toml
crates/recursion/build.rs
crates/recursion/derive/Cargo.toml
crates/recursion/derive/src/lib.rs
crates/recursion/src/batch_constraint/bus.rs
crates/recursion/src/batch_constraint/eq_airs/eq_3b/air.rs
crates/recursion/src/batch_constraint/eq_airs/eq_3b/mod.rs
crates/recursion/src/batch_constraint/eq_airs/eq_neg/air.rs
crates/recursion/src/batch_constraint/eq_airs/eq_neg/mod.rs
crates/recursion/src/batch_constraint/eq_airs/eq_ns/air.rs
crates/recursion/src/batch_constraint/eq_airs/eq_ns/mod.rs
crates/recursion/src/batch_constraint/eq_airs/eq_sharp_uni/air.rs
crates/recursion/src/batch_constraint/eq_airs/eq_sharp_uni/mod.rs
crates/recursion/src/batch_constraint/eq_airs/eq_uni/air.rs
crates/recursion/src/batch_constraint/eq_airs/eq_uni/mod.rs
crates/recursion/src/batch_constraint/eq_airs/mod.rs
crates/recursion/src/batch_constraint/expr_eval/constraints_folding/air.rs
crates/recursion/src/batch_constraint/expr_eval/constraints_folding/mod.rs
crates/recursion/src/batch_constraint/expr_eval/interactions_folding/air.rs
crates/recursion/src/batch_constraint/expr_eval/interactions_folding/mod.rs
crates/recursion/src/batch_constraint/expr_eval/mod.rs
```

```
crates/recursion/src/batch_constraint/expr_eval/symbolic_expression/air.rs
crates/recursion/src/batch_constraint/expr_eval/symbolic_expression/mod.rs
crates/recursion/src/batch_constraint/expression_claim/air.rs
crates/recursion/src/batch_constraint/expression_claim/mod.rs
crates/recursion/src/batch_constraint/fractions_folder/air.rs
crates/recursion/src/batch_constraint/fractions_folder/mod.rs
crates/recursion/src/batch_constraint/mod.rs
crates/recursion/src/batch_constraint/sumcheck/mod.rs
crates/recursion/src/batch_constraint/sumcheck/multilinear/air.rs
crates/recursion/src/batch_constraint/sumcheck/multilinear/mod.rs
crates/recursion/src/batch_constraint/sumcheck/univariate/air.rs
crates/recursion/src/batch_constraint/sumcheck/univariate/mod.rs
crates/recursion/src/bus.rs
crates/recursion/src/gkr/bus.rs
crates/recursion/src/gkr/input/air.rs
crates/recursion/src/gkr/input/mod.rs
crates/recursion/src/gkr/layer/air.rs
crates/recursion/src/gkr/layer/mod.rs
crates/recursion/src/gkr/mod.rs
crates/recursion/src/gkr/sumcheck/air.rs
crates/recursion/src/gkr/sumcheck/mod.rs
crates/recursion/src/gkr/xi_sampler/air.rs
crates/recursion/src/gkr/xi_sampler/mod.rs
crates/recursion/src/lib.rs
crates/recursion/src/primitives/bus.rs
crates/recursion/src/primitives/exp_bits_len/air.rs
crates/recursion/src/primitives/exp_bits_len/mod.rs
crates/recursion/src/primitives/mod.rs
crates/recursion/src/primitives/pow/air.rs
crates/recursion/src/primitives/pow/mod.rs
crates/recursion/src/primitives/range/air.rs
crates/recursion/src/primitives/range/mod.rs
crates/recursion/src/proof_shape/bus.rs
crates/recursion/src/proof_shape/mod.rs
crates/recursion/src/proof_shape/proof_shape/air.rs
crates/recursion/src/proof_shape/proof_shape/mod.rs
crates/recursion/src/proof_shape/pvs/air.rs
crates/recursion/src/proof_shape/pvs/mod.rs
crates/recursion/src/stacking/bus.rs
crates/recursion/src/stacking/claims/air.rs
crates/recursion/src/stacking/claims/mod.rs
crates/recursion/src/stacking/eq_base/air.rs
crates/recursion/src/stacking/eq_base/mod.rs
crates/recursion/src/stacking/eq_bits/air.rs
crates/recursion/src/stacking/eq_bits/mod.rs
crates/recursion/src/stacking/mod.rs
crates/recursion/src/stacking/opening/air.rs
crates/recursion/src/stacking/opening/mod.rs
crates/recursion/src/stacking/sumcheck/air.rs
crates/recursion/src/stacking/sumcheck/mod.rs
crates/recursion/src/stacking/univariate/air.rs
crates/recursion/src/stacking/univariate/mod.rs
crates/recursion/src/stacking/utils.rs
crates/recursion/src/subairs/mod.rs
crates/recursion/src/subairs/nested_for_loop/air.rs
crates/recursion/src/subairs/nested_for_loop/mod.rs
crates/recursion/src/subairs/proof_idx/air.rs
crates/recursion/src/subairs/proof_idx/mod.rs
crates/recursion/src/system/dummy.rs
crates/recursion/src/system/frame.rs
```

```
crates/recursion/src/system/mod.rs
crates/recursion/src/tracegen.rs
crates/recursion/src/transcript/merkle_verify/air.rs
crates/recursion/src/transcript/merkle_verify/mod.rs
crates/recursion/src/transcript/mod.rs
crates/recursion/src/transcript/poseidon2.rs
crates/recursion/src/transcript/transcript/air.rs
crates/recursion/src/transcript/transcript/mod.rs
crates/recursion/src/utils.rs
crates/recursion/src/whir/bus.rs
crates/recursion/src/whir/final_poly_mle_eval/air.rs
crates/recursion/src/whir/final_poly_mle_eval/mod.rs
crates/recursion/src/whir/final_poly_query_eval/air.rs
crates/recursion/src/whir/final_poly_query_eval/mod.rs
crates/recursion/src/whir/folding/air.rs
crates/recursion/src/whir/folding/mod.rs
crates/recursion/src/whir/initial_opened_values/air.rs
crates/recursion/src/whir/initial_opened_values/mod.rs
crates/recursion/src/whir/mod.rs
crates/recursion/src/whir/non_initial_opened_values/air.rs
crates/recursion/src/whir/non_initial_opened_values/mod.rs
crates/recursion/src/whir/query/air.rs
crates/recursion/src/whir/query/mod.rs
crates/recursion/src/whir/sumcheck/air.rs
crates/recursion/src/whir/sumcheck/mod.rs
crates/recursion/src/whir/whir_round/air.rs
crates/recursion/src/whir/whir_round/mod.rs
crates/toolchain/instructions/src/lib.rs # rename NATIVE_AS → DEFERRAL_AS and remove PUBLISH
crates/toolchain/openvm/src/io/mod.rs
crates/toolchain/openvm/src/io/read.rs
crates/toolchain/openvm/src/pal_abi.rs
crates/toolchain/transpiler/src/extension.rs
crates/toolchain/transpiler/src/lib.rs
crates/toolchain/transpiler/src/transpiler.rs
crates/vm/Cargo.toml
crates/vm/derive/src/lib.rs
crates/vm/src/arch/config.rs
crates/vm/src/arch/extensions.rs # delete PUBLIC_VALUES_AIR_ID
crates/vm/src/arch/integration_api.rs
crates/vm/src/arch/mod.rs
crates/vm/src/arch/vm.rs
crates/vm/src/lib.rs
crates/vm/src/system/connector/mod.rs
crates/vm/src/system/memory/merkle/public_values.rs
crates/vm/src/system/mod.rs
crates/vm/src/utils/stark_utils.rs
extensions/algebra/moduli-macros/src/lib.rs # hint_buffer
extensions/rv32-adapters/Cargo.toml
extensions/rv32im/circuit/Cargo.toml
extensions/rv32im/circuit/src/hintstore/mod.rs
extensions/rv32im/guest/src/io.rs
extensions/rv32im/guest/src/lib.rs
guest-libs/k256/Cargo.toml
guest-libs/p256/Cargo.toml
guest-libs/pairing/src/bls12_381/pairing.rs
guest-libs/pairing/src/bn254/pairing.rs
```

Threat Model

During the audit, we considered the following threat models for the components under review: we are mainly protecting against a malicious prover who controls the proof and may submit any byte sequence that makes the verifier accept it. The verifier operates on a pre-processed circuit whose verification key is assumed to have been computed honestly, but we do not make assumptions on the circuit itself: the proof system must be sound even if the circuit is adversarially crafted, as long as the verification key is honestly computed from it. For the recursion circuit, we also considered a similar threat model, but the malicious prover can input the full recursive circuit witness, which includes all values for all AIRs and all interactions.

Overview of SWIRL

In this section we give a high-level overview of the SWIRL proof system. We focus on the main design ideas and the overall structure of the protocol; for more technical details, we refer to the SWIRL paper.

Front-end: AIRs with interactions

The front-end of OpenVM uses the **AIR with interactions** framework, an extension of the algebraic intermediate representation (AIR) designed to support constraints that span multiple tables.

AIRs as variable-length tables. An AIR defines a constraint system over a *trace matrix*: a table of field elements with a fixed number of columns (the *width*) and a number of rows (the *trace length*) that must be a power of 2. The height is not fixed by the AIR itself: it is chosen freely by the prover at proving time, so trace matrices of different heights can all satisfy the same AIR. A circuit may define many AIRs, and the prover is allowed to omit any AIR it does not use. However, the circuit can mark individual AIRs as *required* (via `is_required` in the verification key), forcing the prover to supply a trace matrix for them.

Constraint selectors. Constraints are polynomials over the values of the current row and the "next" row of the trace. Each constraint is paired with a *selector* that restricts on which rows it is enforced:

- **All**: the constraint must hold on every row. The "next row" of the last row wraps around cyclically to the first row.
- **First**: the constraint is only enforced on the first row.
- **Last**: the constraint is only enforced on the last row (using the first row as the "next row", i.e., the cyclic wrap-around pair).
- **Transition**: the constraint must hold on every row *except* the last one. Unlike `All`, there is no wrap-around, making this suitable for state transition constraints that should not hold across the table boundary.

Buses and interactions. A single AIR can only impose constraints *within* its own table. To express relationships *across* different tables, the framework introduces *buses* and *interactions*. A bus is an abstract channel identified by a nonzero field element. Any AIR can declare one or more

interactions on a bus, each specified by a pair of polynomials `(message, multiplicity)` over the symbolic variables $(x_1, \ldots, x_w, y_1, \ldots, y_w)$, where x_i represents the i-th column of the current row and y_i represents the i-th column of the next row (cyclically):

- The **message** is a vector of polynomials $\sigma = (\sigma_1, \ldots, \sigma_s)$. Evaluated on a concrete row i, it produces the field element tuple $\sigma(\mathbf{T}_i, \mathbf{T}_{\text{next}(i)}) \in \mathbb{F}^s$ that this row sends on the bus.
- The **multiplicity** is a single polynomial m. Evaluated on row i, it produces the field element $m(\mathbf{T}_i, \mathbf{T}_{\text{next}(i)}) \in \mathbb{F}$ that weights this row's contribution.

Every row of every present trace matrix thus contributes one message-multiplicity pair to the bus. A bus is *balanced* if, for every possible message tuple $\tau \in \mathbb{F}^s$, the sum of multiplicities over all rows (across all present AIRs) whose message evaluates to exactly τ is zero as a field element. A circuit is satisfied only when all its buses are balanced.

This balancing condition is expressive enough to encode the two main cross-table constraint patterns used throughout OpenVM, which we discuss in later sections:

- **Lookup buses**: expose two operations. `add_key` declares a value as belonging to the table, contributing multiplicity $-n$ where n is left unconstrained by the bus. `lookup_key` asserts that a value belongs to the table, contributing multiplicity $+1$. Bus balancing then forces n to equal the actual number of lookups for that key, guaranteeing every queried value was declared in the table.
- **Permutation buses**: expose `send` (multiplicity $+1$) and `receive` (multiplicity -1) operations, so balancing enforces that the multiset of sent messages equals the multiset of received messages.

Because balancing is checked modulo the field characteristic p, a sum of multiplicities that is a nonzero multiple of p would incorrectly appear balanced. To ensure that the lookup and permutation semantics hold, each bus type assigns a *count weight* to its operations: for lookup buses, each `lookup_key` call contributes weight 1 while `add_key` contributes weight 0; for permutation buses, both `send` and `receive` contribute weight 1. The total weighted count of interactions across all rows and all present AIRs must remain strictly below p to avoid a wrap-around.

Proof system overview

At a high level, the SWIRL proof system proceeds in four stages, each reducing the problem to a simpler claim:

1. **Interactions via LogUp+GKR.** Bus balancing is rephrased as the vanishing of a fractional sum over all rows of all present traces. GKR reduces this to a claim about two polynomials p and q (the stacked numerator and denominator of the LogUp sum) evaluated at a random point. That claim is in turn reduced to evaluations of the message and multiplicity polynomials of each present AIR trace via a batched sumcheck.

2. **Constraints via ZeroCheck.** The AIR constraints (that certain polynomials vanish on the trace domain) are batched and reduced to evaluations of the trace column polynomials at a random point via a batched sumcheck with a ZeroCheck argument.

3. **Stacked opening reduction.** Steps (1) and (2) are arranged to reduce to evaluations of the same trace columns at the same random point. Those evaluations are then reduced to evaluations of the relevant stacked column polynomials at a single shared random point, using the injection from AIR column positions to stacked column positions and a batched sumcheck.

4. **WHIR opening proofs.** The stacked column evaluations are proved using WHIR as a multilinear polynomial commitment scheme.

LogUp+GKR for interactions

LogUp reformulation. Bus balancing requires that, for every message τ, the sum of multiplicities across all rows and all present AIRs whose message equals τ is zero. LogUp allows this check to be expressed using a single algebraic equality: for random challenges α, β sampled by the verifier, the fractional sum

$$\sum_{T,(\widehat{\sigma},\widehat{m},b)\in I_T} \sum_{\mathbf{z}\in\mathbb{D}_{n_T}} \frac{\widehat{m}(\mathbf{T}(\mathbf{z}), \mathbf{T}_{\mathrm{rot}}(\mathbf{z}))}{\alpha + h_\beta(\widehat{\sigma}(\mathbf{T}(\mathbf{z}), \mathbf{T}_{\mathrm{rot}}(\mathbf{z}))\|b)}$$

must equal zero. Here h_β is the linear hash defined by

$$h_\beta(\widehat{\sigma}\|b) := \beta^{\mathrm{len}(\widehat{\sigma})} \cdot b + \sum_{j=1}^{\mathrm{len}(\widehat{\sigma})} \beta^{j-1}\widehat{\sigma}_j$$

which evaluates the polynomial $bX^{\mathrm{len}(\widehat{\sigma})} + \widehat{\sigma}_{\mathrm{len}}X^{\mathrm{len}-1} + \ldots + \widehat{\sigma}_1$ at $X = \beta$. The multi-bus check is reduced to a single-bus check by concatenating each message with its bus index before hashing: since different buses have different values of b, the term $\beta^{\mathrm{len}(\widehat{\sigma})}b$ distinguishes their hashes with high probability over the random choice of β. If this sum is zero, then all buses are balanced with high probability over the choice of α and β.

> ***Remark.*** *The bus index b must be nonzero. If $b = 0$, the leading term $\beta^{\mathrm{len}(\widehat{\sigma})} \cdot 0$ vanishes, so the hash of a message $(\sigma_1, \ldots, \sigma_s)$ on bus 0 equals $h_\beta((\sigma_1, \ldots, \sigma_{s-1})\|\sigma_s)$, i.e., the same value as a shorter message $(\sigma_1, \ldots, \sigma_{s-1})$ on the bus indexed by σ_s. Domain separation between buses therefore breaks down: an adversary could craft interactions on a zero-indexed bus that collide with interactions on a different, legitimately indexed bus, defeating the balancing check.*

GKR reduction. Computing this sum directly is expensive for the verifier, since it involves one fraction per row per interaction across all present AIRs. The GKR protocol computes it as a layered arithmetic circuit: each layer combines adjacent pairs of fractions using the identity $\frac{p_1}{q_1} + \frac{p_2}{q_2} = \frac{p_1 q_2 + p_2 q_1}{q_1 q_2}$, halving the number of terms at each step. The circuit output is the total fractional sum, which must equal zero.

GKR reduces the claim that the circuit output is zero to a claim about the circuit's input layer by applying a sumcheck at each layer, working backwards from output to input. At the end of this reduction the verifier holds a claim about two polynomials at the input layer: a numerator polynomial p and a denominator polynomial q, evaluated at a random point $\boldsymbol{\xi}$.

Stacking interactions. All interactions across all present AIRs and all rows are embedded into a single domain using an injection j that maps each (AIR, row, interaction) triple into a position in a single boolean hypercube $\mathbb{H}_{\ell+n_{\mathrm{LogUp}}}$. The polynomials p and q are then defined as functions on this joint domain, zero-padded outside the image of j. This means the GKR circuit operates on a single pair (p, q) rather than one pair per AIR, keeping the circuit uniform and allowing the GKR sumchecks to be batched.

Reduction to trace evaluations. The claims about $p(\boldsymbol{\xi})$ and $q(\boldsymbol{\xi})$ expand, via the definition of j, into sums over each present trace matrix of terms involving the message and multiplicity polynomials evaluated on rows of $\mathbf{T}$. A batched sumcheck over all present trace matrices reduces these to evaluations $\widehat{\mathbf{T}}(\mathbf{r})$ and $\widehat{\mathbf{T}}_{\mathrm{rot}}(\mathbf{r})$ at a shared random point $\mathbf{r}$, which are the same column evaluations needed by ZeroCheck in the next step.

ZeroCheck for AIR constraints

Each AIR defines constraint polynomials that must vanish on every applicable row of the trace. For a trace matrix $\mathbf{T}$ satisfying an AIR A, each constraint $(C, S) \in A$ must satisfy $C(\mathbf{T}(\mathbf{z}), \mathbf{T}_{\mathrm{rot}}(\mathbf{z})) \cdot S(\mathbf{z}) = 0$ for all $\mathbf{z}$ in the trace domain. The selector S restricts which rows are checked (All, First, Last, or Transition).

All constraints for the present AIRs are algebraically batched into a single polynomial using a random challenge λ, and their simultaneous vanishing is checked via a batched sumcheck (ZeroCheck). This sumcheck reduces to an evaluation of the batched constraint polynomial at a single random point $\mathbf{r}$, which in turn requires knowing the values of the trace columns $\widehat{\mathbf{T}}(\mathbf{r})$ and $\widehat{\mathbf{T}}_{\mathrm{rot}}(\mathbf{r})$ at that point.

Crucially, $\mathbf{r}$ is set to the same point $\boldsymbol{\xi}$ that was produced at the end of the LogUp+GKR reduction. The ZeroCheck sumcheck and the LogUp input-layer sumcheck (which computes $p(\boldsymbol{\xi})$ and $q(\boldsymbol{\xi})$ from the message and multiplicity polynomials) both reduce to evaluations of the trace columns at $\boldsymbol{\xi}$. These two batched sumchecks are merged into one, so a single evaluation of each trace column at $\boldsymbol{\xi}$ covers both the constraint check and the interaction check simultaneously.

Selectors as multiplicative masks. Each constraint C is paired with a selector polynomial $S{:}\mathbb{D}_{n_T} \to \{0, 1\}$ that encodes which rows must satisfy $C = 0$. The condition is written as $C(\mathbf{T}(\mathbf{z}), \mathbf{T}_{\mathrm{rot}}(\mathbf{z})) \cdot S(\mathbf{z}) = 0$ for all $\mathbf{z}$. The selector evaluates to 1 on rows where the constraint applies and to 0 on rows where it does not, so multiplying by S masks out exempt rows. The four selector types have explicit prismalinear extensions (Section 3.3.1 of the SWIRL paper):

- $\widehat{\mathrm{All}} = 1$: every row is selected.
- $\widehat{\mathrm{First}}$: evaluates to 1 only on the first row (the element $\omega_D^0 \in D$ paired with $\mathbf{0} \in \mathbb{H}_n$).

- $\widehat{\mathrm{Last}}$: evaluates to 1 only on the last row.
- $\widehat{\mathrm{Transition}} = 1 - \widehat{\mathrm{Last}}$: every row except the last.

This is analogous in spirit to the traditional STARK approach of dividing a constraint polynomial by a vanishing polynomial that is zero on exempt rows, but the selector-multiplication approach avoids division and works directly with multilinear polynomials. Checking that $C \cdot S$ vanishes everywhere on the domain is handled by the ZeroCheck.

The rotation kernel. Constraint polynomials refer to the current row and the "next" row. In the $\mathbb{D}_n = D \times \mathbb{H}_n$ domain, rows are ordered lexicographically by the D coordinate first, then the binary coordinates. The "next" row is defined by the rotation map

$$\mathrm{rot}: \mathbb{D}_n \to \mathbb{D}_n, \quad \mathbf{z} \mapsto \mathrm{ord}_{\ell,n}((\mathrm{ord}_{\ell,n}^{-1}(\mathbf{z}) + 1) \bmod 2^{\ell+n})$$

which advances to the next position in this lexicographic ordering, wrapping around cyclically (Section 2.5 of the SWIRL paper). Given a column polynomial $\hat{t}$, the rotated column $\hat{t}_{\mathrm{rot}}(\mathbf{z}) := \hat{t}(\mathrm{rot}(\mathbf{z}))$ is computed via the *rotation kernel* $\widehat{\kappa}_{\mathrm{rot}}$ as the convolution $\hat{t} \star \widehat{\kappa}_{\mathrm{rot}}$, which has an explicit polynomial formula in terms of equality polynomials. This lets the verifier express $\widehat{T}_{\mathrm{rot}}(r)$ algebraically and reduce both $\widehat{T}(r)$ and $\widehat{T}_{\mathrm{rot}}(r)$ to the stacked column evaluations in the next step.

Stacked commitments

A circuit may contain dozens of AIRs, but only *present* AIRs contribute trace matrices in a given proof. Each present trace matrix is additionally partitioned into preprocessed, common main, and cached columns. A naive commitment scheme would commit to each column of each one of them independently. SWIRL instead applies the same stacking construction per commitment: all common main partitions from the present AIRs are stacked into one shared matrix, while each nonempty preprocessed partition and each nonempty cached partition is stacked and committed separately. We now describe the stacking construction in more detail.

The stacked matrix. In the SWIRL paper, $\mathbb{D}_n$ denotes the hyperprism $D_n = D \times \mathbb{H}_n$ for $n \geq 0$, where $|D| = 2^\ell$ is the univariate skip domain. The domain $\mathbb{D}_n$ has size $2^{\ell+n}$, and the extended definition for negative n preserves the same formula, for $n \geq -\ell$. The stacked matrix Q is a map

$$Q : \mathbb{D}_{n_{\mathrm{stack}}} \times [w] \to \mathbb{F}$$

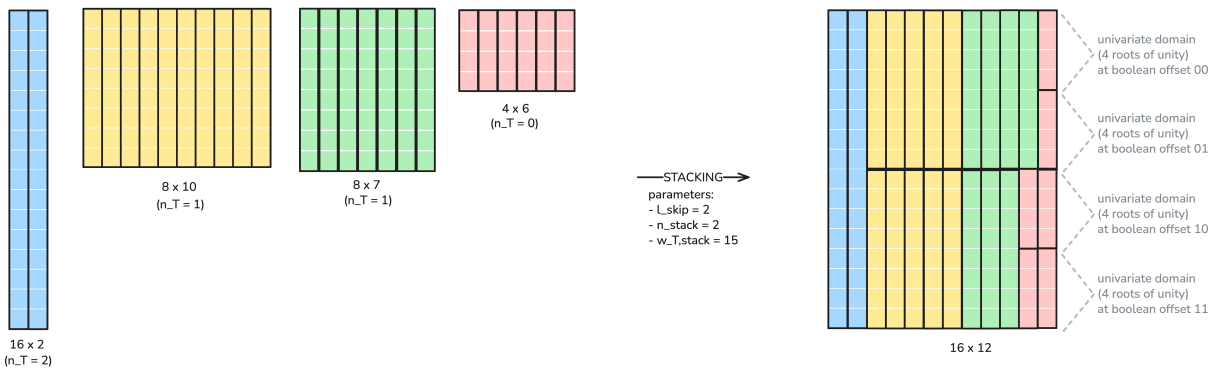
for some width w that depends on n_{stack}, and on the heights and widths of the stacked trace matrices.

Each stacked column has height $2^{\ell+n_{\mathrm{stack}}}$, where n_{stack} is a global parameter chosen so that every included trace dimension satisfies $n_T \leq n_{\mathrm{stack}}$. For an AIR column with $n_T \geq 0$, the column height is $2^{\ell+n_T}$, so it fits within the stacked height. Multiple AIR columns can be packed end-to-end within a single stacked column: for example, two AIR columns of height $2^{\ell+n_{\mathrm{stack}}-1}$ fit exactly into one stacked column of height $2^{\ell+n_{\mathrm{stack}}}$.

Packing algorithm. The allocation of AIR columns into stacked columns follows a greedy bin-packing strategy that exploits the power-of-2 structure of trace heights. For packing, let $\tilde{n}_T = \max(n_T, 0)$, since traces with $n_T < 0$ are first lifted into one full D-slot as described below.

1. Sort all AIR columns by height from tallest to shortest.
2. Assign each column to the current stacked column, advancing to the next stacked column only when the current one is full.

Because every height is a power of 2 and $\tilde{n}_T \leq n_{\text{stack}}$, this greedy strategy is gap-free except possibly at the end of the final stacked column. After sorting from tallest to shortest, the remaining capacity in the current stacked column is always a multiple of the next column height. The next column therefore either fills the remaining capacity exactly or consumes one divisor-sized slot inside it. The only possible waste is at the very end of the last stacked column, where the remaining rows are padded with zeros.



Stacked layout. For each AIR column, the packing algorithm records a *stacked layout* entry consisting of three values:

- the index j of the stacked column it was placed into,
- the row offset $\mathbf{b} \in \{0,1\}^{n_{\text{stack}} - \tilde{n}_T}$ encoding where within that stacked column the AIR column begins,
- the packed height $2^{\ell + \tilde{n}_T}$.

Because the prover chooses which AIRs are present and the height of each trace matrix at proving time, the stacked layout cannot be fixed in the verification key. Instead, the prover declares the heights of the present AIRs as part of the proof, and the verifier recomputes the stacked layout for each commitment from scratch by running the same deterministic packing algorithm on the declared heights and the relevant column partitions. The result is an injection ι from AIR column positions to stacked column positions that both parties agree on for that specific proof.

Commitment. Each stacked matrix is committed by applying Reed-Solomon encoding to its columns and building a Merkle tree over the resulting codeword matrix (as in WHIR). The resulting Merkle root is the stacked polynomial commitment for that matrix. The common main columns

share one such commitment, while each nonempty cached or preprocessed partition has its own stacked commitment.

Opening reduction. Whenever the verifier needs an evaluation of an AIR column T_j at a random point $\mathbf{r}$, this is reduced to an evaluation of the corresponding stacked column $q_{j'}$ at a related point, using a sumcheck. For $n_T \geq 0$, the reduction uses the injection ι to express the AIR column evaluation as a sum over the stacked column:

$$\hat{t}_j(\mathbf{r}) = \sum_{\mathbf{z}' \in \mathbb{D}_{n_{\text{stack}}}} \hat{q}_{j'}(\mathbf{z}') \cdot \text{eq}(\mathbf{r}, \mathbf{z}'_{\leq n_T}) \cdot \text{eq}(\mathbf{b}, \mathbf{z}'_{>n_T})$$

where $\mathbf{z}'_{\leq n_T}$ includes the D coordinate and the first n_T boolean coordinates, and the second equality polynomial pins the rows to the AIR column's slot within the stacked column. For $n_T < 0$, the full protocol inserts the in_{D,n_T} correction factor described below. All such reductions across the relevant trace columns and commitments are batched into a single sumcheck (Protocol 3.6.1 of the SWIRL paper), ultimately collapsing to evaluations of the stacked polynomials at a single shared random point, which are then proved using WHIR as a batched PCS.

Adjustments for univariate skip

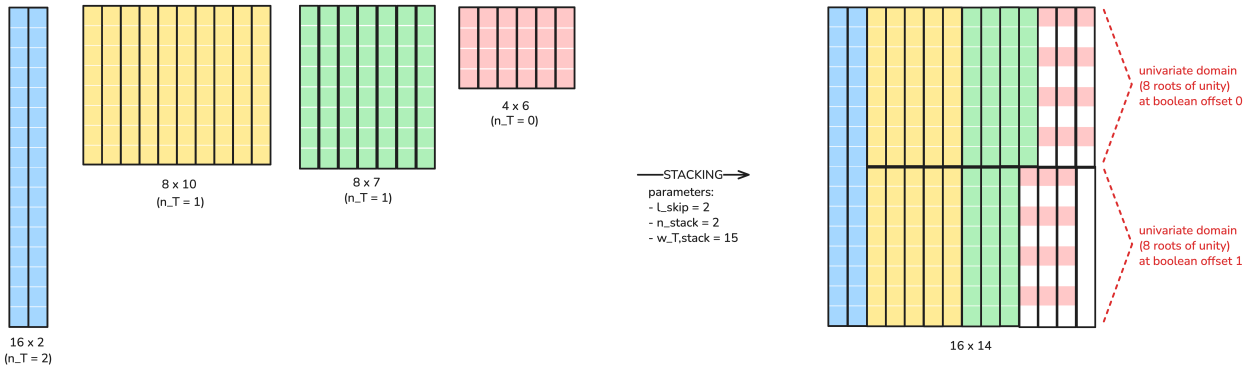
The stacked discussion above uses SWIRL's hyperprism notation. The key point for the univariate skip is that $\mathbb{D}_n = D \times \mathbb{H}_n$ has one smooth multiplicative coordinate and n boolean coordinates. The prismalinear extension $\hat{t}$ of a column $t{:}\mathbb{D}_n \to \mathbb{F}$ has degree $< 2^\ell$ in the Z variable (the D coordinate) and is multilinear in the remaining n boolean variables $X_1, \ldots, X_n$.

The univariate skip exploits this structure in sumchecks over $\mathbb{D}_n$: instead of running ℓ separate binary sumcheck rounds for the D coordinate, the prover handles all of D in a single round by sending a univariate polynomial of degree $< 2^\ell$, from which the verifier samples a single challenge $r_0 \in \mathbb{F}_{\text{ext}}$. The remaining n binary rounds proceed as normal. This saves $\ell - 1$ prover rounds at the cost of the verifier performing a larger interpolation and a small soundness loss.

Contrast with p, q, and the stacked matrix. The LogUp polynomials p and q are defined over the full boolean hypercube $\mathbb{H}_{\ell + n_{\text{LogUp}}}$, with all $\ell + n_{\text{LogUp}}$ variables binary. They do not use the D coordinate at all, but is instead encoded in the first ℓ boolean variables. By contrast, stacked columns are functions on $\mathbb{D}_{n_{\text{stack}}}$ and have height $2^{\ell + n_{\text{stack}}}$. When passed to WHIR, each stacked column is also viewed through the corresponding multilinear representation over $\mathbb{H}_{\ell + n_{\text{stack}}}$, which has the same number of points.

Short columns and lifting. For an AIR with very few rows, the hypercube dimension n_T may be negative. Specifically, if $n_T = -i$ for some $0 \leq i \leq \ell$, then the column domain is $\mathbb{D}_{n_T} = D^{(2^i)}$, a subgroup of D of order $2^{\ell - i}$, and the prismalinear extension $\hat{t}$ is a univariate polynomial of degree $< 2^{\ell - i}$. To bring such columns into the uniform framework, SWIRL defines the *lift* $\tilde{t}(Z) := \hat{t}(Z^{2^i})$, a degree $< 2^\ell$ polynomial over all of D. The lift repeats each column value 2^i times across D with stride 2^i.

The minimum concrete column height handled by the proof system is 2^ℓ, regardless of the original trace height. This is mainly due to the fact that the first univariate sumcheck round has to be executed, and cannot be partially skipped. For packing into the stacked matrix, a column with $n_T < 0$ is treated as having effective stacking dimension $\tilde{n}_T = \max(n_T, 0) = 0$, meaning it occupies one full D-slot of 2^ℓ rows in the stacked matrix.



Correction factors. Because the lift oversamples the actual column values, two correction factors appear in the protocol for $n_T < 0$:

- *In LogUp*: the fractional sum for a trace of dimension $n_T < 0$ is computed by summing over all of D using the lifted polynomials $\tilde{T}$ and $\tilde{T}_{\text{rot}}$. Since each actual row appears 2^{-n_T} times in D, the sum overcounts by that factor. A weight of 2^{n_T} is applied to the entire trace's contribution to the fractional sum to correct for this (Equation 3.11 of the SWIRL paper).
- *In stacked opening reduction*: when reducing the evaluation $\hat{t}_j(r_0, \mathbf{r})$ to a stacked column evaluation, the factor $\text{in}_{D,n_T}(Z)$ is inserted. For $n_T < 0$ this polynomial equals $2^{n_T} \cdot \frac{Z^{2^\ell}-1}{Z^{2^{\ell+n_T}}-1}$, which is 1 on the subgroup $D^{(2^{-n_T})}$ and 0 elsewhere in D, weighted by 2^{n_T} to account for the lift. For $n_T \geq 0$ it is identically 1.

Native SWIRL verifier

The native verifier is the Rust implementation of the SWIRL verifier in `stark-backend`.

Plonky3 AIR interface

The SWIRL proof system is built on top of the standard Plonky3 AIR interface, making it compatible with any circuit that implements Plonky3's `Air` trait. The interface centers on two traits: `BaseAir<F>`, which declares the trace width and an optional preprocessed (constant) trace, and `Air<AB: AirBuilder>`, which exposes a single `eval` method where the circuit author writes algebraic constraints against the trace columns using a builder pattern. The builder provides selectors (`is_first_row`, `is_last_row`, `is_transition`) and the assertion method `assert_zero`, from which all constraint types are derived.

OpenVM extends this base interface with two additions. First, `PartitionedBaseAir<F>` splits the main trace into a set of cached partitions and a common partition, allowing different parts of the trace to be committed at different points in the protocol. Second, `InteractionBuilder` augments the builder with a `push_interaction` method, through which a circuit registers sends and receives on named buses; these interactions are compiled into the LogUp-GKR multiset argument described above. Any circuit that implements `Air`, `BaseAirWithPublicValues`, and `PartitionedBaseAir` satisfies the `AnyAir` trait object bound used internally by SWIRL, and can be passed directly to the prover without any additional glue code.

Interaction builders

Interactions between trace rows are expressed through typed buses. A bus is simply a `u16` index that namespaces a set of interactions; any number of AIRs, or even different rows within the same AIR, communicate by sending and receiving on a shared bus index. Self-communication within a single AIR is in fact the common case: for example, an AIR may send a value on one row and receive it on another to enforce a range check or a memory consistency constraint, all without involving a second AIR. OpenVM provides two bus abstractions:

PermutationCheckBus enforces a multiset equality between its senders and receivers: the multiset of messages sent (with multiplicity $+1$) must equal the multiset of messages received (with multiplicity -1), so the signed sum over all rows and all participating AIRs is zero. A circuit calls `bus.send(builder, message, enabled)` to contribute a message with boolean multiplicity `enabled` and `bus.receive(builder, message, enabled)` to consume one with multiplicity `-enabled`. The two-sided nature means every message that is sent must be received somewhere, and vice versa.

LookupBus enforces a subset relation: a querying AIR asserts that its values appear in a fixed table. The table AIR calls `add_key_with_lookups` to register a key with a negative multiplicity equal to the total number of lookups for that key, and the querying AIRs call `lookup_key` to send each query with multiplicity `+1`. Soundness only requires bounding the total number of queries (the positive side); the table side is unbounded by design.

Both bus types ultimately lower to `push_interaction`, which records an `Interaction { message, count, bus_index, count_weight }`. The `count` expression is the signed multiplicity for that row. The `count_weight` field is a $\mathbb{u32}$ that controls a separate verifier-side linear constraint on trace heights: at key generation time, for each bus the verifier accumulates $\sum_i \texttt{count\_weight}_i \cdot h_i$ across all AIRs i with height h_i, and checks that this sum does not exceed the field characteristic p. This ensures the total number of interactions of the bounded kind stays below p, which is a necessary condition for soundness of the LogUp argument (a wraparound in the count would cancel to zero without representing a genuine multiset equality). For the permutation bus, both senders and receivers carry `count_weight = 1` because both sides must be bounded. For the lookup bus, table keys carry `count_weight = 0` (no bound needed on the negative side) while queries carry `count_weight = 1`.

Fiat-Shamir security

The SWIRL proof system is made non-interactive via the Fiat-Shamir transform: all verifier challenges are replaced by outputs of a duplex sponge transcript that absorbs proof elements as they are produced. We verified that every value on which a subsequent challenge depends is absorbed into the transcript before that challenge is sampled, ruling out the class of "weak Fiat-Shamir" attacks where a challenge is sampled before all relevant commitments or claims are bound to the transcript.

Sponge collisions. Duplex sponges are inherently susceptible to trivial collisions between transcripts that absorb different sequences of values yet arrive at the same internal state:

- In XOR mode, absorbing `0` is a no-op: the state is unchanged because $s \oplus 0 = s$.
- In overwrite mode, absorbing the value that is already present in the rate slot is a no-op: the state is unchanged because overwriting s with s leaves s.

In both cases, there exist pairs of distinct absorption sequences that produce identical sponge states and therefore identical challenges. For instance, in XOR mode:

```
[absorb(42), absorb(0), sample()]
[absorb(42),            sample()]
```

yield the same challenge.

This raises a concern in a protocol where the prover dynamically controls how many values it absorbs, for example because AIR heights are prover-chosen and some AIRs are optional. A malicious prover could attempt to shift the transcript by absorbing fewer elements, or by inserting absorptions that are no-ops, in order to obtain a challenge it could not otherwise produce.

In the SWIRL verifier this is handled by the fact that the transcript encodes the prover's choices in a self-describing way that admits a unique sequential decoding. Every degree of prover freedom is made explicit in the transcript before the values that depend on that freedom are absorbed. For example, for each AIR the prover first absorbs a presence bit (1 if the AIR is used, 0 if omitted); only if that bit is 1 does the prover then send the relevant trace commitments. The verifier reads these bits in order and knows, at each step, exactly how many and which field elements to expect next. There is therefore exactly one way to decode the field elements stream into a sequence of prover choices and absorbed values. A prover that tries to shift the transcript, say by skipping a commitment or inserting a no-op absorption, produces a byte stream whose presence bits and subsequent values are read by the verifier in a different order than intended, yielding a completely different set of commitments and claims that will fail verification.

Recursive verification circuit

Multiproof verification

The recursion circuit is parameterized by a compile-time constant `MAX_NUM_PROOFS` and is designed to verify up to that many proofs of the same child verification key simultaneously. Supporting multiple proofs in a single circuit is important for recursion efficiency: rather than wrapping each inner proof in its own separate recursion step, a single recursion circuit can absorb a batch of inner proofs at once, amortizing the fixed overhead of the outer circuit across all of them.

The `proof_idx` index serves as a routing key throughout the circuit. All inter-module buses are *per-proof*: every message sent or received on a bus carries `proof_idx` as its first field, so the multiset-equality or lookup argument only matches messages that belong to the same proof. This is realized by two families of typed bus macros in `bus.rs`: `define_typed_per_proof_lookup_bus!` and `define_typed_per_proof_permutation_bus!`, which both prepend `proof_idx` to every message automatically. As a result, each protocol module (GKR, constraint batching, stacking, WHIR, transcript) operates independently on each proof's data, with the bus arguments enforcing consistency within but not across different proofs.

Recursion circuit architecture

The recursive circuit is composed of 39 AIRs organized into five modules plus shared primitives, all wired together through a rich network of buses. The modules mirror the sequential stages of the SWIRL verifier.

Transcript bus. The backbone of the circuit is the `TranscriptBus`, a per-proof permutation bus that encodes the Fiat-Shamir sponge operations. The `TranscriptAir` (in the Transcript module) is the sole *sender* on this bus: it replays the complete transcript log, emitting one message `(tidx, value, is_sample)` per field element, where `is_sample = 0` denotes an observe (absorb) and `is_sample = 1` denotes a sample (squeeze). Every other AIR in the circuit is a *receiver*: whenever a module needs to absorb a commitment or squeeze a challenge, it receives the corresponding `TranscriptBus` message. Bus balancing then ensures that the entire transcript is consistent and that every observe/sample performed by any AIR was actually executed in the correct order by the Poseidon2 sponge.

Module handoff buses. The five protocol modules are chained together by a sequence of four permutation buses, each carrying exactly one message per proof: `GkrModuleBus`, `BatchConstraintModuleBus`, `StackingModuleBus`, and `WhirModuleBus`. Each bus carries the output of one stage to the input of the next, together with the current transcript index `tidx` so the receiving module knows where in the transcript to continue.

Forced computation via bus balancing. The design enforces that the full verification pipeline must execute. The `ProofShape` module starts by sending messages on its output buses: those messages must be received by `GKR`, which in turn must send on `BatchConstraintModuleBus`, and so on down the chain. The `WhirModuleBus` is the terminal bus: it is sent by the Stacking module but received by the WHIR module, and the WHIR module itself sends no further handoff messages. If

any module fails to produce its output messages, the bus imbalance propagates and the overall circuit has no satisfying witness. In this sense the circuit computes a *pull* pipeline: the WHIR module can only balance if it receives from Stacking, which can only balance if it receives from BatchConstraint, and so on, forcing every prior stage to run.

Overview of protocol modules

ProofShape module. The ProofShape module (3 AIRs: `ProofShapeAir`, `PublicValuesAir`, `RangeCheckerAir`) is the preamble of the verifier. It processes the proof's structural metadata: it verifies that each AIR's trace height is a valid power of two, observes the VK pre-hash and all trace commitments into the transcript, computes the two key dimensions used by downstream modules ($n_{\max}$, the maximum hypercube dimension across all AIRs, and n_{logup}, the number of GKR layers), and broadcasts AIR shape data on a set of lookup buses (`AirShapeBus`, `HyperdimBus`, `LiftedHeightsBus`, `CommitmentsBus`, `AirPresenceBus`) for other modules to consume.

GKR module. The GKR module (4 AIRs) receives the parameters from ProofShape via `GkrModuleBus` and then executes the GKR reduction for the LogUp argument layer by layer, verifying the sumcheck at each GKR layer. At the end of the reduction it holds two polynomial claims at the input layer (a numerator and a denominator) and forwards them to the BatchConstraint module via `BatchConstraintModuleBus`.

BatchConstraint module. The BatchConstraint module (13 AIRs) receives the GKR input-layer claims and verifies them by evaluating the constraint and interaction expressions of the child circuit at a random point. It reads the child circuit's column evaluations from the `ColumnClaimsBus` and the public values from `PublicValuesBus`, evaluates the full batched constraint/interaction polynomial using the symbolic expression DAG embedded in the cached trace, and reduces everything to a single polynomial opening claim. The StackingModuleBus signals to the Stacking module that this is complete.

Stacking module. The Stacking module (6 AIRs) receives the column opening claims from BatchConstraint and reduces them to a single batched claim on the stacked polynomial, using a univariate sumcheck followed by a multilinear sumcheck. The final batched claim, together with the mu batching challenge, is forwarded to the WHIR module via `WhirModuleBus` and `WhirMuBus`.

WHIR module. The WHIR module (8 AIRs) is the terminal stage. It receives the batched opening claim and verifies it using the WHIR polynomial commitment protocol: multiple rounds of folding sumcheck, Merkle path queries, and a final low-degree polynomial check. It has no output handoff bus; its only obligation is to balance all incoming messages, which it does by fully executing the WHIR verification.

DAG commitment. When the SymbolicExpressionAir operates in non-cached mode (i.e., when there is no pre-committed cached trace), it computes a hash of the constraint/interaction DAG row by row using an inline Poseidon2 computation (`DagCommitSubAir`). The resulting digest is exported as a public value of the SymbolicExpressionAir. This lets the calling circuit check that the recursion circuit evaluated the correct constraint DAG without having to supply a pre-committed VK.

§03

Phase 2: Continuations and Deferral

The audit covered the following components in the OpenVM repository:

- `crates/continuations/`: aggregation circuits for the recursion tree (leaf, internal-for-leaf, internal-recursive, and root layers), plus the deferral framework core under `src/circuit/deferral/`. The deferral framework is a parallel aggregation tree that links into the VM continuations tree; the continuations tree adds the constraints needed to bind the deferral side correctly.
- `extensions/deferral/`: VM-side interface to deferral circuits: a new VM opcode, the transpiler shim, the guest library, and the circuit linking deferral input/output commits to VM memory through the RV32IM memory interface.
- `crates/verify/`: host-side Rust verifier for the final aggregated VM STARK proof, intended for use cases such as the ethproofs WASM verifier.
- `guest-libs/verify-stark/{circuit, guest}/`: circuits and guest library wrapper that wrap the deferral framework to provide a `verify_stark` capability for Rust guest programs, replacing the older `guest-libs/verify_stark`.

Overview

This section provides a high-level technical overview of the audited modules.

Continuations aggregation tree

OpenVM handles long executions by splitting them into VM segments and then recursively aggregating the resulting segment proofs. Each segment proof exposes the public values needed to connect it to the next segment: the app program commitment, the initial and final program counters, the exit code, the termination flag, and the initial and final memory roots. The continuations tree verifies child proofs and folds these public values upward until a single proof represents the whole execution.

The STARK aggregation pipeline has four main layers:

- **Leaf** verifies app segment proofs.
- **Internal-for-leaf** verifies leaf proofs.
- **Internal-recursive** verifies internal-for-leaf proofs at the first recursive level, then verifies internal-recursive proofs at later levels.
- **Root** wraps one final internal-recursive proof and checks the final conditions needed by the outer verifier.

The first three layers use the same inner aggregation subcircuit. The verifier part of the circuit checks each child proof against the relevant child verifying key, while the public-value aggregation part checks that adjacent VM segments line up. In particular, a non-terminal child must suspend successfully, and its `final_pc` and `final_root` must match the next child's `initial_pc` and `initial_root`. The app program commitment is also kept constant across all valid children. The output proof re-exposes the first child's initial state and the last child's final state, so the same invariant can be checked again at the next aggregation layer.

The inner circuit also carries verifier public values that describe which verifier keys have been fixed so far. This is tracked by two flags:

LAYER	INTERNAL FLAG	RECURSION DEPTH	NEWLY EXPOSED VK COMMIT
Leaf	`0`	`0`	`app_vk_commit`
Internal-for-leaf	`1`	`0`	`leaf_vk_commit`
First internal-recursive	`2`	`1`	`internal_for_leaf_vk_commit`
Second internal-recursive	`2`	`2`	`internal_recursive_vk_commit`
`n`-th internal-recursive	`2`	`n`	`internal_recursive_vk_commit`

Each VK commit contains the cached commitment to the verifier circuit's constraint DAG together with the child verifying key pre-hash. Unused VK commits are constrained to be unset, and already exposed commits are propagated unchanged. This gives later layers enough information to know which verifier circuit was used below them without exposing the full verifying keys.

The important fixed point is the internal-recursive layer. The internal-recursive prover is constructed so that, after the first internal-recursive proof has exposed `internal_for_leaf_vk_commit`, later internal-recursive proofs can use the internal-recursive verifying key as their own child key. From that point on, the verifier circuit is stable: an internal-recursive proof can verify other internal-recursive proofs, and the tree can keep reducing an arbitrary number of child proofs until only one remains.

Deferral framework: aggregation tree

The deferral framework has its own aggregation tree, separate from the VM segment aggregation tree. Each deferral circuit proof exposes `DeferralCircuitPvs`, which are just the `input_commit` and `output_commit` for one deferred computation. The deferral aggregation tree verifies these proofs, collects the IO commitments into a Merkle root, and keeps a count of how many real deferral circuit proofs were included.

At the leaf layer, `DeferralAggPvsAir` compresses `(folded_input_commit, output_commit)` into a leaf `merkle_commit`, and the proof count is `1` for each present child. At internal layers, the children already expose `merkle_commit`s and counts, so the parent either passes through a single

child or compresses two child roots into a new parent root while adding the counts.

The AIR supports three trace shapes:

SHAPE	MEANING
One row	Wrapper node: pass the child `merkle_commit` and count through unchanged.
Two present rows	Binary aggregation node: hash the left and right child roots and add both counts.
One present row plus padding	Tail node: hash the present child with the padding subtree root, while the count only includes the present child.

The resulting instance of this tree is a final internal-recursive deferral aggregation proof for one deferral circuit. The `merkle_commit` is a commitment to the list of folded input/output commitment pairs, and its `num_def_circuit_proofs` counts how many deferral circuit proofs were included, thus how many leaves the hook layer should expect to open.

Deferral framework: hook layer

The deferral hook layer sits between a per-circuit deferral aggregation proof and the combined VM/deferral continuations tree. It verifies one final internal-recursive deferral aggregation proof, decommits the proof's `merkle_commit` into IO leaves, folds those leaves into the VM-style deferral accumulators, and exposes the result as `DeferralPvs`.

`MerkleDecommitAir` rebuilds the IO Merkle tree whose root is the child proof's `merkle_commit`. Each real leaf is a pair `(input_commit, output_commit)`. The trace may include padded leaves to make the tree size a power of two, but only the real prefix is sent onward through `IoCommitBus`; non-sent leaf rows are constrained to contain zero commitments.

`OnionHashAir` receives those IO pairs in order and folds them into two Poseidon2 onion accumulators:

```
input_onion_0  = def_circuit_commit
input_onion_i  = H(input_onion_{i-1}, input_commit_i)

output_onion_0 = 0
output_onion_i = H(output_onion_{i-1}, output_commit_i)
```

The initial input onion is the `def_circuit_commit`, not zero. This is how the VM-side accumulator is seeded with the identity of the deferral circuit whose outputs are being consumed.

`DeferralHookPvsAir` computes `def_circuit_commit` from the deferral aggregation VK commits, checks that the child proof is from the internal-recursive deferral layer, and receives the final onion values. It then exposes `DeferralPvs`:

- `initial_acc_hash`: the memory-subtree leaf representing the initial accumulator state, built from `def_circuit_commit` and a zero output accumulator.

- `final_acc_hash`: the memory-subtree leaf representing the final accumulator state, built from the final input and output onions.
- `depth = 1`: the hook proof represents one input/output accumulator pair for a single deferral circuit.

These hook proofs are then aggregated by the combined VM/deferral continuations tree. There, `DeferralPvsAir` recursively Merklizes the hook outputs across deferral circuits by hashing child `initial_acc_hash` values together, hashing child `final_acc_hash` values together, and incrementing `depth` at each binary aggregation step. The prover can choose to omit some proofs for some subtree, but in this case the initial and final roots for those subtrees must be equal. Since the initial tree is checked to be equal to the expected deferral address space commitment, this means that any omitted subtrees must be unchanged from the initial state, so the omitted subtrees cannot contribute any new deferral IO pairs.

Root wrapper

The root wrapper is the final STARK layer before the static/EVM-oriented proof format. It takes one internal-recursive child proof and replaces the recursive aggregation public values with the compact public claims expected by the outer verifier.

The wrapper checks that the child proof represents a successful terminated execution, that it comes from the internal-recursive layer, and that the child verifier commitment matches either the first internal-recursive case or the self-recursive case. It then computes:

- `app_exe_commit`: a commitment to the app program commitment, the initial memory root, and the initial program counter.
- `app_vm_commit`: a commitment to the app, leaf, and internal-for-leaf VK commits.

The user public values are exposed by the root proof itself through `UserPvsCommitAir`. This AIR chunks them into digests and builds a Merkle root, while `UserPvsInMemoryAir` proves that this public-values root opens at `PUBLIC_VALUES_AS` under the child's final memory root. This is the step that links the user-visible public values to the VM execution state.

When deferrals are enabled, the root also receives the child proof's `VerifierDefPvs` and `DeferralPvs`. If the child has accumulated deferrals, the root checks that the propagated `def_hook_commit` matches the expected hook circuit commitment and sends the `initial_acc_hash`, `final_acc_hash`, and `depth` to `DeferralAccMerklePathsAir`. That AIR opens the initial accumulator hash to the VM's initial memory root and the final accumulator hash to the VM's final memory root under the `DEFERRAL_AS` subtree. If the child claims no deferrals, the root constrains the deferral public values to be unset and proves the relevant deferral address-space region stayed unchanged.

Deferral VM extension

The deferral extension adds two VM opcodes, `CALL` and `OUTPUT`, that let a guest program delegate a computation to an external deferral circuit while keeping the VM proof linked to the deferred computation through commitments. The guest calls `deferred_compute::<IDX>(&input_commit)`, where `IDX` selects the deferred function/circuit and `input_commit` is a 32-byte commitment to the deferred input. The `CALL` opcode returns an `OutputKey { output_commit, output_len }`: `output_commit` commits to the deferred output bytes, while `output_len` tells the guest how much memory to allocate before retrieving the raw output.

The raw output is then consumed through `get_deferred_output::<IDX>(&mut output, &output_key)`, which emits the `OUTPUT` opcode. `OUTPUT` reads the `(output_commit, output_len)` pair, writes the raw output bytes to guest memory, and constrains that those bytes hash back to the supplied `output_commit` under the selected deferral index. In the intended safe pattern, `output_len` should be treated as an allocation hint until `OUTPUT` has succeeded; application logic should branch on the actual output only after `OUTPUT` has authenticated it.

Internally, the extension maintains per-circuit accumulator state in the dedicated deferral address space, `DEFERRAL_AS = 4`. Here `4` is the address-space identifier, not the accumulator value itself: concrete memory addresses are pairs of the form `(address_space, pointer)`, so `DEFERRAL_AS[k]` below is shorthand for `(DEFERRAL_AS, k)`.

Note that there can be multiple deferral circuits, so for each deferral circuit index `i`, the VM stores an input accumulator and an output accumulator at fixed offsets:

```
input_acc_i  at DEFERRAL_AS[2*i*DIGEST_SIZE]
output_acc_i at DEFERRAL_AS[(2*i + 1)*DIGEST_SIZE]
```

These offsets are field-element pointers. The memory tree groups cells into `CHUNK = DIGEST_SIZE = 8`-field blocks, so the `input_acc_i` pointer corresponds to block `2*i` and the `output_acc_i` pointer corresponds to block `2*i + 1` in the diagram below.

`CALL` folds the input commitment into `input_acc_i` and the output commitment into `output_acc_i`. Later, the continuation/root deferral machinery checks that these accumulator updates are consistent with the separate deferral proofs, so the final proof ties together both sides: the VM execution that requested deferred work, and the external proof that the deferred computation was valid.

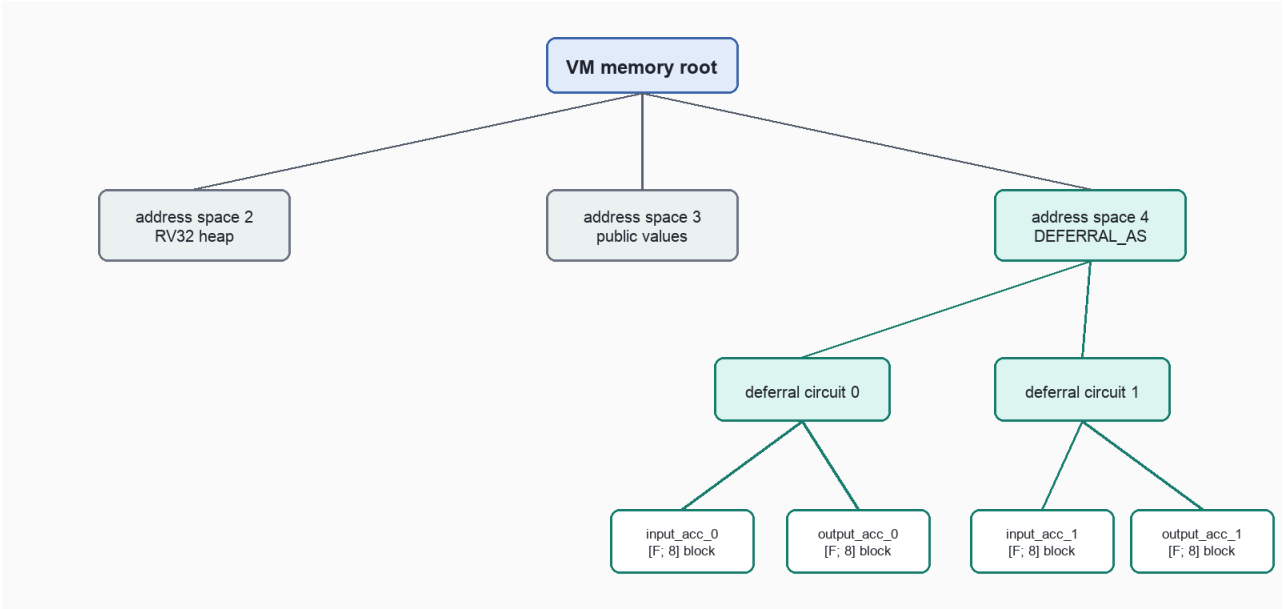
Memory tree and the DEFERRAL_AS subtree

OpenVM represents VM memory with a Merkle-tree-like commitment so that each segment can summarize a large memory state with a single root. A caveat is that the internal node operation is a Poseidon2-based fixed-width compression, not a generic collision-resistant hash function by itself. This is the same design family as the Plonky3-style Merkle tree discussed in The Billion Dollar Merkle Tree: the internal compression is not a collision-resistant hash function, but instead it is a truncated permutation. The security of the construction relies on the fact that the leaf hashes are computed using a collision-resistant hash function.

During a segment, the VM only touches a small subset of memory blocks. A tree commitment lets the prover update and authenticate the changed blocks through short Merkle paths, instead of recomputing or exposing the entire memory image at every segment boundary. The resulting roots are the compact public commitments to the segment's initial and final memory states.

OpenVM organizes memory as a tree of fixed-size memory blocks. Each leaf represents one block of `8` field elements, and the tree path first identifies the address space, then the block inside that address space. The root of this tree is a compact commitment to the whole VM memory state.

At a segment boundary, the VM exposes the memory root before and after execution as `initial_root` and `final_root`, so the verifier can check that memory changed consistently without seeing the entire memory.



`DEFERRAL_AS = 4` is one subtree inside this memory tree. The deferral extension stores the per-circuit input and output accumulators in fixed slots under that subtree. `CALL` updates those accumulator slots during VM execution, while `OUTPUT` later authenticates raw output bytes against the returned `output_commit` without changing the accumulator state. Separately, the deferral aggregation and hook layers expose `initial_acc_hash`, `final_acc_hash`, and `depth`, which describe the accumulator subtree that should connect the VM execution to the deferred computations.

At the root layer, `DeferralAccMerklePathsAir` links these two worlds: it checks that the `initial_acc_hash` and `final_acc_hash` exposed by the deferral hook open, via Merkle paths, to the VM's `initial_root` and `final_root` under the `DEFERRAL_AS` subtree. The intended invariant is that `depth` stays within the `DEFERRAL_AS` subtree height (`address_height`), so the accumulator opening cannot point above the deferral subtree into unrelated memory regions.

Host verifier

The host verifier is the host-side Rust verifier for a final internal-recursive `VmStarkProof`. It verifies the STARK proof with the supplied aggregate verifying key, checks the proof's public values, and also aggregates/validates deferrals by linking the deferral accumulator state back to the VM memory roots. Conceptually, it plays the same boundary-verification role as the root verifier circuit, but in Rust code instead of AIR constraints; therefore, consistency between these two verifiers should be maintained as closely as possible. This verification function is exposed through the SDK as `Sdk::verify_proof` and is also what runs behind the scenes for the `cargo openvm verify stark` CLI command.

The verifier essentially verifies two things:

1. Verify the inner proof, which contains the STARK proof of the outermost internal-recursive circuit.
2. Verify the proof's public values.

The first verification simply invokes `stark-backend` proof verification against the inner proof using the fixed `VmStarkVerifyingKey`. The second verification is where most of the specific checks happen:

- Verifies that the user public values open to the claimed final memory root.
- Recomputes and checks the application executable commitment from the VM public values.
- Checks that execution terminated successfully.
- Enforces that the recursion depth is in the range `[1, MAX_RECURSION_DEPTH]`
- Compares the exposed app, leaf, and internal verifier commitments against the baseline.
- Checks the recursive cached trace commitment.
- Validates the deferral state:
    - If `deferral_flag == 0`: checks that no deferral public values are set and that the deferral address space is unchanged.
    - If `deferral_flag == 2`: checks that the exposed `def_hook_commit` matches the expected baseline and that the deferral accumulator roots are linked to the VM memory roots.

verify-stark guest library

The `verify-stark` guest library is a supported deferral use case: it lets a Rust guest ask the deferral framework to verify another OpenVM STARK proof. Instead of executing the full STARK verifier inside the guest program, the guest passes an `input_commit` to `verify_stark::<DEF_IDX>(input_commit, expected)`. The `DEF_IDX` parameter selects the registered verify-stark deferral circuit. The `input_commit` is the guest-visible deferral input key for the child proof verification claim; the verify-stark circuit constrains it from the child verifier transcript state, and the deferral state supplies the corresponding output bytes.

The guest wrapper is intentionally small. At a high level, its flow is:

```
guest calls verify_stark::<DEF_IDX>(input_commit, expected)
        |
        v
CALL / deferred_compute
        |
        v
OutputKey { output_commit, output_len }
        |
        v
OUTPUT / get_deferred_output
        |
        v
raw output bytes
        |
        v
parse as app_exe_commit || app_vm_commit || user_public_values
        |
        v
compare parsed ProofOutput against expected
```

The middle part is the generic deferral pattern: `CALL` returns an `OutputKey`, `OUTPUT` authenticates and materializes the raw output bytes, and only then should the guest parse or branch on the output. In the verify-stark wrapper, `verify_stark_unchecked::<DEF_IDX>`, after calling `deferred_compute::<DEF_IDX>(input_commit)`, it allocates a buffer of length `output_len` taken from the `output_key`, and immediately calls `get_deferred_output::<DEF_IDX>(&mut output_bytes, &output_key)` to validate the output buffer. After the output is validated, it checks the output length to be sufficiently large, and it starts to parse the result as `ProofOutput = app_exe_commit || app_vm_commit || user_public_values`. The first two fields are 32-byte commitments and `user_public_values` is the remaining byte encoding of the child proof's public values. The checked wrapper, `verify_stark::<DEF_IDX>`, compares this parsed `ProofOutput` against the `expected` value supplied by the guest and panics if they differ.

On the deferral-circuit side, `guest-libs/verify-stark/circuit` proves that the verification claim represented by `input_commit` comes from a valid aggregated OpenVM STARK proof for the expected child verifier key and proof shape. It constrains the child proof's VM and verifier public values, checks that the child exited successfully, binds the child user public values to the child final memory root, and computes the output commitment for the serialized `ProofOutput`. If the child proof itself used deferrals, the verify-stark circuit can also include the deferral accumulator Merkle-path checks needed to bind the child's deferral public values back to its memory roots.

This composes with the generic deferral framework in the usual way: the guest-side `CALL` records the claimed `(input_commit, output_commit)` pair in the deferral accumulators, `OUTPUT` authenticates the raw bytes against `output_commit`, and the external deferral proof shows that the output was produced by the verify-stark circuit for the child proof verification claim represented by `input_commit`.

§04

Phase 3: Static Verifier

The audit focused on the `static-verifier` crate (part of the OpenVM v2 implementation). The crate implements a STARK verifier using Halo2 (with KZG polynomial commitments) and is used to "compress" the final STARK proof from OpenVM into a small PlonK proof: 2080 bytes for STARK-in-Halo2 and 1376 + 384 = 1760 bytes for the STARK-in-Halo2-in-Halo2 proof. The goal is for this proof to be posted on-chain without excessive gas costs: posting the original STARK proof would require a large amount of calldata.

Overview of Key Concepts

In this section we give an overview of key concepts specific to the Halo2 verifier circuit.

Lazy foreign-field arithmetic

The Halo2 circuit natively enforces constraints on the scalar field of the BN254 curve. We write `Fr` to denote the BN254 scalar field and `Fr::MODULUS` to denote its modulus. In order to verify an OpenVM STARK proof, the Halo2 circuit needs to enforce constraints on BabyBear field operations. To bridge this gap, the circuit needs to implement arithmetic in a foreign field. If implemented naively, these foreign-field operations can be prohibitively costly.

One key idea in optimizing the foreign-field arithmetic is to leverage the fact that `Fr` is much larger than the BabyBear field. A single `Fr` cell can store an *unreduced* BabyBear element as a *centered lift*: `value` is a signed integer, with a negative `-v` encoded as `Fr::MODULUS - v`. The signed value is only recoverable as long as operations do not over- or underflow `Fr::MODULUS/2`; past that bound the positive and negative ranges collide and the sign is lost.

Towards this, the Halo2 circuit defines a `BabyBearWire` structure that stores a value `value` and a number of bits `max_bits`. The former is the signed, unreduced BabyBear element; the latter bounds its magnitude as `|value| < 2^max_bits`, with `max_bits <= Fr::CAPACITY - RESERVED_HIGH_BITS` so that no operation can reach `Fr::MODULUS/2` and wrap the centered lift. It is crucial for soundness that `value` and `max_bits` are always consistent with each other: when the `BabyBearWire` is instantiated and through every operation applied to it.

Multi-field transcripts

In order to represent hashing operations efficiently as an `Fr` circuit, the `static-verifier` crate uses a Poseidon2 sponge over `Fr`. However, this sponge needs to absorb and squeeze BabyBear field elements. The `static-verifier` implementation therefore needs to provide a form of *codec*: packing BabyBear field elements into `Fr` elements (for absorbing) and converting `Fr` elements into multiple BabyBear elements (when squeezing).

Specifically, packing is implemented by treating the input BabyBear field elements as the 2^{31}-ary representation of a `Fr` field element:

$$\mathsf{pack}\left(v_0, \ldots, v_{k-1}\right) = \sum_{i=0}^{k-1} v_i \ \cdot \ 2^{i \cdot 31}$$

This packing assumes that we are not packing more than $k = 8$ BabyBear field elements (see `NUM_OBS_PER_WORD`).

Unpacking is slightly different since a base-2^{31} decomposition of an `Fr` element would yield a vector of elements in $\left[0, 2^{31}\right)$ rather than a vector of BabyBear field elements. The unpack operation takes as input a value in in `Fr` and outputs $u_0, \ldots, u_4$ and top such that:

$$\begin{aligned} u_i &\in [0, p) \quad \text{for } i \in \{0, \ldots, 4\} \\ \mathsf{top} &\in \left[0, \left\lfloor \frac{\mathsf{Fr::MODULUS} - 1}{p^5} \right\rfloor\right] \\ \mathsf{in} &= \sum_{i=0}^{4} u_i \cdot p^i + \mathsf{top} \cdot p^5 \end{aligned}$$

The elements $u_0, \ldots, u_4$ are now guaranteed to be BabyBear elements derived from the sponge output.

§05

Phase 4: RC.1 (SHA-2, Keccak, and Memory-Adapter Removal)

The audit focused on the rc.1 changes, which can be categorized in three main areas.

- **New SHA-2 family AIRs and guest libraries.** A redesigned SHA-2 implementation that adds SHA-256, SHA-384, and SHA-512 (previously only SHA-256 was supported). The relevant crates (`crates/circuits/sha2-air`, `extensions/sha2`, `guest-libs/sha2`, and the `crates/circuits/primitives/derive/src/cols_ref/` support) were reviewed as new code.
- **New Keccak-256 AIRs and guest libraries.** A simpler Keccak-256 redesign (`extensions/keccak256`, `guest-libs/keccak256`), also reviewed as a new implementation.
- **Removal of memory-access adapters.** Memory-access adapters previously reconciled different block sizes on the memory bus. rc.1 removes them: every memory access now uses a single `DEFAULT_BLOCK_SIZE = 4` cells, and `PersistentBoundaryAir` (`crates/vm/src/system/memory/persistent.rs`) is updated to bridge 4-cell bus accesses against the 8-cell Merkle leaves of the persistent-memory tree.

Following the guidance from the OpenVM team, we treated the two hash redesigns as new implementations and the memory change as a diff.

In addition to the main repository, the following forked guest libraries were in scope, as they patch standard hash crates to call OpenVM intrinsics on the `zkvm` target:

- `openvm-org/hashes` (a RustCrypto `hashes` fork) on branches `openvm/sha2-v0.10.8` and `openvm/sha3-v0.10.8`, along with the corresponding `v0.11.0` patches.
- `openvm-org/tiny-keccak`, patched for the OpenVM target.

Overview of Key Mechanisms

This subsection describes three recurring design patterns encountered during the rc.1 review. They are not findings; they illustrate how OpenVM achieves soundness across independent AIRs and help frame the findings that follow.

Cross-AIR binding of a permutation via a timestamp-keyed bus

The Keccak-f permutation is split across two AIRs that never share a row. `KeccakfOpAir` handles the VM-facing side of one `KECCAKF` instruction: it reads the state pointer, binds the 200-byte pre-state and post-state to memory, and drives the execution bus. `KeccakfPermAir` wraps the Plonky3 `keccak-air`, which actually constrains `post = keccak_f(pre)` over 24 rounds. Neither AIR proves the other's part; they are joined by a single permutation-check bus. The op AIR *sends* `(0, t, pre)` and `(1, t, post)` and the perm AIR *receives* the same two messages, so multiset equality forces every claimed `(pre, post)` pair emitted by an op row to be matched by a perm row that internally proves it is a real permutation pair.

The two messages share a timestamp `t` but are sent separately (to avoid one very large message), so the bus binds the multiset of preimages at time `t` and the multiset of postimages at time `t` independently. The pre/post pairing then rests on `t` being unique per enabled instruction, which the perm AIR documents it assumes but does not enforce. That uniqueness is supplied by the global execution model: the execution bus is a permutation anchored by the connector, every instruction advances the timestamp by a positive amount, and all timestamps are bounded below the field by the memory checker, so two enabled instructions cannot share a timestamp. We reviewed this and consider it sound; it is a compact illustration of the project's "correctness lives on the buses" philosophy, and the same shape recurs (the SHA-2 main chip and block-hasher, range/bitwise lookups, and the memory bus itself).

The memory bus, the boundary anchor, and 4-to-8 chunk reconciliation

OpenVM has no single global memory trace. Every chip that touches memory posts its reads and writes to a shared memory bus through a memory bridge; each access carries `(address_space, pointer, value, timestamp)`, and the offline checker enforces that timestamps strictly increase per cell so values flow forward in time. For the bus to balance, the **boundary chip** is the unique source of each cell's initial value (at timestamp 0) and the unique sink of its final value; the boundary's initial and final states are hashed into a Merkle tree whose roots are public, anchoring the whole memory history.

The interesting rc.1 detail is the consequence of removing the access adapters. Every memory access now uses `DEFAULT_BLOCK_SIZE = 4` cells, but the persistent-memory Merkle tree hashes leaves of `CHUNK = 8` cells. So the boundary chip must bridge a 4-cell bus against an 8-cell Merkle leaf, which it does by splitting each 8-cell chunk into two 4-cell memory-bus messages: initial rows pin both sub-blocks' timestamps to 0; final rows carry an independent timestamp per 4-cell sub-block; and an untouched 8-cell chunk must still match its initial value on both halves so untouched siblings cancel cleanly on the bus. The recurring audit question for this scope is whether the 4-to-8 split preserves the invariant that the boundary is the unique source and sink, with no way to inject or hide a cell. We reviewed the split and the Merkle sparse-subtree reuse and found them consistent.

`hash` versus `final_hash`: a free chaining payload

The standalone SHA-2 sub-AIR processes a sequence of compression blocks and, on each block's digest row, stores two state-shaped fields that look interchangeable but are not. `final_hash` is the computed output of the block's compression (range-checked, the real result). `hash` is a *free* field whose addition carries are not range-checked on digest rows, so the prover may set it to any bit-valid value; it serves only as the outbound payload of an internal chaining bus, where each block sends `hash` and receives the next block's `prev_hash`. So `hash` is not "this block's result" but "the value to hand to the next block": within a single multi-block hash the honest prover sets `hash = final_hash`, and at an invocation boundary it sets `hash` to the next invocation's IV. VM-level correctness does not flow through `hash` at all; the wrapper chip binds each block's state to memory and to the SHA-2 bus using `prev_hash` and `final_hash`. This pattern surprised several reviewers, who each re-derived "`hash` is never constrained to equal `final_hash`" and briefly read it as a soundness gap before seeing that `final_hash` is what is authenticated downstream. The lesson, shared with the first mechanism above, is that an unconstrained-looking column can be intentional communication slack rather than an under-constraint.

§06

Findings

Below are listed the findings found during the engagement. High severity findings can be seen as so-called "priority 0" issues that need fixing (potentially urgently). Medium severity findings are most often serious findings that have less impact (or are harder to exploit) than high-severity findings. Low severity findings are most often exploitable in contrived scenarios, if at all, but still warrant reflection. Findings marked as informational are general comments that did not fit any of the other criteria.

HIGH

#00	Padded deferral aggregation roots can be reinterpreted as hook IO leaves	`crates/continuations/src/circuit/deferral/inner/def_pvs/air.rs`
#01	Host verifier doesn't verify the deferral Merkle proofs on the zero deferral flag	`crates/verify/src/lib.rs`
#02	Unchecked initial and final sibling of the deferral memory tree allows deferral hook proof forgery	`crates/verify/src/deferral.rs`
#03	Aggregation counters can overflow in the field	`Deferral and continuation aggregations`

MEDIUM

#04	Bus index increment can overflow in interaction hashing, allowing interaction forgery	`stark-backend/src/verifier/batch_constraints.rs`
#05	Non-canonical proof encodings allow proof malleability	`stark-backend/src/proof.rs, stark-sdk/src/config/baby_bear_bn254_poseidon2.rs`
#06	Arbitrary trailing bytes after a proof are accepted	`stark-backend/src/codec.rs, stark-backend/src/proof.rs`
#07	Incorrect GKR with zero-interaction AIRs check allows proof malleability	`stark-backend/crates/stark-backend/src/verifier/batch_constraints.rs`
#08	Statement malleability due to unpinned public values length	`crates/verify/src/lib.rs`
#09	Deferral Merkle opening depth is not bounded by the deferral subtree height	`crates/verify/src/deferral.rs`
#0a	Unreduced BabyBear Wires Are Absorbed Into the Transcript	`crates/static-verifier/src/transcript/mod.rs`

#0b tiny-keccak finalize Silently Truncates Output to 32 Bytes on the zkVM Target
`openvm-org/tiny-keccak src/keccak.rs, guest-libs/keccak256/src/zkvm_impl.rs`

LOW

#0c Biased sampling in sample_bits
`stark-backend/crates/stark-backend/src/transcript.rs`

#0d Same hash function used for Fiat-Shamir and PoW
`stark-backend/crates/stark-backend/src/transcript.rs`

#0e Recursive verifier missing q0_claim constraint for zero-interaction GKR proofs
`openvm/crates/recursion/src/gkr/input/air.rs`

#0f Proof decoding allocates vectors of untrusted size
`stark-backend/crates/stark-backend/src/proof.rs`

#10 Deferral CALL's OutputKey.output_len is an unauthenticated prover hint
`extensions/deferral/circuit/src/call/air.rs`

#11 Non-canonical deferral tree and unset-path representations
`crates/continuations/src/circuit/inner/def_pvs/air.rs`

#12 verify-stark no-deferral def_hook_commit mismatch
`guest-libs/verify-stark/circuit/src/verifier/air.rs`

#13 Non-canonical CommitBytes allow byte-level commitment aliases
`crates/continuations/src/commit_bytes.rs`

#14 BabyBear Div Can Overflow Its Bit Budget and Panic in assert_zero
`crates/static-verifier/src/field/baby_bear/base.rs`

#15 Observation Is Not Sound for Arbitrary BabyBear Wires
`crates/static-verifier/src/transcript/mod.rs`

#16 Incorrect Bound Argument in signed_div_mod for Negative Dividends
`crates/static-verifier/src/field/baby_bear/base.rs`

#17 native_xorin and the sha3 Fork Accept Sponge Rates the XORIN Circuit Cannot Prove
`extensions/keccak256/guest/src/lib.rs, extensions/keccak256/circuit/src/xorin/air.rs`

#18 tiny-keccak v384 and v512 Panic at Runtime on the zkVM Target Despite Being Circuit-Supportable
`openvm-org/tiny-keccak src/keccak.rs`

INFORMATIONAL

#19 Round-by-round soundness calculations
`SWIRL whitepaper & stark-backend/crates/stark-backend/src/soundness.rs`

#1a Recursive verifier reuses stacking bus for distinct messages

openvm/crates/recursion/src/batch_constraint/sumcheck/,
openvm/crates/recursion/src/stacking/opening/air.rs

#1b Unnecessary and redundant constraints in recursive verifier
openvm/crates/recursion/src/*

#1c OUTPUT relies on the guest convention that OutputKey came from CALL
extensions/deferral/circuit/src/output/air.rs

#1d verify-stark API and hygiene notes
guest-libs/verify-stark/circuit/src/commit/air.rs

#1e Defense-in-Depth Observations and Nits
crates/static-verifier/src

#1f Performance Observations and Nits
crates/static-verifier/src

#20 BabyBear Reduce Does Not Assert Its max_bits Precondition
crates/static-verifier/src/field/baby_bear/base.rs

#21 Sampling Zero Bits Skips the Sponge Transition and Can Desync From the Native Transcript
crates/static-verifier/src/transcript/mod.rs

#22 signed_div_mod proof intuition does not match the code
crates/static-verifier/src/field/baby_bear/base.rs

#23 Formal-Verification Theorems Do Not Assume a Uniform Extraction-Derived Constraint Interface
openvm-fv VmExtensions/{Extraction,Constraints,Soundness}

#24 Documented ptr_max_bits Precondition Is Not Asserted in SHA-2 and Keccak XORIN
extensions/sha2/circuit/src/sha2_chips/main_chip/air.rs, extensions/keccak256/circuit/src/xorin/air.rs

#25 SDK App Keygen Does Not Mark Required System AIRs and verify_segments Omits the Boundary Check
crates/sdk/src/keygen/mod.rs, crates/vm/src/arch/config.rs, crates/vm/src/arch/vm.rs

KNOWN ISSUE

#26 Underconstrained pairing hint allows proof forgery
openvm/guest-libs/pairing/src/*/pairing.rs

HIGH · #00

Padded deferral aggregation roots can be reinterpreted as hook IO leaves

crates/continuations/src/circuit/deferral/inner/def_pvs/air.rs

Description. The deferral aggregation root is not a canonical commitment to the proved deferral IO list. A malicious prover can make an absent padding child contribute an attacker-chosen digest to the aggregation root, and the hook layer can then interpret that same digest as a valid IO leaf.

The immediate AIR bug is that `DeferralAggPvsAir` does not enforce that padding children use a canonical padding value. The child-specific constraints are gated by `is_present` (`crates/continuations/src/circuit/deferral/inner/def_pvs/air.rs:94-126`), so when `is_present = 0`, the child's values are mostly unconstrained. However, the parent aggregation hash still includes `next.merkle_commit` (`crates/continuations/src/circuit/deferral/inner/def_pvs/air.rs:206-214`):

```
self.poseidon2_bus.lookup_key(
    builder,
    Poseidon2CompressMessage {
        input: digests_to_poseidon2_input(local.merkle_commit, next.merkle_commit)
            .map(Into::into),
        output: merkle_commit.map(Into::into),
    },
    is_first_of_two_rows,
);
```

The honest trace generator fills the absent row slot with canonical padding (`crates/continuations/src/circuit/deferral/inner/def_pvs/trace.rs:79-84`), but the AIR does not require a malicious prover to do the same. This lets an attacker produce a valid aggregation proof with a public root of the form `R = H(real_child_merkle_commit, attacker_chosen_padding)` while still reporting `num_def_circuit_proofs = 1`.

The count check does not prevent this. `num_def_circuit_proofs` counts the number of present deferral proofs, but it does not bind the Merkle tree shape or role of the resulting root. A two-row aggregation node with one present child and one absent child still reports one real proof. The hook can therefore decommit the same root `R` as a one-leaf IO Merkle tree where `input_commit = real_child_merkle_commit`, `output_commit = attacker_chosen_padding`, and `leaf_hash = H(input_commit, output_commit)`. The hook decommit layer only sends leaves marked by `send_commits` (`crates/continuations/src/circuit/deferral/hook/decommit/air.rs:61-97`), and the hook verifier checks that the number of onion-hashed leaves equals `num_def_circuit_proofs` (`crates/continuations/src/circuit/deferral/hook/verifier/air.rs:257-264`). Since both views contain exactly one real item, the count check does not detect that an aggregation internal node is being interpreted as a hook IO leaf.

This reinterpretation is possible because the same Poseidon2 compression format is used for aggregation internal nodes and hook IO leaves. There is no domain-separation tag distinguishing `H(left_child_root, right_child_root)` from `H(input_commit, output_commit)`.

Impact. The verifier accepts a proof claiming a valid deferred call `synthetic_input_commit -> attacker_chosen_output_commit` even though the real underlying deferral proof was for an unrelated computation.

We confirmed this with a test against the production root verifier. The PoC attack constructs a malicious deferral aggregation by mutating the absent padding child's `merkle_commit` to encode a desired forged output commitment, routes the resulting padded aggregation root through the deferral hook layer and the full VM/deferral continuation aggregation up to the root prover, and confirms that the final root verifier accepts the resulting proof. Using a small guest that reads an input commit and an expected output (a pattern similar to `crates/continuations/programs/examples/multiple.rs` and analogous in style to `crates/sdk/programs/examples/verify-stark.rs`), the PoC produces two independently verifying root proofs for the same guest binary and same claimed input commit, accepting two different outputs.

Recommendation.

The immediate fix is to ensure absent children cannot contribute unconstrained data. In `DeferralAggPvsAir`, constrain every non-present child row's `merkle_commit` to the depth-appropriate canonical padding digest already used by the honest trace generator. The important property is that the public aggregation root must be a deterministic function of the present children and the canonical tree shape, not of prover-chosen padding columns.

As additional hardening, bind the Merkle tree shape/depth to `num_def_circuit_proofs` so that a root produced as an internal aggregation node cannot be decommitted later as a one-leaf IO tree with the same count. Finally, add domain separation between aggregation leaf hashes, aggregation internal hashes, hook IO leaf hashes, and hook internal hashes, for example by including a role tag in the Poseidon2 input.

Client response. PR #2815 partially fixes this issue by constraining absent deferral aggregation padding to the canonical zero hash and adding Merkle depth to the deferral aggregation public values. This addresses the direct attacker-chosen-padding primitive and adds shape/depth accounting. Additionally, domain separation between the input and output commitments compression and internal compression has been implemented in PR #2857, which solves this class of reinterpretation issues altogether.

HIGH · #01

Host verifier doesn't verify the deferral Merkle proofs on the zero deferral flag

crates/verify/src/lib.rs

Description. The host `VmStarkProof` verifier accepts `deferral_flag = 0` by checking only that the deferral public values are unset: `def_hook_commit`, `initial_acc_hash`, `final_acc_hash`, and `depth` are zero values.

```
// Deferral verification
if let Some(expected_def_hook_commit) = vk.baseline.expected_def_hook_commit {
    let &VerifierDefPvs {
        deferral_flag,
        def_hook_commit,
    } = verifier_def_pvs_slice.borrow();

    let &DeferralPvs {
        initial_acc_hash,
        final_acc_hash,
        depth,
    } = proof.inner.public_values[DEF_PVS_AIR_ID]
        .as_slice()
        .borrow();

    if deferral_flag == F::ZERO {
        if !is_unset(&def_hook_commit) {
            return Err(VerifyStarkError::DefHookCommitSet {
                actual: def_hook_commit,
            });
        } else if !is_unset(&initial_acc_hash) {
            return Err(VerifyStarkError::DefInitialAccHashCommitSet {
                actual: initial_acc_hash,
            });
        } else if !is_unset(&final_acc_hash) {
            return Err(VerifyStarkError::DefFinalAccHashCommitSet {
                actual: final_acc_hash,
            });
        } else if depth ≠ F::ZERO {
            return Err(VerifyStarkError::DefDepthSet { actual: depth });
        }
    } ...
```

However, these checks were not enough because they do not prove that the `DEFERRAL_AS` memory subtree stayed untouched throughout VM execution.

This also differs from the root verifier AIR. In the root circuit, even when the deferrals are unset, the deferral Merkle path still proves that the deferral address space did not change by enforcing the equality between the initial and final path nodes inside `DEFERRAL_AS`:

```
assert_array_eq(
    &mut builder.when(and(local.is_within_deferral_as, local.is_unset)),
    local.initial_node_commit,
```

```
    local.final_node_commit,
);
```

As a result, a malicious prover can produce a valid `VmStarkProof` whose execution uses deferral `CALL` instructions and mutates deferral memory by setting `deferral_flag = 0` and omitting the deferral Merkle proof, causing the verifier to skip the deferral Merkle opening entirely.

Impact. A malicious prover can make native SDK verification accept a VM proof that consumed forged or unproven deferred inputs and/or outputs. The VM proof remains internally valid, and its `final_root` commits to the mutated deferral memory, but the verifier never checks that the mutation is represented by `initial_acc_hash` / `final_acc_hash`. Practically, this can forge a proof guarded by the guest-level `verify_stark` function that relies on deferred verification.

Note that this only affects `VmStarkProof` SDK verification and does not affect EVM verification, since the root circuit should reject the same condition.

Recommendation. Even if `deferral_flag == 0`, the verifier should enforce the Merkle opening check that requires all initial/final nodes inside `DEFERRAL_AS` are equal, mirroring the root verifier AIR constraints.

Client response. PR #2817 addresses this finding. The host verifier now requires and verifies deferral Merkle proofs whenever deferrals are enabled, regardless of the `deferral_flag` value.

HIGH · #02

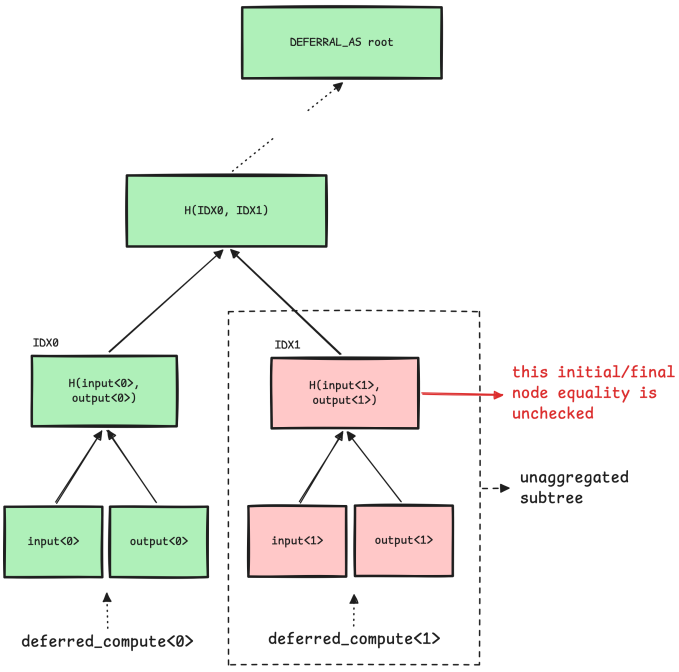
Unchecked initial and final sibling of the deferral memory tree allows deferral hook proof forgery

crates/verify/src/deferral.rs

Description. The host deferral verifier checks the initial and final deferral accumulator paths (`initial_acc_hash` and `final_acc_hash`) independently using the supplied Merkle proofs, such that it recomputes:

- `initial_acc_hash` ⇒ `initial_root`
- `final_acc_hash` ⇒ `final_root`

However, it never checks that `initial_merkle_proof[level] == final_merkle_proof[level]` for a `level` that is inside `DEFERRAL_AS` memory **but** outside the aggregated deferral subtree. This means that if there are 2 deferrals at the same tree level and only one of them is aggregated, the other deferral (the sibling node) is not enforced to remain unchanged throughout the VM execution. The simplified tree is depicted below:



This is in contrast with the root verifier AIR, which explicitly enforces this missing invariant: while the path is still inside `DEFERRAL_AS`, it requires the initial and final siblings to be equal, proving that all deferral accumulator subtrees not covered by the aggregate were unchanged:

```
assert_array_eq(
    &mut builder.when(local.is_within_deferral_as),
    local.initial_sibling,
```

```
    local.final_sibling,
);
```

Impact. This effectively allows a malicious prover to forge the effect of a deferral hook proof for an omitted deferral subtree: the VM can consume a deferred input/output at that index, while the verification accepts without any corresponding hook proof being aggregated.

As a proof of concept, consider the following guest program which has two deferred `verify_stark` calls:

```
#![cfg_attr(target_os = "zkvm", no_main)]
#![cfg_attr(target_os = "zkvm", no_std)]

extern crate alloc;

use alloc::vec::Vec;

use openvm::io::read;
use openvm_deferral_guest::Commit;
use openvm_verify_stark_guest::{verify_stark, ProofOutput};

openvm::entry!(main);

fn read_expected() -> (ProofOutput, Commit) {
    let app_exe_commit: Commit = read();
    let app_vm_commit: Commit = read();
    let user_public_values: Vec<u8> = read();
    let input_commit: Commit = read();

    (
        ProofOutput {
            app_exe_commit,
            app_vm_commit,
            user_public_values,
        },
        input_commit,
    )
}

pub fn main() {
    let (expected_0, input_commit_0) = read_expected();
    verify_stark::<0>(&input_commit_0, &expected_0);

    let (expected_1, input_commit_1) = read_expected();
    verify_stark::<1>(&input_commit_1, &expected_1);
}
```

An SDK prover can execute the guest program, while supplying a hook proof for only one of them. The second deferral's input/output can still be supplied through the VM execution deferral state, so the guest observes it as if it came from deferred verification, but the host verifier only checks the aggregate for the first deferral, as seen in the following snippet:

```
// Honest child proof for deferral index 0.
let honest_child_vk = VmStarkVerifyingKey {
    mvk: child_agg_vk,
```

```
    baseline: child_baseline,
};

let honest_state = get_deferral_state(
    &honest_child_vk,
    from_ref(&honest_child_proof),
    0,
)?;

// Forged deferral result for index 1.
// This output is chosen by the malicious prover and is not backed by a hook proof.
let fake_state = DeferralState::new(generate_deferral_results(
    vec![RawDeferralResult {
        input: fake_input_commit.to_vec(),
        output_raw: fake_output_raw, // app_exe_commit || app_vm_commit || user_public_values
    }],
    1, // deferral index 1
    &deferral_poseidon2_chip::<F>(),
));

// Guest stdin contains the expected values that the guest will compare against
// the two deferred outputs.
let mut stdin = StdIn::default();
stdin.write(&honest_app_exe_commit);
stdin.write(&honest_app_vm_commit);
stdin.write(&honest_user_public_values);
stdin.write(&honest_input_commit);

stdin.write(&fake_app_exe_commit);
stdin.write(&fake_app_vm_commit);
stdin.write(&fake_user_public_values);
stdin.write(&fake_input_commit);

// VM execution sees both deferrals.
stdin.deferrals = vec![honest_state, fake_state];

// But the deferral proof path receives only the honest child proof for idx0.
let def_input = DeferralInput::from_inputs(from_ref(&honest_child_proof));

// Host aggregation proof is generated with one hook proof,
// while VM execution used two deferral states.
let (proof, _) = prover.prove(stdin, from_ref(&def_input))?;

// Vulnerable acceptance: host verification accepts because it checks the
// covered aggregate path, but does not prove the omitted sibling stayed unchanged.
Sdk::verify_proof(agg_vk, baseline, &proof)?;
```

Note that this only affects `VmStarkProof` SDK verification and does not affect EVM verification, since the root circuit should reject the same condition.

Recommendation. The host verifier should enforce the equality of the initial and final siblings for every Merkle tree level inside `DEFERRAL_AS` that is outside the aggregated subtree, mirroring the root verifier AIR invariant.

Client response. PR #2817 addresses this finding. The host verifier now checks that the deferral address space outside the claimed aggregated subtree is unchanged.

HIGH · #03

Aggregation counters can overflow in the field

Deferral and continuation aggregations

Description. The recursive aggregation depth is unbounded, so a malicious prover can perform any polynomial number of aggregation steps. This is not inherently a problem for Merkle claims: a Merkle root remains binding for polynomial-length paths. However, some aggregation metadata is accumulated as native field elements rather than as bounded integers.

Two such values are:

- The `depth` used in memory Merkle aggregation, which is incremented as a field element.
- `num_def_circuit_proofs`, which is read from child proofs and summed into the parent proof.

Because the BabyBear field is relatively small, these counters can wrap around. For the memory Merkle `depth`, an overflow would require $O(|\mathbb{F}|)$ aggregation steps. This could let a prover craft, for example, a root with `depth = 0` or a pseudo-random root with `depth = 1`, but we did not identify an immediate soundness issue from this path alone.

For `num_def_circuit_proofs`, overflow is more practical. At each binary aggregation step, the counter can double, so wrapping the counter only requires $O(\log |\mathbb{F}|)$ aggregation steps. By aggregating trees of different heights, the prover can effectively choose the overflowed value using a double-and-add strategy. In particular, the prover can make the overflowed `num_def_circuit_proofs` equal to `1`, which lets the hook layer interpret the aggregated Merkle root as if it contained a single real IO leaf right below the root.

This has the same shape as the padded-root reinterpretation issue: an aggregation internal node can be opened as a hook IO leaf. The root cause is different from arbitrary padding, but the issue relies on the same lack of separation between leaf hashes and internal-node hashes.

Impact. A malicious prover can use counter overflow to make an aggregated deferral tree appear to contain a different number of deferral circuit proofs than it actually contains. With the concrete `num_def_circuit_proofs = 1` case, this enables the same kind of Merkle-root reinterpretation as the padded deferral aggregation issue.

Recommendation. Treat aggregation counters as bounded integers, not unconstrained field accumulators. Enforce range bounds or structural constraints that prevent `depth` and `num_def_circuit_proofs` from wrapping, and bind `num_def_circuit_proofs` to the aggregation tree shape. As hardening, domain-separate leaf hashes from internal-node hashes so that an aggregation root cannot be reinterpreted as a hook IO leaf.

Client Response. This issue has been addressed in PRs #2815 and #2838. The circuits now keep track of the Merkle depth and range-check it against `MAX_DEF_AGG_MERKLE_DEPTH`, set to 20. Additionally, domain separation between the input and output commitments compression and

internal compression has been implemented in PR #2857, which solves this class of reinterpretation issues altogether.

MEDIUM · #04

Bus index increment can overflow in interaction hashing, allowing interaction forgery

stark-backend/src/verifier/batch_constraints.rs

Description. Interaction hashing adds `1` to `interaction.bus_index`, but `BusIndex` is a `u16`.

```
let num = evaluator.eval_expr(&interaction.count);
let denom = interaction
    .message
    .iter()
    .map(|expr| evaluator.eval_expr(expr))
    .chain(std::iter::once(SC::EF::from_u16(interaction.bus_index + 1)))
    .zip(beta_logup.powers())
    .fold(SC::EF::ZERO, |acc, (x, y)| acc + x * y);
(num, denom)
```

In release builds, this addition can wrap around, so `interaction.bus_index = 2^16 - 1` is mapped to `0` instead of `2^16`. As a result, a user defining an AIR with interactions on bus `2^16 - 1` can cause the verifier to hash the zero bus value, which would completely break soundness for interactions and allow arbitrary forged interactions.

The documentation for `BusIndex` states that all valid instantiations of `BusIndex` are safe, which suggests that the verifier should not be performing arithmetic on bus indices that can overflow.

```
// Must be a type smaller than u32 to make BusIndex p - 1 unrepresentable.
pub type BusIndex = u16;

// ...

/// The bus index specifying the bus to send the message over. All valid instantiations of
/// `BusIndex` are safe.
pub bus_index: BusIndex,
```

However, the current hashing logic computes `+1` over `u16`, which breaks this expectation at the maximal value. This is particularly relevant because the low-level AIR builder allows interactions to be pushed to arbitrary bus indices by circuit designers, including `2^16 - 1`.

Impact. A user can define a circuit using bus index `2^16 - 1` for which the verifier computes the wrong message hash. This can enable forged interactions and breaks soundness for such circuits.

Recommendation. Perform the increment after widening the bus index to a larger integer or directly to a field element. For example, compute the bus index as `SC::EF::from_u16(interaction.bus_index) + SC::EF::ONE`. Alternatively, reject `2^16 - 1` as an invalid bus index at key generation time.

Client response. This issue has been fixed in PR #312 by computing the increment after casting the bus index to a field element, avoiding overflow.

MEDIUM · #05

Non-canonical proof encodings allow proof malleability

stark-backend/src/proof.rs, stark-sdk/src/config/baby_bear_bn254_poseidon2.rs

Description. The proof codec admits multiple distinct byte encodings for the same logical proof through two independent decoding issues.

First, `trace_vdata` presence flags are encoded as a bitmap in `Proof::encode`. When the number of AIRs is not a multiple of 8, the final bitmap byte contains unused padding bits. During decoding, those bits are ignored rather than required to be zero:

```
for byte in bitmap {
    for i in 0u8..8 {
        if trace_vdata.len() ≥ num_airs {
            break;
        }
        if byte & (1u8 << i) ≠ 0 {
            trace_vdata.push(Some(TraceVData::decode(reader)?));
        } else {
            trace_vdata.push(None);
        }
    }
}
```

As a result, an attacker can flip the unused high bits of the last bitmap byte without changing the decoded `Proof`.

Second, `BabyBearBn254Poseidon2Config` canonically encodes BN254 digest elements, but its decoder accepts non-canonical 32-byte encodings:

```
let big = BigUint::from_bytes_be(&buf);
*val = Bn254Scalar::from_biguint(big)
    .ok_or_else(|| io::Error::other("invalid Bn254 element"))?;
```

`Bn254Scalar::from_biguint` does not enforce that the input is strictly less than the BN254 modulus, so distinct byte strings such as `x` and `x + p` may decode to the same field element. Since digests appear throughout proof decoding, including commitments and Merkle proof siblings, this also creates alternative serialized forms of the same proof.

Together, these issues make the proof encoding non-canonical.

Impact. Given a valid proof π, anyone can produce a distinct proof π' by either modifying ignored `trace_vdata` bitmap padding bits or replacing canonical BN254 digest encodings with non-canonical ones.

Recommendation. Enforce canonical proof decoding in both places. In `Proof::decode`, reject any non-zero padding bits in the final `trace_vdata` bitmap byte. In `BabyBearBn254Poseidon2Config::decode_digest`, reject any 32-byte integer greater than or equal to the BN254 modulus instead of accepting it via `from_biguint`.

Client Response. This issue has been acknowledged and fixed in PR #356.

MEDIUM · #06

Arbitrary trailing bytes after a proof are accepted

stark-backend/src/codec.rs, stark-backend/src/proof.rs

Description. The proof codec does not enforce full consumption of the input byte slice when decoding a standalone proof. The generic helper `Decode::decode_from_bytes` wraps the input in a `Cursor` and returns after a single `decode` call:

```
fn decode_from_bytes(bytes: &[u8]) -> Result<Self> {
    let mut reader = Cursor::new(bytes);
    Self::decode(&mut reader)
}
```

At the top level, `Proof::decode` reconstructs a proof from the reader and returns immediately after decoding the expected fields:

```
Ok(Self {
    common_main_commit,
    trace_vdata,
    public_values,
    gkr_proof: GkrProof::decode(reader)?,
    batch_constraint_proof: BatchConstraintProof::decode(reader)?,
    stacking_proof: StackingProof::decode(reader)?,
    whir_proof: WhirProof::decode(reader)?,
})
```

No check is performed to ensure that the reader has reached EOF. As a result, if a proof is decoded from a standalone byte blob, any arbitrary suffix appended after a valid proof is ignored. Distinct byte strings of the form `proof || suffix` therefore decode to the same `Proof` object.

Impact. Given a valid serialized proof, anyone can append extra trailing bytes to produce a different proof blob that still decodes to the same proof and verifies successfully. This makes proof bytes non-canonical and can break systems that compare, hash, or deduplicate proofs by raw bytes.

Recommendation. After `Proof::decode` finishes decoding the expected proof fields, check that the input stream is at EOF and reject the input if any bytes remain. This ensures that only exact proof encodings are accepted and prevents arbitrary trailing-byte malleability.

Client Response. This issue has been acknowledged and fixed in PR #356.

MEDIUM · #07

Incorrect GKR with zero-interaction AIRs check allows proof malleability

stark-backend/crates/stark-backend/src/verifier/batch_constraints.rs

Description. The fractional sumcheck GKR verification is supposed to handle zero-interaction AIRs by asserting that `q0_claim` is exactly one.

```
if total_rounds == 0 {
    // Proof shape asserts that `proof.claims_per_layer` and `proof.sumcheck_polys` are empty.
    debug_assert!(proof.claims_per_layer.is_empty());
    debug_assert!(proof.sumcheck_polys.is_empty());
    if proof.q0_claim ≠ SC::EF::ONE {
        return Err(GkrVerificationError::InvalidZeroRoundValue {
            actual: proof.q0_claim,
        });
    }
    return Ok((SC::EF::ZERO, SC::EF::ONE, vec![]));
}
```

However, this check is incorrect and unreachable because of the following:

1. This check can only be reached if `total_rounds == 0`, which is unreachable as long as `l_skip > 0`, since if the `total_interactions == 0`, then `total_rounds = l_skip + n_logup = l_skip + 0`.
2. This check should return `(0, alpha_logup)` instead of `(0, 1)` to satisfy the downstream denominator check.
3. In the higher function call, there is an additional conditional statement that guards the `verify_gkr` function, which is only called if `total_interactions` is more than zero:

```
if total_interactions > 0 {
    (p_xi_claim, q_xi_claim, xi) =
        verify_gkr::<SC, TS>(gkr_proof, transcript, l_skip + n_logup)?;
    debug_assert_eq!(xi.len(), l_skip + n_logup);
}
```

As a result, `q0_claim` is never checked, so it is freely malleable and can be mutated to any arbitrary field element while the modified proof still passes verification.

Impact. Given a valid proof π, anyone can produce a distinct proof π' by changing `q0_claim` to any value in the extension field. While the soundness of statement verification itself is not affected, the ability to generate multiple valid proof representations has concrete implications for systems built on top of the verifier, such as bypassing proof-based replay protection mechanisms.

Recommendation. Make the early return inside `verify_gkr` reachable when there are no interactions, and remove the outer `if total_interactions > 0` guard so that `verify_gkr` runs unconditionally. The early return value should also be corrected from `(0, 1)` into `(0,`

`alpha_logup)` to satisfy the downstream denominator check.

Client Response. This issue has been acknowledged and fixed in PR #326.

MEDIUM · #08

Statement malleability due to unpinned public values length

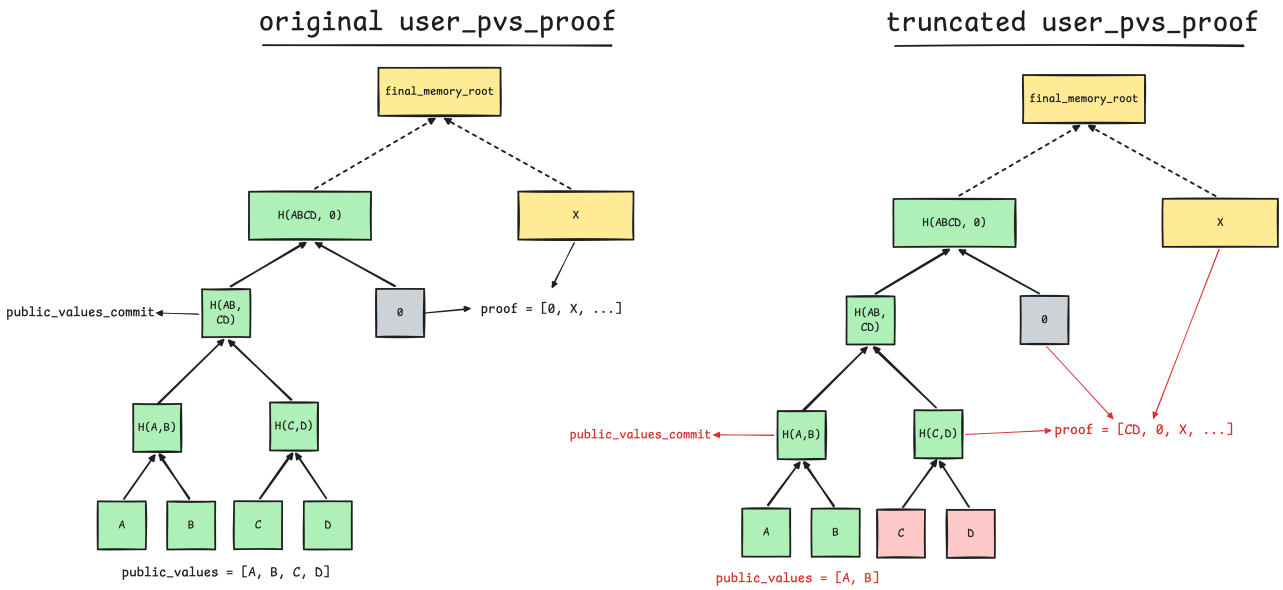
crates/verify/src/lib.rs

Description. The host verifier checks the validity of the public values (`UserPublicValuesProof`) by deriving the public values subtree height from the prover-supplied length instead of checking it against the publicly configured `num_user_pvs` . The underlying check in `UserPublicValuesProof::verify()` only requires the supplied length to be a power-of-two multiple of `CHUNK` :

```
let pvs = &self.public_values;
if !pvs.len().is_multiple_of(CHUNK) || !(pvs.len() / CHUNK).is_power_of_two() {
    return Err(UserPublicValuesProofError::UnexpectedLength(pvs.len()));
}
let pv_height = log2_strict_usize(pvs.len() / CHUNK);
let proof_len = memory_dimensions.overall_height() - pv_height;
```

As a result, the values in `proof.user_pvs_proof` can be malleated in two possible ways:

1. **Truncation**: The adversary truncates `public_values` with shorter values, recomputes `public_values_commit` for the shorter vector, and adds the Merkle root of the removed values to the `proof` . See the diagram below for the concrete attack example.
2. **Extension**: The adversary appends `public_values` from the adjacent authenticated subtree, recomputes `public_values_commit` for the larger vector, and removes the Merkle sibling that previously represented those appended values from `proof` . Note that the appended values cannot be arbitrary since they must match the adjacent memory subtree already authenticated by the original Merkle path.



Impact. Given a valid proof π and public statement x, anyone can produce a distinct statement x' by truncating or extending the attached user public values vector from x, as long as the mutated vector still Merkle-opens to the same final memory root.

While the extension is possible, the public values vector can only be extended with values that match the adjacent authenticated memory subtree already committed by the original Merkle path. In practice, this is usually only feasible for known subtrees such as unused zero-filled memory. So, truncation is the main practical impact here.

This issue affects both the SDK verification of `ContinuationVmProof` and `VmStarkProof`.

Recommendation. We suggest pinning `num_user_pvs` in every host verifier entry point. It is recommended that `num_user_pvs` be added to `VerificationBaseline` so that it is tied to the verifier configuration material.

Client response. PR #2817 addresses this finding for `VmStarkProof` verification. `VerificationBaseline` now records the expected number of raw user public values, and the host verifier rejects proofs whose user public value proof length does not match the baseline.

Since the same issue also affects SDK verification of `ContinuationVmProof`, this is addressed separately in PR #2835, as it is outside the scope of this phase.

MEDIUM · #09

Deferral Merkle opening depth is not bounded by the deferral subtree height

crates/verify/src/deferral.rs

Description. `DeferralPvs.depth` controls how many low Merkle levels are skipped before `initial_acc_hash` and `final_acc_hash` are opened against the VM memory roots. The host verifier checks that `depth` is not larger than the overall memory tree height, but it does not require nonzero depths to satisfy `depth <= memory_dimensions.address_height`, which is the height of the deferral address-space subtree. As a result, an oversized `depth` can move the opening above `DEFERRAL_AS`.

In `DeferralMerkleProofs::verify`, the only validation on the proof shape is that each Merkle proof has length `overall_height()`; `depth` is then passed straight through as the number of skipped levels (`crates/verify/src/deferral.rs:53-68`):

```
let overall_height = memory_dimensions.overall_height();
if self.initial_merkle_proof.len() ≠ overall_height {
    return Err(VerifyStarkError::DeferralMerkleProofLengthMismatch { /* ... */ });
}
// ... same check for final_merkle_proof ...

let is_unset = depth == 0;
let skip_depth = if is_unset { 0 } else { depth };
let idx_prefix =
    usize::try_from(memory_dimensions.label_to_index((DEFERRAL_AS, 0))).unwrap();
```

`skip_depth` is used directly in `merkle_path_root`, which skips the first `skip_depth` siblings and walks the remaining levels to the root (`crates/verify/src/deferral.rs:110-131`). There is no constraint relating `depth` to `address_height`, so any `depth` in `[1, overall_height]` is accepted, including values that skip past the deferral subtree boundary.

This is reachable through the full public-value verifier: `depth` is read from `DeferralPvs` and forwarded into `DeferralMerkleProofs::verify` with no bound check (`crates/verify/src/lib.rs:275-317`).

This was confirmed with proof-of-concept tests. One sets `depth = address_height + 1` (greater than `address_height`, still within `overall_height`) and shows the Merkle-path check accepts it. Another drives the full public-value verifier with an oversized `depth` whose opened subtree also covers `PUBLIC_VALUES_AS`, and the verifier accepts.

Impact. The binding between the deferral hook's accumulator claims and the VM's actual deferral accumulator memory is weakened. Instead of proving that the accumulator lives inside the deferral region, the proof may authenticate a larger subtree that also includes unrelated memory regions (for

example `PUBLIC_VALUES_AS`). A direct end-to-end exploit is not obvious without another way to make the accumulator public values match such an oversized subtree root, but the verifier is checking the wrong invariant: the opening should be pinned to the `DEFERRAL_AS` subtree.

Recommendation. Enforce `1 <= depth <= memory_dimensions.address_height` so the deferral opening always terminates at the root of the `DEFERRAL_AS` subtree and cannot be raised into sibling address spaces. This bound should be enforced consistently in both the native host verifier (`crates/verify`) and the root AIR (`crates/continuations/src/circuit/root/def_paths`).

Client response. PR #2817 addresses this finding. It adds the missing bound in both the native verifier and the root AIR path logic.

MEDIUM · #0A

Unreduced BabyBear Wires Are Absorbed Into the Transcript

crates/static-verifier/src/transcript/mod.rs

Description. The verifier absorbs proof-derived BabyBear wires into the Fiat-Shamir transcript, and hashes them into WHIR Merkle leaves, without first reducing them to a canonical representative. The wires it absorbs are loaded from the proof by `load_witness`, which only range-checks the magnitude to 31 bits:

```
pub fn load_witness(&self, ctx: &mut Context<Fr>, value: BabyBear) → BabyBearWire {
    let value = ctx.load_witness(Fr::from(PrimeField64::as_canonical_u64(&value)));
    self.range.range_check(ctx, value, BABYBEAR_MAX_BITS); // BABYBEAR_MAX_BITS = 31
    BabyBearWire { value, max_bits: BABYBEAR_MAX_BITS }
}
```

A prover-supplied wire is therefore constrained only to lie in $[0, 2^{31})$, not in the canonical range $[0, p)$ with $p = p_{BB} = \texttt{0x78000001}$. Since $p < 2^{31}$, the interval $[0, 2^{31})$ contains non-canonical representatives: for every $x < 2^{31} - p$ both x and $x + p$ are valid 31-bit witnesses, distinct cells, but equal as BabyBear elements. The arithmetic gadgets route everything through `signed_div_mod` and so only ever compare values modulo p; the two cells are interchangeable for every algebraic constraint.

The transcript and the Merkle leaf hash, on the other hand, consume the **raw** cell, packed in base 2^{31}:

```
/// Result = values[0] + values[1]*2^31 + values[2]*2^62 + ...
fn pack_base_2_31(ctx, gate, values: &[BabyBearWire]) → AssignedValue<Fr> {
    gate.inner_product(ctx, values.iter().map(|v| v.value),

gate.pow_of_two().iter().step_by(BABY_BEAR_BITS).take(values.len()).copied().map(Constant))
}
```

`observe` buffers up to `NUM_OBS_PER_WORD = 8` wires and packs them this way; WHIR Merkle leaves are loaded with the same 31-bit `load_witness` (`stages/whir/mod.rs:108-147`) and hashed by `hash_babybear_slice_to_digest`, which packs identically (`hash/poseidon2.rs:204`). The packed word, and hence every transcript challenge and every Merkle digest, is a function of the raw 31-bit limbs, not of the BabyBear field elements they represent.

Concretely: replacing any absorbed value $x < 2^{31} - p$ by $x + p$ leaves every algebraic constraint satisfied (they all reduce modulo p) but changes the packed word, and therefore the transcript from that point on. Exactly $2^{31} - p = \texttt{0x7ffffff}$ field elements admit such a second encoding (one in sixteen of the field), so the values the verifier absorbs are frequently not bound to a unique encoding.

Impact. The transcript and the WHIR Merkle commitment do not *canonically* bind the field-element content of the values the live verifier absorbs. The concrete consequence is challenge grinding. A malicious prover can swap any absorbed value below $2^{31} - p$ for its non-canonical representative

$x + p$ without disturbing a single algebraic constraint (they all reduce modulo p); each swap changes the packed word, and therefore the transcript from that point on, hence every subsequent challenge: sumcheck challenges, WHIR folding and query challenges, and the proof-of-work check. The prover thereby obtains re-randomizations of the challenges for free, outside the soundness and PoW analysis, lowering the cost of steering the transcript toward a state that accepts a false statement.

The same ambiguity makes proofs non-unique: a given statement has many distinct accepting proofs, since each absorbed value with a second representative yields another valid transcript, so a proof cannot be treated as a unique token. This is a grinding and uniqueness weakness, not a cheap proof rewrite. Re-encoding an absorbed value cascades through the Fiat-Shamir transcript, so every later challenge changes; and under a fixed commitment the Merkle path binds the exact raw leaf, so the digest cannot be reopened to a different representative. Any alternative proof must be produced by re-running the prover.

Unlike the companion finding *"Observation is not sound for arbitrary BabyBear wires"*, this weakness is reachable on the live absorb paths. The wires fed into `observe`, `observe_ext`, and the leaf hash in the actual verifier (public values, sumcheck claims, codeword and out-of-domain openings, and so on) are exactly the `load_witness` outputs above; none of them is canonicalized before being hashed. The values are non-negative in $[0, 2^{31})$, so the signed-limb collisions of the companion finding do not arise here, but the non-canonical representatives do.

Recommendation. Fully reduce every `BabyBearWire` to its canonical range $[0, p)$ before it is absorbed into the transcript or hashed into a Merkle leaf, rather than only bounding `max_bits`. Once each limb is in $[0, p)$ with $p < 2^{31}$, the base-2^{31} packing is injective and canonical, and both the grinding and the malleability above disappear. Asserting `max_bits <= 31` is *not* sufficient: it does not remove the non-canonical representatives.

`reduce` already produces a canonical representative; `signed_div_mod` constrains its remainder to $[0, p)$. The robust fix is to lift this into the type system: introduce a `ReducedBabyBearWire` newtype, constructible only through `reduce`, exposing `AsRef<BabyBearWire>` so it can be used anywhere a wire is expected, and have `observe`, `observe_ext`, `pack_base_2_31`, and the Merkle leaf hash accept only this type. Absorbing a raw, possibly non-canonical wire then becomes a compile error. The codebase already has the underlying primitive, used on sampled digits in `constrain_base_baby_bear_decomposition`:

```
range.check_less_than_safe(ctx, digit.value, BABY_BEAR_MODULUS_U64);
```

Apply the same `check_less_than_safe(_, _, BABY_BEAR_MODULUS_U64)` to proof field elements at load time (cleanest inside the proof-data loader), or reduce each wire to a canonical representative immediately before `pack_base_2_31` and before leaf hashing.

Client Response. This issue has been addressed in PR #2848 by introducing reduced BabyBear newtypes and requiring them on transcript absorption and Merkle leaf hashing paths. The reduced witness loaders range-check values against the BabyBear modulus before they can be packed.

MEDIUM · #0B

tiny-keccak finalize Silently Truncates Output to 32 Bytes on the zkVM Target

openvm-org/tiny-keccak src/keccak.rs, guest-libs/keccak256/src/zkvm_impl.rs

Description. On `target_os = "zkvm"`, the `openvm-org/tiny-keccak` fork patches `tiny_keccak::Keccak` to wrap `openvm_keccak256::Keccak256`, whose `finalize_ptr` always writes exactly `KECCAK_OUTPUT_SIZE = 32` bytes regardless of the caller's buffer length:

```
// zkvm_impl.rs
core::ptr::copy_nonoverlapping(self.state.0.as_ptr(), output, KECCAK_OUTPUT_SIZE);
```

The fork's `finalize` guards only the too-small case:

```
// keccak.rs (zkvm path)
assert!(output.len() ⩾ 32, "output buffer too small");
unsafe { self.state.finalize(output) };
```

The `assert!` covers memory safety (`output.len() < 32`) but not `output.len() > 32`. The host and upstream `Keccak::finalize` squeeze `output.len()` bytes (Keccak-256's rate is 136, so squeezing up to 136 bytes from one permutation is well-defined). So for any `output.len() > 32`, the zkVM build writes 32 correct bytes and leaves `output[32..]` stale, while the host build and the Keccak spec produce a full `output.len()`-byte squeeze. The path is reachable without triggering the fork's non-256 panic: `Keccak::v256()` is the supported constructor, and `finalize(&mut [0u8; N])` with `N > 32` then silently truncates.

Impact. Medium. A guest using `tiny_keccak::Keccak::v256` as a short extendable-output function (for example a 48 or 64 byte derivation) computes a value in the zkVM that differs from the host build and from the Keccak specification, silently: the first 32 bytes are correct, the rest are stale buffer bytes. A program that relies on those bytes (key or seed derivation, multi-value expansion) would compute a wrong value in the zkVM and still pass native tests. This is not a circuit-soundness break (the proof faithfully attests to whatever the guest computed), but it is a real guest-library correctness and compatibility bug. Likelihood is low, since Keccak-256 is overwhelmingly used as a 32-byte digest and the main ecosystem consumer routes through the fixed-32 `native_keccak256` path, but the failure is silent and correctness-affecting.

Recommendation. On the zkVM target, make the contract explicit instead of silently truncating: either reject `output.len() != 32` (failing loudly, matching the spirit of the non-256 panic) or fall back to the software `KeccakState` squeeze for `output.len() != 32` so the full squeeze matches upstream. Document that on the zkVM target `tiny_keccak::Keccak` supports only the 32-byte Keccak-256 digest.

Client Response. The issue is fixed in PR #2872, by adding an exact check to enforce an output size of 32 bytes in `finalize`, and in PR #7 of the tiny-keccak fork, by replacing the wrapper that caused the truncation so arbitrary output lengths are squeezed correctly.

LOW · #0C

Biased sampling in sample_bits

stark-backend/crates/stark-backend/src/transcript.rs

Description. The `sample_bits` method samples a random field element, interprets it as an integer, and then reduces it modulo `2^bits`. Since the field modulus is not a power of two, this does not sample uniformly from `[0, 2^bits)`: values below a threshold determined by the modulus occur slightly more often than values above it.

This affects at least two uses of `sample_bits`:

- In the PoW check, the sampled value is required to be zero. For example, when requesting a 30-bit PoW challenge, the success probability is `2/p` rather than exactly `2^-30`, which corresponds to approximately `29.91` bits of work rather than `30`.
- The WHIR verifier uses `sample_bits` to derive query indices. As a result, these indices are also not sampled uniformly from the intended range, and lower indices are selected with a slight preference over higher ones.

If `x` is sampled uniformly from `F_p` and `y = x mod 2^bits`, then letting `t = p mod 2^bits`, the output distribution satisfies

$$\Pr\left[y = a \mid a \in [0, t)\right] = \frac{\lfloor p/2^{bits} \rfloor + 1}{p},$$

and

$$\Pr\left[y = a \mid a \in [t, 2^{bits})\right] = \frac{\lfloor p/2^{bits} \rfloor}{p}.$$

In particular, the PoW check succeeds when `sample_bits(bits) == 0`, so its success probability is

$$\Pr\left[\text{PoW succeeds}\right] = \frac{\lfloor p/2^{bits} \rfloor + 1}{p}.$$

For the modulus `p` used here, some representative examples are:

BITS	MORE LIKELY OUTPUT INTERVAL	POW SUCCESS PROBABILITY (LOG2)
8	$[0, 1)$	-7.9999998
16	$[0, 1)$	-15.99995
24	$[0, 1)$	-23.9880

BITS	MORE LIKELY OUTPUT INTERVAL	POW SUCCESS PROBABILITY (LOG2)
27	$[0, 1)$	-26.9069
28	$[0, 134217729)$	-27.9069
29	$[0, 402653185)$	-28.9069
30	$[0, 939524097)$	-29.9069

The following Python snippet reproduces these probabilities:

```python
from math import log2

p = 2**31 - 2**27 + 1

for bits in [8, 16, 24, 27, 28, 29, 30]:
    threshold = p % 2**bits
    p_below = (p // 2**bits + 1) / p
    p_above = (p // 2**bits) / p
    print(
        f"{bits} bits: threshold={threshold}, "
        f"log2(p_below)={log2(p_below):.18g}, "
        f"log2(p_above)={log2(p_above):.18g}"
    )
```

Impact. The induced bias is small, but it means that values returned by `sample_bits` are not distributed exactly as assumed by idealized soundness and PoW calculations. In particular, the actual PoW success probability is slightly higher than the uniform `2^-bits` target, and WHIR queries exhibit a slight preference toward low-index positions.

Recommendation. If uniform sampling over `[0, 2^bits)` is required, implement `sample_bits` using rejection sampling instead of truncating a uniformly sampled field element. Otherwise, document the bias explicitly and account for it in any soundness or PoW parameter calculations that rely on `sample_bits`.

Client response. The OpenVM team is aware of this bias. Since it is relatively small, especially when sampling less than 27 bits, the team is not planning to change the sampling method. The soundness calculator was updated to account for this bias in PR #357.

LOW · #0D

Same hash function used for Fiat-Shamir and PoW

stark-backend/crates/stark-backend/src/transcript.rs

Description. The design of the `FiatShamirTranscript` trait forces the protocol to use the same hash function to implement the Fiat-Shamir transform and to enforce proof-of-work (PoW) challenges. In particular, the associated function `check_witness` shown below forces the proof-of-work bits to be derived from `self`:

```
    fn check_witness(&mut self, bits: usize, witness: SC::F) -> bool {
        if bits == 0 {
            return true;
        }
        self.observe(witness);
        self.sample_bits(bits) == 0
    }
```

Impact. This design limits flexibility for parameter setting and may cause a degradation in the overall security level when changing hash functions. For example, the BLAKE3 hash function is a valid choice to implement the Fiat-Shamir transcript but is substantially faster to compute than SHA-3 or Poseidon2. This means that n bits of PoW using BLAKE3 provides much less security than n bits of PoW using Poseidon2 (as used in the current parameter config).

Recommendation. We give two orthogonal recommendations. The first is what we consider to be best practice, while the second is more immediate and avoids changes to the codebase.

- *(best practice)*. We recommend "separating responsibilities" of the hash functions. Concretely, one could modify the `check_witness` function above by replacing `self.sample_bits(bits)` with a call to another PoW-specialized hash function. This modification removes a potential foot-gun and allows for future configurations using more elaborate PoW functions (for example using memory-hard functions to defend against malicious provers that use custom and highly-parallel hardware).
- *(simpler mitigation)*. Alternatively, this issue can be mitigated by clearly documenting this fact and encouraging anyone modifying the configuration parameters to think about adjusting the PoW difficulty for their chosen hash function.

Client Response. The client has included new documentation for this behavior in the soundness calculator in PR #357.

LOW · #0E

Recursive verifier missing q0_claim constraint for zero-interaction GKR proofs

openvm/crates/recursion/src/gkr/input/air.rs

Description. In `GkrInputAir`, when the GKR proof has no interactions (`n_logup = 0`), the `q0_claim` column is completely unconstrained. In contrast, the native verifier (in the `verify_gkr` function) contains a corresponding check that enforces `q0_claim == 1` when there are no interactions, which the recursive verifier does not replicate.

As a result, besides the inconsistency, the prover can set `q0_claim` to any arbitrary extension field element without affecting the verification in the same way as the related finding in the native verifier.

Impact. As in the native verifier, the unconstrained `q0_claim` allows proof malleability. But unlike the native verifier, where the malleable field is directly in the proof struct and exploitable by any external party, the recursive verifier's `q0_claim` is a witness column inside the recursion circuit's trace (i.e., witness malleability rather than proof malleability). Although the practical impact is negligible and very limited, it might still warrant a correction to increase robustness and keep consistency with the native verifier.

Recommendation. Add a constraint in `GkrInputAir` that pins `q0_claim` to the canonical value one in the extension field (`[1, 0, 0, 0]`) when there are no interactions.

Client Response. This issue has been acknowledged and fixed in PR #2742.

LOW · #0F

Proof decoding allocates vectors of untrusted size

stark-backend/crates/stark-backend/src/proof.rs

Description. The `Decode` trait (`codec.rs`, lines 28-35) provides an interface to produce proof structures from an untrusted reader (or byte stream). The trait is implemented for SWIRL proofs and its constituent components in `proof.rs` (lines 421-692).

As part of the decoding process, the implementations must allocate vectors of variable sizes. The sizes are directly taken from the reader without sanitization or bounding (beyond what the type system enforces). We highlight an example from the implementation of `Decode` for `BatchConstraintProof` (specifically, lines 532-533) and list all occurrences of similar cases:

```
    let n_max = usize::decode(reader)?;
    let mut sumcheck_round_polys = Vec::with_capacity(n_max);
```

LINE #	PARENT STRUCT	ALLOCATION METHOD	SIZE VARIABLE
457	`Proof`	`Vec::with_capacity()`	`bitmap.len()` (derive from `num_airs`)
461	`Proof`	`Vec::with_capacity()`	`num_airs`
477	`Proof`	`Vec::with_capacity()`	`num_pvs`
501	`GkrProof`	`Vec::with_capacity()`	`num_sumcheck_polys`
504	`GkrProof`	`Vec::with_capacity()`	`n` (from 1 to `num_sumcheck_polys`)
526	`BatchConstraintProof`	`SC::decode_extension_field_vec`	N/A
528	`BatchConstraintProof`	`SC::decode_extension_field_n`	`num_present_airs`
533	`BatchConstraintProof`	`Vec::with_capacity()`	`n_max`
542	`BatchConstraintProof`	`Vec::with_capacity()`	`num_present_airs`
546	`BatchConstraintProof`	`Vec::with_capacity()`	`num_parts`
565	`StackingProof`	`SC::decode_extension_field_vec`	N/A
568	`StackingProof`	`Vec::with_capacity()`	`nums_rounds`
577	`StackingProof`	`Vec::with_capacity()`	`num_openings`

LINE #	PARENT STRUCT	ALLOCATION METHOD	SIZE VARIABLE
579	`StackingProof`	`SC::decode_extension_field_vec`	N/A
594	`WhirProof`	`Vec::with_capacity()`	`num_whir_sumcheck_rounds`
602	`WhirProof`	`SC::decode_digest_vec`	N/A
611	`WhirProof`	`SC::decode_extension_field_n`	`num_whir_rounds` - 1
613	`WhirProof`	`SC::decode_base_field_n`	`num_whir_rounds`
631	`WhirProof`	`Vec::with_capacity()`	`num_commits`
633	`WhirProof`	`Vec::with_capacity()`	`initial_num_whir_queries`
636	`WhirProof`	`Vec::with_capacity()`	`k_whir_exp`
638	`WhirProof`	`SC::decode_base_field_n`	`width`
645	`WhirProof`	`Vec::with_capacity()`	`num_commits`
647	`WhirProof`	`Vec::with_capacity()`	`initial_num_whir_queries`
649	`WhirProof`	`SC::decode_digest_n`	`merkle_depth`
654	`WhirProof`	`Vec::with_capacity()`	`num_whir_rounds` - 1
657	`WhirProof`	`Vec::with_capacity()`	`num_queries`
659	`WhirProof`	`SC::decode_extension_field_n`	`k_whir_exp`
665	`WhirProof`	`Vec::with_capacity()`	`num_whir_rounds` - 1
668	`WhirProof`	`Vec::with_capacity()`	`num_queries`
670	`WhirProof`	`SC::decode_digest_n`	`merkle_depth`
676	`WhirProof`	`SC::decode_extension_field_vec`	N/A

Impact. A malicious prover could abuse this lack of sanitization to force the verifier to allocate arbitrarily large vectors. Depending on the hardware and OS running the verifier, this could cause resource exhaustion. Note, however, that all these sizes are later checked by `verify_proof_shape()` and therefore do not cause soundness issues.

Recommendation. It is good practice to sanitize inputs before allocating vectors.

Client Response. This issue has been acknowledged and fixed in PR #356.

LOW · #10

Deferral CALL's OutputKey.output_len is an unauthenticated prover hint

extensions/deferral/circuit/src/call/air.rs

Description. The deferral `CALL` opcode writes an `OutputKey` into guest heap memory containing both `output_commit` and `output_len`, laid out as one contiguous write `[output_commit || output_len_le]` (`extensions/deferral/circuit/src/call/air.rs:55-67`). Only `output_commit` is bound into the deferral accumulators; `output_len` is not.

In `DeferralCallCoreAir::eval`, the accumulator update hashes only the two commits:

```
let input_f_commit = byte_commit_to_f(&cols.reads.input_commit);
let output_f_commit = byte_commit_to_f(&cols.writes.output_commit);

self.poseidon2_bus
    .lookup(
        cols.reads.old_input_acc,
        input_f_commit,
        cols.writes.new_input_acc,
        AB::Expr::ONE,
    )
    .eval(builder, cols.is_valid);

self.poseidon2_bus
    .lookup(
        cols.reads.old_output_acc,
        output_f_commit,
        cols.writes.new_output_acc,
        AB::Expr::ONE,
    )
    .eval(builder, cols.is_valid);
```

`output_len` never enters any accumulator. The only constraints applied to it are range checks: a byte-pair range check (`extensions/deferral/circuit/src/call/air.rs:151-155`) and a top-byte `< 2^address_bits` check in the adapter (`extensions/deferral/circuit/src/call/air.rs:342-347`). The AIR does not constrain `output_len` to equal the length of the output that produced `output_commit`. (The `DeferralCallWrites` doc comment states `output_len` "must be divisible by `DIGEST_SIZE`", but this is not enforced at `CALL` either.)

The true output length is bound, but inside `output_commit`, not in the separate field. The output commitment is an onion hash seeded with the real length (`extensions/deferral/circuit/src/def_fn.rs:84-85`):

```
let mut state = [F::ZERO; POSEIDON2_WIDTH];
state[0] = F::from_u32(deferral_idx);
state[1] = F::from_usize(output_ref.len());
```

That binding is only re-derived and checked by the `OUTPUT` opcode, which reads the full `[output_commit || output_len]` pair from memory, rebuilds the sponge initial state `[deferral_idx, output_len, 0, ...]`, and on the last row requires both that the recomputed hash equals `output_commit` and that `output_len == section_idx * DIGEST_SIZE` (`extensions/deferral/circuit/src/output/air.rs:196-223`). Consequently, `output_len` as written by `CALL` is an unauthenticated prover hint: a malicious prover can write any range-valid `output_len` into the `OutputKey` while keeping the genuine `output_commit`, and the `CALL` proof still verifies. This is demonstrated by the test `poc_deferral_call_accepts_forged_output_len` (`extensions/deferral/circuit/src/call/tests.rs:478-609`), which keeps the real `output_commit` for an 8-byte output but writes `output_len = 16`, and the trace verifies.

A subsequent `OUTPUT` call on the full `(output_commit, output_len)` pair detects the mismatch, because the genuine commit was computed with the real length and the forged pair fails the hash and `section_idx * DIGEST_SIZE` checks. The risk is therefore confined to guest code that consumes `output_len` before (or without) an `OUTPUT` that authenticates it. The current examples do this. `single.rs` asserts on the `CALL`-returned length before fetching the output:

```
let output_key_0 = deferred_compute::<0>(&INPUT_COMMIT_0);
assert_eq!(output_key_0.output_len as usize, EXPECTED_OUTPUT_0.len());
```

(`extensions/deferral/tests/programs/examples/single.rs:16-18`). And `verify_stark_unchecked` branches on the hint before calling `get_deferred_output` (which is what emits `OUTPUT`):

```
let output_key = deferred_compute::<DEF_IDX>(input_commit);
let output_len = output_key.output_len as usize;

const MIN_OUTPUT_BYTES: usize = 2 * COMMIT_NUM_BYTES;
if output_len < MIN_OUTPUT_BYTES {
    panic!("output_len too small for a ProofOutput");
}
```

(`guest-libs/verify-stark/guest/src/lib.rs:16-26`).

Impact. By contract, `output_len` is a hint that becomes authenticated only when `OUTPUT` is invoked with the matching `output_commit`. The soundness of the deferral framework itself is not affected: `OUTPUT` binds the pair. However, guest code that branches on `output_len`, allocates based on it, meters based on it, or exposes it as part of verified state before `OUTPUT` succeeds can be misled by a forged length. The convention is implicit and is not followed by the shipped examples, which makes the footgun easy to reproduce in downstream guest programs.

Recommendation. Make the intended contract explicit in the documentation: `OutputKey.output_len` is an unauthenticated prover hint until an `OUTPUT` call authenticates it against `output_commit`.

In addition to the docs, adjust the examples to model the safe pattern:

- For fixed-size outputs, allocate the expected-size buffer directly and immediately pass the full `(output_commit, output_len)` pair to `OUTPUT`, without first branching on `CALL`'s returned `output_len`.
- For dynamic-size outputs, treat `output_len` only as an unauthenticated allocation hint: first check it against an application-defined maximum, allocate the buffer, and immediately call `OUTPUT`. Any further logic based on the output (such as parsing, minimum length checks, or branching) should happen only after `OUTPUT` has succeeded.

Client response. The client confirmed that `OutputKey.output_len` returned by `CALL` is intended to be treated as an unauthenticated allocation hint that becomes authenticated only when `OUTPUT` consumes the full `(output_commit, output_len)` pair. The client plans to update the documentation and examples to model safe deferral extension usage. No circuit-level binding of the `CALL`-returned `output_len` to the committed output length is included in the reviewed fixes. Extensive documentation has been added in PR #2866.

LOW · #11

Non-canonical deferral tree and unset-path representations

crates/continuations/src/circuit/inner/def_pvs/air.rs

Description. There are two low-severity canonicality and consistency issues in the deferral tree/path machinery.

First, in `crates/continuations/src/circuit/inner/def_pvs/air.rs`, a two-row aggregation with exactly one real child and one absent child lets the prover-controlled `single_present_is_right` bit decide whether the real child is hashed on the left or the right. The absent child is still constrained to represent an unchanged subtree, so this is not an arbitrary absent-subtree forgery. The AIR checks the one-present-child case around lines `116–122` and uses `single_present_is_right` in the parent hash selection around lines `234–265`, but does not bind that bit to a canonical tree position. This is mainly a malleability issue: we confirmed by testing that the same underlying child proof/public values can produce two different valid parent roots depending only on this position bit.

Second, there is a small inconsistency in how `depth == 0` (unset Merkle paths) are represented. The root AIR's unset path forces the first branch bit to the right in `crates/continuations/src/circuit/root/def_paths/air.rs:147–151`, so its effective path anchors at `(DEFERRAL_AS, 1)`, while the `depth == 0` branch in `DeferralMerkleProofs::verify` uses `label_to_index((DEFERRAL_AS, 0))` in `crates/verify/src/deferral.rs:67–74`. The native `deferral_flag == 0` verifier flow does not call this helper branch; it only checks that the deferral public values are unset in `crates/verify/src/lib.rs:283–299`. The helper branch would matter only in a `deferral_flag == 2 && depth == 0` corner, which does not appear reachable for valid root-AIR proofs because set deferral paths require positive depth. We confirmed the convention mismatch with a trace/unit test showing that the root AIR unset trace maps to `(DEFERRAL_AS, 1)` while the native helper convention maps to `(DEFERRAL_AS, 0)`.

Impact. These are canonicality and consistency issues rather than direct soundness breaks. The single-child position bit gives a prover one bit of root malleability at each single-child aggregation step, but does not let the prover choose an arbitrary root or bypass child-proof verification. The unset-path mismatch appears to be a convention mismatch in an edge/helper path rather than a live native/root verifier divergence.

Recommendation. Canonicalize the single-present-child case by carrying or deriving constrained tree-position metadata and requiring the real child to occupy that position. For the unset path, either align the native helper branch with the root AIR convention or document why the root AIR intentionally anchors the unset case at block `1`.

Client response. PR #2817 aligns the native unset-path convention with the root AIR by requiring deferral Merkle proofs even in the `deferral_flag == 0` case and by using the same depth-zero right-child convention. Additionally, PR #2862 makes each deferral proof carry its position in the deferral memory subtree, enforcing that the deferral index corresponds to the position in memory.

LOW · #12

verify-stark no-deferral def_hook_commit mismatch

guest-libs/verify-stark/circuit/src/verifier/air.rs

Description. There is an AIR/native-verifier consistency mismatch in the verify-stark in-circuit verifier. In `guest-libs/verify-stark/circuit/src/verifier/air.rs`, the check `def_hook_commit == expected_def_hook_commit` was gated by `deferral_flag`, while the no-deferral branch only zeroed `depth`, `initial_acc_hash`, and `final_acc_hash`, leaving `def_hook_commit` unconstrained when `deferral_flag == 0`. The native verifier, by contrast, explicitly rejects non-zero `def_hook_commit` in the `deferral_flag == 0` path.

In other words, the in-circuit verifier could accept malformed child public values that the native verifier rejects:

```
native_verifier(child_proof) = reject
```

but:

```
prove_and_verify(verify_stark_circuit(child_proof)) = accept
```

Impact. This is a consistency mismatch, but `def_hook_commit` does not appear to flow into verify-stark outputs in the no-deferral path, so we did not identify a concrete exploit.

Recommendation. Add an `assert_zeros` constraint for `def_hook_commit` in the no-deferral branch anywhere `deferral_flag == 0` is allowed.

Client response. The client reported this as a known issue and fixed it in PR #2817 before reporting. The fix adds an `assert_zeros` constraint for `def_hook_commit` in the no-deferral branch of the verify-stark AIR, matching the native verifier's `deferral_flag == 0` behavior. The same PR applies the analogous zeroing fix to the root continuation verifier AIR.

LOW · #13

Non-canonical CommitBytes allow byte-level commitment aliases

crates/continuations/src/commit_bytes.rs

Description. `CommitBytes::new` accepts arbitrary 32-byte values, but converting `CommitBytes` into a BabyBear `[F; 8]` digest is not injective. The conversion interprets the bytes as a big integer and decomposes it modulo the BabyBear modulus across 8 limbs, effectively reducing the value modulo `p^8`. As a result, multiple distinct 32-byte strings can map to the same field digest. In practice, most `[F; 8]` digests have 429 valid byte encodings, and some have 430. For example, `0x00..00` and the 32-byte encoding of `p^8` both decode to the same `[F; 8]` value.

Impact. In the current code, this affects byte-ingestion paths such as `VerificationBaselineJson`, where commitment fields are deserialized as raw 32-byte `CommitBytes` and then converted into field digests for verification. This means the same valid proof can verify against multiple distinct raw commitment strings for fields such as `app_exe_commit`, VK commit components, or `expected_def_hook_commit`, as long as those strings decode to the same `[F; 8]`. This does not appear to be a soundness break or a cryptographic hash collision, but it creates byte-level statement malleability and can confuse downstream systems that use raw commitment bytes as app/VK identity.

Recommendation. Enforce a single canonical byte representation for `CommitBytes` values before they are used as BabyBear digests. In particular, external byte-ingestion paths should reject or normalize non-canonical encodings.

Client response. PR #2839 addresses this finding by enforcing canonical `CommitBytes` encodings. `CommitBytes::new` now validates that supplied bytes round-trip through the BabyBear digest encoding, limb-based conversions check canonical field bounds, and the `Bn254 -> CommitBytes` path uses the same checked constructor.

LOW · #14

BabyBear Div Can Overflow Its Bit Budget and Panic in assert_zero

crates/static-verifier/src/field/baby_bear/base.rs

Description. `BabyBearChip::div` constrains `a = b * c (mod p)` by forming `diff = a - b*c` and calling `assert_zero` on it. Before doing so it pre-reduces its operands, and then tags the difference with a `max_bits` that adds one carry bit:

```
let mut c = self.load_witness(ctx, a.to_baby_bear() * b_inv_val);
if a.max_bits > Fr::CAPACITY as usize - RESERVED_HIGH_BITS {
    a = self.reduce(ctx, a);
}
if b.max_bits + c.max_bits > Fr::CAPACITY as usize - RESERVED_HIGH_BITS {
    b = self.reduce(ctx, b);
}
if b.max_bits + c.max_bits > Fr::CAPACITY as usize - RESERVED_HIGH_BITS {
    c = self.reduce(ctx, c);
}
let diff = self.gate().sub_mul(ctx, a.value, b.value, c.value);
let max_bits = a.max_bits.max(b.max_bits + c.max_bits) + 1;
self.assert_zero(ctx, BabyBearWire { value: diff, max_bits });
```

With `RESERVED_HIGH_BITS = 2` and `Fr::CAPACITY = 253`, the threshold is `251`. Observe that the pre-reduction guards use a strict `>`, so an operand sitting *exactly* at the boundary is left untouched: if `a.max_bits == 251`, or `b.max_bits + c.max_bits == 251`, neither reduction fires. The subsequent `+ 1` then pushes `max_bits` to `252`. But `assert_zero` opens by asserting the wire respects the invariant:

```
assert!(a.max_bits <= Fr::CAPACITY as usize - RESERVED_HIGH_BITS);
```

i.e. `assert!(252 <= 251)`, which panics. The bug is that the pre-reduction test omits the carry bit that the final `max_bits` accounts for. Contrast `add` and `sub`, which fold the `+ 1` into the same comparison:

```
if a.max_bits.max(b.max_bits) + 1 > Fr::CAPACITY as usize - RESERVED_HIGH_BITS {
```

A minimal reproduction:

```
let a = BabyBearWire { value: ctx.load_witness(Fr::from(1)), max_bits: 251 };
let b = chip.load_constant(ctx, BabyBear::ONE);
chip.div(ctx, a, b); // a.max_bits == 251 is not reduced; assert_zero panics
```

Impact. This is a completeness defect, not a soundness one: the prover panics during witness generation and cannot produce a proof for the affected operands. A wire only reaches `max_bits == 251` through a deliberate sequence of unreduced operations, so I am not aware of an in-tree call site that triggers it today; the guard is nonetheless off-by-one and will fail closed on a panic rather than silently miscompute.

Recommendation. Fold the carry bit into the pre-reduction guards, mirroring `add` / `sub`: reduce when `a.max_bits + 1 > Fr::CAPACITY - RESERVED_HIGH_BITS` and when `b.max_bits + c.max_bits + 1 > Fr::CAPACITY - RESERVED_HIGH_BITS`, so that `diff.max_bits` never exceeds the `assert_zero` bound.

Client Response. This issue has been addressed by PR #2858 by adding the carry bit to the `div` pre-reduction guards. At the boundary case, operands are reduced before `assert_zero`, so `diff.max_bits` stays within the allowed bound.

LOW · #15

Observation Is Not Sound for Arbitrary BabyBear Wires

crates/static-verifier/src/transcript/mod.rs

Description. `observe` enforces no bound on its input at all:

```
pub fn observe(&mut self, ctx: &mut Context<Fr>, value: &BabyBearWire) {
    self.invalidate_samples();
    self.observe_buf.push(*value);
    if self.observe_buf.len() == NUM_OBS_PER_WORD { self.flush_observe_buf(ctx); }
}
```

So the packed word is whatever raw cells the caller supplies. This matters because a `BabyBearWire` is a BN254 cell that logically holds a *signed* integer, with the invariant documented on the type:

```
/// Logically `value` is a signed integer represented as `Bn254`.
/// Invariants:
/// - `|value|` never overflows `Bn254`
/// - `|value| < 2^max_bits` and `max_bits ⩽ Fr::CAPACITY - RESERVED_HIGH_BITS`
```

The transcript and the Merkle leaf hash, on the other hand, do not look at the field elements; they pack the raw signed cells in base 2^{31}. The transcript buffers up to `NUM_OBS_PER_WORD = 8` wires and packs them with `pack_base_2_31`, and `hash_babybear_slice_to_digest` packs each leaf chunk the same way with `pack_base_2_31_cells` (base 2^{31}). In both cases the absorbed word is the linear combination:

$$\mathsf{pack}\,(x_0, \ldots, x_{k-1}) = \sum_i x_i \cdot 2^{31i}$$

where each x_i is the raw signed BN254 cell of the i-th wire and $k \leq 8$. Soundness of the transcript and the commitment requires pack to be injective on the BabyBear vectors being absorbed; otherwise distinct field content can produce the same word, and the prover is free to move between the preimages.

The map pack is injective only when each limb x_i is confined to a set of residues of width strictly less than the base 2^{31}. The canonical range $[0, p)$ is such a set: with $p < 2^{31}$, the map is ordinary base-2^{31} positional notation, the limbs never overlap, and distinct BabyBear vectors give distinct integers and hence distinct words. `observe` guarantees nothing of the sort. Two ways the injectivity fails for the wires `observe` accepts:

- *Oversized limbs.* For a wire with `max_bits > 31` the cell can exceed 2^{31} and spill into the next limb's place outright, so the encoding is trivially ambiguous.
- *Signed limbs.* Even within the type's nominal 31-bit width the cell is signed, and the signed range is wider than the base. The key observation is that L has a nontrivial kernel over such limbs: the carry vector $c_i = 2^{31}e_i - e_{i+1}$ (with e_i the i-th unit vector) satisfies:

$$\text{pack}\,(c_i) = 2^{31} \cdot 2^{31i} - 2^{31(i+1)} = 0$$

Adding c_i to a limb vector therefore leaves the absorbed word unchanged while changing the underlying field vector. Hence over the domain `observe` accepts, the transcript and the WHIR Merkle commitment are not binding on field content; the hashing is genuinely not collision-resistant.

Unsound even for max_bits-reduced wires

The natural mitigation is to reduce each wire before observing it. That is not enough. `reduce_max_bits` only reduces a wire whose `max_bits` exceeds 31; it leaves any wire with `max_bits <= 31` untouched, sign included:

```
pub fn reduce_max_bits(&self, ctx: &mut Context<Fr>, a: BabyBearWire) -> BabyBearWire {
    if a.max_bits > BABYBEAR_MAX_BITS { self.reduce(ctx, a) } else { a }
}
```

So a `max_bits`-reduced wire still ranges over the signed interval $(-2^{31}, 2^{31})$, an interval of width 2^{32}, which is *twice* the packing base 2^{31}. The carry vector c_i from above stays inside this box, so the collision survives. Take the two-limb example:

$$v = (-p,\ 0), \qquad w = \left(2^{31} - p,\ -1\right)$$

Here $w = v + c_0$, so both pack to the same value, since:

$$v_0 + v_1 2^{31} = -p = \left(2^{31} - p\right) + (-1) \cdot 2^{31} = w_0 + w_1 2^{31}$$

But coordinate-wise modulo p they are the distinct vectors $(0, 0)$ and $\left(2^{31} - p,\ p - 1\right)$. Both limbs of v and w lie in $\left(-2^{31}, 2^{31}\right)$, so both are legal `max_bits`-reduced inputs to `observe`. The collision is confirmed by cvc5 over the signed-limb model:

```
(set-logic QF_LIA)
(define-fun B () Int 2147483648)  ; 2^31
(define-fun P () Int 2013265921)  ; BabyBear modulus = 0x78000001
(declare-fun v0 () Int) (declare-fun v1 () Int)
(declare-fun w0 () Int) (declare-fun w1 () Int)
(define-fun signed_31 ((x Int)) Bool (and (< (- B) x) (< x B)))
(assert (and (signed_31 v0) (signed_31 v1) (signed_31 w0) (signed_31 w1)))
; the two vectors differ as BabyBear vectors (coordinate-wise mod p)
(assert (or (not (= (mod v0 P) (mod w0 P))) (not (= (mod v1 P) (mod w1 P)))))
; but pack to the same raw base-2^31 value
(assert (= (+ v0 (* v1 B)) (+ w0 (* w1 B))))
(check-sat)
```

```
sat
((v0 (- 2013265921)) (v1 0) (w0 134217727) (w1 (- 1)))
(mod: v = (0, 0)   w = (134217727, 2013265920))
```

Reducing `max_bits` to 31 is therefore not the same as canonicalizing to $[0, p)$, and asserting `max_bits <= 31` (or even `== 31`) is likewise insufficient: neither removes the signed range, and the carry collision survives. Only confining each limb to $[0, p)$, where $p < 2^{31}$ and the limbs cannot

overlap, makes pack injective.

Impact. In the reviewed code these collisions are not reachable. Every wire that actually reaches `observe`, `observe_ext`, or the leaf hash is loaded via `load_witness` and is therefore non-negative in $[0, 2^{31})$; no signed limb and no oversized wire is ever absorbed. The non-canonical-representative weakness that *is* reachable on those non-negative inputs is the subject of the companion finding *"Unreduced BabyBear wires are absorbed into the transcript"*.

The finding here is about the API contract rather than a demonstrated forgery. `observe` accepts an arbitrary `BabyBearWire`, and a signed wire is the documented, legal state of the type. It silently produces a non-binding encoding for such a wire, with no guard and no compile-time or run-time signal that anything is wrong. A future caller that observes a signed wire, or a wire with `max_bits > 31`, would reintroduce the grinding weakness of the companion finding, and, because signed limbs genuinely collide, a true binding failure of the commitment.

Recommendation. The transcript and leaf-hash paths should canonicalize every input to $[0, p)$ before packing, rather than trusting the caller to pass a wire in a particular sub-range. With each limb in $[0, p)$ and $p < 2^{31}$, the limbs cannot overlap and pack is injective, which removes both the signed-limb carry collisions and the oversized-limb overlap. As shown above, neither asserting `max_bits <= 31` nor calling `reduce_max_bits` achieves this: the former keeps the signed range, the latter is a no-op at `max_bits <= 31`.

`reduce` already produces a canonical representative; `signed_div_mod` constrains its remainder to $[0, p)$. The robust fix is to make non-reduced absorption unrepresentable: introduce a `ReducedBabyBearWire` newtype, constructible only through `reduce`, exposing `AsRef<BabyBearWire>` so it can stand in wherever a wire is expected, and have `observe`, `observe_ext`, and the leaf hash accept only this type. The compiler then rejects any attempt to absorb a raw, possibly non-canonical or signed `BabyBearWire`, closing both this finding and its companion at the type level.

Client Response. This issue has been addressed in PR #2848 by introducing reduced BabyBear newtypes and making `observe`, `observe_ext`, and leaf hashing accept only those reduced types. Raw `BabyBearWire`s, including signed or oversized ones, no longer reach the base-2^{31} packing interface.

• LOW · #16

Incorrect Bound Argument in signed_div_mod for Negative Dividends

crates/static-verifier/src/field/baby_bear/base.rs

Description. `signed_div_mod` introduces witnesses `div` and `rem` with `div * b + rem ≡ a (mod p)` and `0 <= rem < b`, where `b` is the BabyBear order. To range-check `div` it relies on the following argument in the proof-of-correctness comment:

```
// Logically we want `div = a // b`. Because (2) and `a` could be negative, `div` could
// be negative. Therefore, we have `|div| = |a // b| = |a| // b < 2^max_bits // b = bound` and
// we can say `shifted_div = div + bound` is in `[0, 2 * bound)`.
```

The middle identity `|a // b| = |a| // b` is false for negative `a`: floor division and absolute value do not commute. Take $a = -1$ and $b = p_{BB}$:

$$\lfloor a/b \rfloor = \lfloor -1/p_{BB} \rfloor = -1, \qquad \lfloor |a|/b \rfloor = \lfloor 1/p_{BB} \rfloor = 0$$

So `|div| = 1` while `|a| // b = 0`; the claimed bound underestimates `|div|`. This is not merely a comment defect, because the code computes `bound` from exactly that quantity:

```
// Constrain that `abs(div) ⩽ 2 ** (2 ** a_num_bits / b).bits()`.
let bound = (BigUint::from(1u32) << (a_num_bits as u32)) / &b;
let shifted_div = range.gate().add(ctx, div, QuantumCell::Constant(biguint_to_fe(&bound)));
range.range_check(ctx, shifted_div, (bound * 2u32 + 1u32).bits() as usize);
```

For a negative value with a tight width, e.g. `a = -1` with `a_num_bits = 1` (which is what `neg(one)` produces: `value = -1`, `max_bits = 1`), we get `bound` $= \lfloor 2^1/p_{BB} \rfloor = 0$. Then `div = -1` and `shifted_div = div + bound = -1`, which the range check to `(2*bound + 1).bits() = 1` bit rejects: it expects `shifted_div` in `[0, 2*bound]`, but `-1` reduces modulo `Fr` to a large element. The constraint is unsatisfiable, so the prover cannot complete `signed_div_mod` for such an input.

Impact. A completeness edge, not a soundness one. `signed_div_mod` backs `reduce`, `assert_zero`, and `assert_equal`, so any of these is unprovable on a small negative value carrying a tight `a_num_bits`. In practice the gadget is reached with `a_num_bits = BABYBEAR_MAX_BITS = 31`, where `bound >= 1` and the negative quotient is covered, so the live code does not appear to hit it; the documented invariant is wrong regardless, and the safe range is narrower than the comment claims.

Recommendation. Derive `bound` so that it covers the floor-division of negative dividends rather than routing through `|a| // b`: the magnitude of `div` $= \lfloor a/b \rfloor$ for $|a| < 2^{a\_num\_bits}$ is at most $\lceil (2^{a\_num\_bits} - 1)/b \rceil$, so use that (or equivalently `bound` $= \lfloor 2^{a\_num\_bits}/b \rfloor$ together with an asserted lower bound `a_num_bits >= BABYBEAR_ORDER_BITS`). At minimum, correct the proof comment and document that the gadget must only be called with `a_num_bits` large enough that `bound` covers the negative quotient.

Client Response. This issue has been addressed in PR #2860 by changing the quotient bound to `ceil((2^a_num_bits - 1) / b)`. The tight negative case is now inside the shifted-div range check.

LOW · #17

native_xorin and the sha3 Fork Accept Sponge Rates the XORIN Circuit Cannot Prove

extensions/keccak256/guest/src/lib.rs, extensions/keccak256/circuit/src/xorin/air.rs

Description. The `XORIN` circuit can absorb at most 136 bytes per call: its columns are sized to `KECCAK_RATE_BYTES = 136` (34 four-byte chunks), and the AIR enforces `len = 4 * (#non-padding) <= 136`. A single `XORIN` therefore supports any per-block absorb length in `{0, 4, ..., 136}`. The limit is purely the absorb width; `KECCAKF` permutes the full 200-byte state, so the permutation itself is rate-agnostic.

`native_xorin` is an exported low-level intrinsic whose documented contract only mentions pointer validity. It rounds `len` up to a multiple of four (`adjusted_len = len.next_multiple_of(4)`) and emits the instruction with no check that `adjusted_len <= 136`. A caller passing `len = 140` emits `len = 140`, which the circuit cannot represent: pure execution proceeds, then preflight or trace generation panics (140 bytes need 35 chunks versus 34 slots). The patched RustCrypto `sha3` fork routes all generic block absorption through `native_xorin(state, block, block.len())`, so honest use of any variant whose rate exceeds 136 bytes calls the intrinsic with `len > 136`.

A sponge function is circuit-provable through this path if and only if its rate is at most 136 bytes (state 1600 bits = 200 bytes; rate = 200 - capacity):

FUNCTION (FAMILY)	CAPACITY (BITS)	RATE (BYTES)	XORIN-PROVABLE
Keccak-512 / SHA3-512	1024	72	yes
Keccak-384 / SHA3-384	768	104	yes
Keccak-256 / SHA3-256 (intended target)	512	136	yes
SHAKE256 / cSHAKE256 / KMAC256 (256-bit XOF)	512	136	yes
Keccak-224 / SHA3-224	448	144	no (>136)
SHAKE128 / cSHAKE128 / KMAC128 / K12 (128-bit XOF)	256	168	no (>136)

So the dividing line is capacity at least 512 bits, equivalently rate at most 136. The unprovable set is the 128-bit-security XOF family (rate 168) plus SHA3-224 / Keccak-224 (rate 144).

Impact. Low (honest-prover completeness and API compatibility, not soundness). Honest guest programs using the affected low-level API, or affected RustCrypto `sha3` APIs under the OpenVM fork, can execute but fail to prove (preflight or trace-generation panic). Note the limit is the absorb

width, not the variant per se: SHA3-384 and SHA3-512 (rates 104 and 72) are provable through the fork, while SHA3-224 and SHAKE128 are not.

Recommendation. Never emit an unprovable `XORIN`, and never fail late. Harden `native_xorin` to reject `len.next_multiple_of(4) > 136` at the API boundary, so misuse fails loudly there rather than deep in trace generation, and document the alignment and length limit. For sponge integrations whose rate exceeds 136 (the `sha3` fork's SHA3-224, SHAKE128, cSHAKE128, and so on), fall back to a software absorb for the oversized block while still using the native `KECCAKF` for the permutation; since the permutation dominates proving cost, the de-accelerated absorb is nearly free and the variant stays usable. Preferred ordering of options: software fallback over compile-time error over runtime panic.

Client Response. The issue has been fixed in PR #2845, PR #13 of the hashes fork, and PR #7 of the tiny-keccak fork. `native_xorin` explicitly rejects lengths over the limit, and the patched hash functions do not emit `XORIN` instructions exceeding it.

LOW · #18

tiny-keccak v384 and v512 Panic at Runtime on the zkVM Target Despite Being Circuit-Supportable

openvm-org/tiny-keccak src/keccak.rs

Description. On `target_os = "zkvm"`, the `openvm-org/tiny-keccak` fork rejects every non-256 variant at runtime in `Keccak::new`:

```
#[cfg(target_os = "zkvm")]
fn new(bits: usize) -> Keccak {
    if bits ≠ 256 {
        panic!("Only Keccak256 is supported for ZKVM");
    }
    ...
}
```

So the public constructors `Keccak::v384()` and `Keccak::v512()` still compile cleanly for the zkVM target but panic when the guest actually runs. Crucially, these two variants have rates 104 and 72 bytes respectively, both at most 136, so they are within the `XORIN` circuit's absorb capability (see the related rate finding): the limitation is an artifact of the fork wrapping the fixed-rate, fixed-output `Keccak256` type, not a circuit constraint. (Keccak-224, rate 144 > 136, is genuinely unprovable and is covered separately.)

Impact. Low (API compatibility and late, runtime failure). A guest constructing `Keccak::v384()` or `Keccak::v512()` builds successfully and then panics at execution time, a target-specific failure that does not exist on the host build and surfaces only when the program is run or proved. For these circuit-capable rates the failure is doubly unfortunate, because the variant could have been supported.

Recommendation. Since `v384` and `v512` fit the `XORIN` width (rates 104 and 72 at most 136), the proper fix is to support them by absorbing at their own rate instead of hardcoding Keccak-256's 136. If the fork chooses not to implement them, it should at least fail at compile time rather than runtime: on the zkVM target, `#[cfg]`-gate the constructors so that using `Keccak::v384()` is a compile error rather than a runtime panic. (For the genuinely unprovable `v224`, rate 144, use the software-absorb fallback recommended in the related rate finding, not a compile-time rejection, since it remains a usable hash.)

Client Response. The issue has been fixed in PR #7 of the tiny-keccak fork. When a variant's absorb rate exceeds the `XORIN` width, the block is now absorbed in chunks. This allows all variants, even `v224`, to use the `XORIN` intrinsic instead of being rejected.

● INFORMATIONAL · #19

Round-by-round soundness calculations

SWIRL whitepaper & stark-backend/crates/stark-backend/src/soundness.rs

Description. The SWIRL whitepaper and soundness calculator in `soundness.rs` treat the GKR fractional sumcheck and the ZeroCheck as fully independent protocols, respectively computing their error probabilities and security bits separately. The paper takes a maximum over the error probabilities and, accordingly, the soundness calculator takes a minimum across all components (`soundness.rs:143-149`). To avoid confusion, we will only write in terms of error probabilities (and therefore taking the maximum when rounds are independent).

In contrast to the above calculations, several challenges are shared or co-derived, which means their error probabilities should be combined via a union bound (summation of the failure probabilities) rather than treated independently. We identify three specific issues:

1. **Shared challenge `xi` between GKR and ZeroCheck.** The evaluation point `xi` produced by `verify_gkr()` (`batch_constraints.rs:97-98`) is reused directly as the evaluation point for the ZeroCheck. A malicious prover succeeds if `xi` is a bad challenge for *either* protocol. Therefore, the correct error for this round is a union bound over the probability of getting a "bad" challenge for the GKR round and for the ZeroCheck.
2. **Incremental construction of `xi`.** The point `xi` is not sampled atomically. It is built incrementally during the GKR sumcheck: each layer's sub-rounds sample one coordinate via the `mu` challenges (`fractional_sumcheck_gkr.rs:108` , `fractional_sumcheck_gkr.rs:144`). If `n_global > n_logup` , additional coordinates are sampled afterward (`batch_constraints.rs:104-106`). This incremental construction complicates the round-by-round analysis: the GKR soundness for each sub-round is entangled with the eventual use of `xi` in the ZeroCheck, and the current soundness model does not account for this. We do not know how to prove round-by-round soundness in this case.
3. **`lambda` and final coordinates of `xi` are sampled from the same transcript state.** When `n_global > n_logup` , the final coordinates of `xi` and `lambda` are derived from the same transcript state with no intervening prover message. They cannot be treated as separate rounds in an RBR analysis; their error probabilities should be combined via a union bound.

Impact. The RBR error in certain rounds is slightly higher than shown in the paper and soundness calculator. In particular, (1) above approximately doubles the error associated with the sampling of `xi` (1-bit security loss). Similarly, assuming that the errors associated with `xi` and `lambda` are comparable, taking a union over both events as in (3) leads to another 1-bit reduction in security level. The impact of (2) is yet to be determined; it is unclear to us whether existing theory is enough to give RBR soundness guarantees or to indicate a possible attack.

Recommendation. With respect to (1) and (3), we recommend revising the RBR soundness analysis. (2) is likely to warrant further investigation.

Client Response. The client acknowledged the issue and presented our team with a new draft of the SWIRL paper updating the relevant theorems. In particular, the revised theorems account for the incremental construction of `xi`, the multiple uses of the `xi` value and the fact that `lambda` is sampled alongside the final coordinates of `xi`. The soundness calculator was updated to reflect the paper changes in PR #357.

INFORMATIONAL · #1A

Recursive verifier reuses stacking bus for distinct messages

openvm/crates/recursion/src/batch_constraint/sumcheck/,
openvm/crates/recursion/src/stacking/opening/air.rs

Description. The recursive verifier uses `stacking_module_bus` to carry the transcript index from the batch-constraint sumcheck module into the stacked-opening module. The bus message contains only a single field, `tidx`, so the consumer distinguishes messages only by the value of the transcript index and by the global balance of sends and receives.

The same bus is used for two semantically different handoffs:

- `UnivariateSumcheckAir` sends the transcript index immediately after the univariate sumcheck round, i.e. the initial `tidx` for `MultilinearSumcheckAir`.
- `MultilinearSumcheckAir`, when it has non-zero rounds, receives that `tidx`, performs its transcript observations and challenges, and sends the transcript index immediately before column openings.
- `StackingOpeningAir` receives the transcript index immediately before column openings.

This overloading is done to enable the zero-round multilinear sumcheck case. When `MultilinearSumcheckAir` has no rounds, its trace contains a dummy row and its `stacking_module_bus` receive/send constraints are disabled. In that case, the message sent by `UnivariateSumcheckAir` is consumed directly by `StackingOpeningAir`, because the transcript index before the multilinear sumcheck is also the transcript index before column openings.

There is currently no concrete message-confusion attack. If the prover tries to swap the non-zero-round and zero-round handoffs, the surrounding transcript constraints cause some transcript operations to be consumed twice or not consumed at all. Still, the bus does not encode which module boundary a message belongs to, so the intended protocol sequencing is implicit rather than reflected in the bus domain.

Recommendation. Consider adding semantic separation between the two handoffs. For example, `MultilinearSumcheckAir` could always receive the `tidx` from `UnivariateSumcheckAir`, including on rows where `is_dummy = 1`, and then forward the same `tidx` to `StackingOpeningAir` on a distinct bus when there are zero multilinear rounds. This would make the protocol stages explicit while preserving the current zero-round behavior.

Client Response. The client has decided not to address this comment.

INFORMATIONAL · #1B

Unnecessary and redundant constraints in recursive verifier

`openvm/crates/recursion/src/*`

Description. While auditing the recursive verifier AIRs, several constraints were found to be redundant: they are either fully implied by other constraints in the same AIR or by stricter constraints that fire on a superset of rows. Removing them does not weaken the AIR but reduces the constraint count (and therefore prover work and verifier degree-bounding cost).

The redundant constraints identified so far are listed below. Each item names the location, restates the constraint, and gives the reason it is implied.

1. **GKR AIRs - transition-gated dummy-row constraints** (`crates/recursion/src/gkr/layer/air.rs:153-159` , `crates/recursion/src/gkr/sumcheck/air.rs:171-178` , `crates/recursion/src/gkr/xi_sampler/air.rs:115-122`). In each of the three AIRs the same pair of transition constraints on `local.is_dummy` is already implied by the singleton-row constraints immediately below them (`is_dummy` -> `is_first=1` and `is_last=1` , plus the analogous round/layer flags in the sumcheck case), which already make any dummy row a singleton segment with no internal transition.
2. **Univariate eq-AIR - boolean check on `local_is_last`** (`crates/recursion/src/batch_constraint/eq_airs/eq_uni/air.rs:95-96`). `assert_bool(next.is_first + not(next.is_valid))` only excludes `is_first = 1` and `is_valid = 0` , which is already unreachable via the existing `is_first` -> `is_valid` invariant.

Client Response. This issue has been acknowledged, and the client has decided not to address it for the moment.

INFORMATIONAL · #1C

OUTPUT relies on the guest convention that OutputKey came from CALL

extensions/deferral/circuit/src/output/air.rs

Description. The `OUTPUT` opcode enforces the local relation "these raw bytes hash to this supplied `OutputKey`", but it does not prove that the supplied `OutputKey` was previously produced by a `CALL`. The intended usage is documented: the guest calls `deferred_compute`, receives an `OutputKey`, and then passes that same key to `get_deferred_output` (`docs/vocs/docs/pages/book/acceleration-using-extensions/deferral.mdx:11-12`, `docs/vocs/docs/pages/specs/openvm/isa.mdx:448`). In the AIR, however, `CALL` folds the returned `output_commit` into the output accumulator (`extensions/deferral/circuit/src/call/air.rs:157-181`), while `OUTPUT` independently proves only that `(deferral_idx, output_len, output_raw)` reconstructs the supplied `output_commit` (`extensions/deferral/circuit/src/output/air.rs:193-223`). The `DeferralCircuitCountBus` validates the deferral index, but not that this particular key came from a prior `CALL`. In addition, `OutputKey::new` is public in `extensions/deferral/guest/src/lib.rs:32-39`.

Impact. This is primarily a guest API contract footgun. A guest that manually constructs or accepts an `OutputKey` from untrusted input can call `OUTPUT` and authenticate bytes against that arbitrary key without proving that the key came from a deferred computation. If all trusted code uses the intended pattern:

```
let key = deferred_compute::<IDX>(&input_commit);
get_deferred_output::<IDX>(&mut output, &key);
```

then the key comes from `CALL` and the deferral accumulators bind it to the deferred computation path.

Recommendation. Either document very explicitly that manually constructed `OutputKey`s are not trusted deferral results, or harden the circuit/API so `OUTPUT` consumes a key that is linked to a prior `CALL`. A circuit-level fix would require a lookup/bus relation such as `CALL` sending `(deferral_idx, output_commit, output_len)` and `OUTPUT` receiving it, with a clear multiplicity rule for whether outputs can be read once or multiple times.

Client response. The client indicated that deferral opcode usage should be heavily restricted to the provided guest libraries. Extensive documentation has been added in PR #2866.

INFORMATIONAL · #1D

verify-stark API and hygiene notes

guest-libs/verify-stark/circuit/src/commit/air.rs

Description. We observed several low-risk verify-stark API and hygiene issues.

- `UserPvsCommitValuesAir::new` relies on `debug_assert!` for configuration shape checks. In `guest-libs/verify-stark/circuit/src/commit/air.rs:35-48`, the constructor expects `num_user_pvs >= DIGEST_SIZE`, `num_user_pvs % DIGEST_SIZE == 0`, and `(num_user_pvs / DIGEST_SIZE).is_power_of_two()`, but these checks are debug-only. In release builds, malformed configurations could pass construction and fail later in less obvious ways.
- `DeferredVerifyCircuitProver`'s generic `DeferralCircuitProver::prove` path always uses `prove_no_def`. In `guest-libs/verify-stark/circuit/src/prover/mod.rs:253-270`, `prove(&self, input_bytes)` decodes a `VmStarkProof` and calls `self.prover.prove_no_def(...)`. This means the trait path does not appear to support proving child STARKs that themselves require deferral Merkle inputs, even though the verifier machinery has deferral-aware concepts. This may be intended, but then the trait path should be documented as no-deferral-only.
- `row_idx_flags` in `UserPvsCommitValuesAir` appears dead. `guest-libs/verify-stark/circuit/src/commit/air.rs:80-83` slices `row_idx_flags` from the row after `MerkleTreeCols`, but the AIR width is exactly `MerkleTreeCols::<u8>::width()` at lines `62-65`, and `debug_assert_eq!(row_idx_flags.len(), 0)` at line `102` confirms it is always empty.

Impact. These do not appear to be soundness issues. They can cause configuration failures to be delayed or opaque, can surprise users who expect the verify-stark deferral prover trait path to support child proofs with deferrals, and leave minor dead code in the AIR.

Recommendation. Replace the debug-only configuration checks with ordinary assertions or fallible constructor errors. Document whether `DeferralCircuitProver::prove` is intentionally no-deferral-only for verify-stark, or add a deferral-aware trait path if that use case is supported. Remove the dead `row_idx_flags` slice.

Client response. The `DeferredVerifyCircuitProver::prove` item is present in the audited snapshot at commit `f01c912`, but it had already been fixed on another development line by PR #2717 and thus it is fixed in the later rc branches. The remaining API/hygiene notes have been addressed in PR #2837.

INFORMATIONAL · #1E

Defense-in-Depth Observations and Nits

crates/static_verifier/src

Description. This finding collects some low-impact observations made during the review. None of them are soundness issues in the code as it stands; they are defense-in-depth hardening, contract hygiene, and code-quality suggestions, grouped by theme.

Canonical form and range invariants

The core soundness problem in this area, that non-canonical 31-bit BabyBear representatives are absorbed into the transcript and Merkle leaves, is written up separately as a Medium finding ("Non-canonical BabyBear representatives produce multiple transcript and Merkle encodings of the same field element"). The remaining items here are orthogonal hardening:

- `pack_base_2_31_cells` (`hash/poseidon2.rs:204`) does not assert `values.len() <= MULTI_FIELD32_NUM_F_ELMS`. With too many limbs the base-`2^31` packing exceeds the field and stops being an injective encoding; I recommend asserting the bound (the canonicality of each limb is covered by the Medium finding above).
- `load_base_baby_bear_decomposition_witness` (`transcript/mod.rs:74`) constructs `BabyBearWire { value: digit, max_bits: BABY_BEAR_BITS }` and only range-checks the digits afterwards in `constrain_base_baby_bear_decomposition` (which does enforce `< BABY_BEAR_MODULUS_U64`, so these particular wires end up canonical). The type's range invariant should nonetheless hold by construction: I suggest returning the raw `AssignedValue`s here and only minting a `BabyBearWire` where a range check accompanies it, so the type is never observable in an unchecked state.
- `hash_babybear_slice_to_digest` (`hash/poseidon2.rs:219`) is an unpadded sponge with no length domain separation, so it is not collision-resistant across variable-length inputs: trailing zero limbs are absorbed identically, giving `Hash(a) = Hash(a || 0)` when the extra zero falls in the same rate block. The current callers hash fixed-shape data, so this is latent; worth a note (or a length tag) to avoid future misuse.

Type-safety preconditions

- `BabyBearChip::select` (`base.rs:340`) and `BabyBearExt4Chip::select` (`field/baby_bear/extension.rs`) take the condition as a raw `AssignedValue<Fr>` and forward it to `gate().select`, which is only a branch selection when `cond ∈ {0,1}`. The wrapper adds no boolean constraint; soundness rests on the caller having constrained `cond`. In-tree callers do (bits from `num_to_bits`, or `SafeBool`), so there is no live bug, but the API is unsafe. I recommend taking a `SafeBool` (or a boolean new-type) for `cond`.

- `eval_var` (`stages/batch_constraints/mod.rs:479`) maps a symbolic entry's rotation offset with `if offset == 0 { value.local } else { value.next }`, so any offset `>= 2` silently resolves to `value.next`. This is correct only for AIRs whose rotations are limited to current/next. I suggest a typed `Offset` enum (`Cur` / `Next`) or a `match` with `_ => unreachable!()` so an unexpected offset fails loudly.

API and contract hygiene

- `prove_verify_stark_constraints_only` (`prover.rs:99`) proves with `instance_columns == 0`, so it does not export the verifier's public inputs and cannot be used as a real verifier; it exists for integration tests. The constraints-only path should be clearly marked test-only (in name and/or docs) so it is not mistaken for a statement-binding proof.

Suggested refactors and code-quality nits

These are stylistic and do not affect correctness:

- `gkr-proof-wire` (`GkrProofWire`): `claims_per_layer` is always four claims; `Vec<[BabyBearExtWire; 4]>` would encode that.
- `whir-proof-wire-struct` (`WhirProofWire`): `whir_sumcheck_polys` always has 2 evals per round; `Vec<[BabyBearExtWire; 2]>` would encode that.
- `verify-final-residual`: `let r = ext_chip.sub(final_acc, final_claim); assert_equal(r, zero)` is just `ext_chip.assert_equal(final_acc, final_claim)`.
- `query-root-from-bits-helper`: the inline "select a power of `omega` or one, then wrap in a `BabyBearWire`" should be a `select_constant` helper on the BabyBear chip rather than open-coded.
- `binary-k-fold-loop`, `eval-lagrange-on-integer-grid`, `eval-eq-mle-ef-f-assigned`: each open-codes an extension-field +/- base-field operation by mutating `.0[0]`; these belong behind dedicated `Ext ± Base` methods on the extension chip.
- `eval-eq-mle-assigned`: the cost comment is inaccurate, since `y` is also an extension-field element.
- `tree-compress-helper`: takes `ext_chip: &BabyBearExtChip` but does nothing field-specific; it should take `gate: &impl GateInstructions<F>`.
- `invert-base-helper`, `ext-to-coeffs-helper`: these helpers logically belong in the `baby_bear` module.
- `digest-to-fr-helper`: could be an `Into` implementation.
- `babybear-ext4-from-base-const`: the explicit caching is redundant; the base layer already caches in `BabyBearChip::load_constant`.
- `poseidon2-babybear-rate-ratio`: the rate constants are defined by division; defining `BABY_BEAR_PER_BN256` and deriving the rate from it reads more clearly.

- `constrain-batch-gkr-cubic-subrounds`: the repeated cubic sumcheck rounds could share a sumcheck subroutine.
- `wrapper-generate-circuit-object-for-proving-block`: introduce a named constant for the accumulator size (`12`).
- `BabyBearChip::special_inner_product` can be simplified by noting that `range` and `other_range` are always equal. The function actually computes the `s`-th term of a convolution. The function also hardcodes a magic number rather than using the extension degree (constant).

Client Response. This issue has been addressed by PRs #2868 and #2909, which cover the concrete defense-in-depth items: packing bounds, typed boolean selects, rotation checks, fixed proof shapes, test-only prover cleanup, and delayed `BabyBearWire` construction until after range checks. The finding remains partially resolved because the length-agnostic `hash_babybear_slice_to_digest` note and some style refactors are still open.

INFORMATIONAL · #1F

Performance Observations and Nits

crates/static-verifier/src

Description. This finding collects some low-impact observations made during the review that may improve performance.

Unused and duplicate code

Some sections of the code are either unused, unreachable, or duplicated:

- Functions for packing BabyBear elements into `Fr` elements appear twice: `pack_base_2_31` (in `transcript.rs`) and `pack_base_2_31_cells` (in `poseidon2.rs`). The only difference is whether the input is the assigned value or the wire. For both of these functions, the caller has access to both representations.
- `add`, `sub`, `mul`, `mul_add` in `BabyBearChip` all implement an additional `max_bits` check on the output `c`, even though this is already guaranteed not to overflow.
- The `select` method of `Poseidon2State` is unused.

The duplicate packing and redundant output checks produce additional and unnecessary constraints.

Equality checks

The chips `BabyBearChip` and `BabyBearExt4Chip` both expose `assert_zero` and `assert_equal` functions, where the latter works by performing a subtraction and an internal call to `assert_zero`.

However, we have observed the following patterns in the code:

- Using `assert_equal` with a fixed 0 instead of using `assert_zero` (example 1, example 2).
- Performing a subtraction followed by an `assert_equal` with a fixed 0, instead of using `assert_equal` in the first place (example 1, example 2, example 3).

Operand ordering affects number of constraints

`add` and `sub` in `BabyBearChip` are cheaper if the first operand is larger. `mul` avoids this with a `mem::swap` pattern. This same pattern could be replicated in `add` and `sub` to potentially save on reduce operations.

Client Response. This issue has been addressed by PR #2868, which resolves the duplicate packing, unused selector, redundant output reductions, and equality-check items. The `add` / `sub` operand-ordering optimization was implemented in #2868 but reverted in PR #2909 for clarity, so that suggestion was not adopted.

INFORMATIONAL · #20

BabyBear Reduce Does Not Assert Its max_bits Precondition

crates/static-verifier/src/field/baby_bear/base.rs

Description. The soundness of `signed_div_mod` rests on an upper bound on the input width. `BabyBearWire` documents the invariant, and `signed_div_mod` repeats it as an explicit assumption:

```
/// ## Assumptions
/// * `a_max_bits < F::CAPACITY = F::NUM_BITS - RESERVED_HIGH_BITS`
///   * Unsafe behavior if `a_max_bits ≥ F::CAPACITY`
```

The overflow argument in `signed_div_mod` (that `|div * b + rem|` does not wrap modulo `Fr`) depends on this bound. Yet neither `reduce` nor `signed_div_mod` enforces it. `reduce` forwards `a.max_bits` straight through, checking only the *value* under a debug-only guard:

```
pub fn reduce(&self, ctx: &mut Context<Fr>, a: BabyBearWire) -> BabyBearWire {
    guarded_debug_assert!(fe_to_bigint(a.value.value()).bits() as usize ⩽ a.max_bits);
    let (_, r) = signed_div_mod(&self.range, ctx, a.value, a.max_bits);
    ...
}
```

and `signed_div_mod` itself only asserts `a_val.bits() <= a_num_bits`, never `a_num_bits <= Fr::CAPACITY - RESERVED_HIGH_BITS`. Recall that `assert_zero`, which performs the same shifted-quotient range check, *does* guard the width:

```
assert!(a.max_bits ⩽ Fr::CAPACITY as usize - RESERVED_HIGH_BITS);
```

So a wire carrying an oversized `max_bits` tag flows into `signed_div_mod` through `reduce` without the gadget failing closed.

Impact. Defense in depth. `max_bits` is circuit-generation metadata rather than prover-controlled witness data, and no in-tree caller constructs a `BabyBearWire` violating the invariant, so there is no concrete attack. The concern is that `reduce` and `signed_div_mod` are public over the field module and do not reject an input that breaks the assumption their soundness relies on.

Recommendation. Add the same guard `reduce` (and/or `signed_div_mod`) already documents, mirroring `assert_zero`:

```
assert!(a.max_bits ⩽ Fr::CAPACITY as usize - RESERVED_HIGH_BITS);
```

Client Response. This issue has been addressed in PR #2860, which adds the precondition assertion in `signed_div_mod`, and in PR #2868, which adds the same guard in `reduce`. PR #2909 removes a duplicate assertion, leaving both checks in final code.

INFORMATIONAL · #21

Sampling Zero Bits Skips the Sponge Transition and Can Desync From the Native Transcript

crates/static-verifier/src/transcript/mod.rs

Description. `sample_bits` short-circuits when asked for zero bits:

```
pub fn sample_bits(&mut self, ctx: &mut Context<Fr>, bits: usize) → AssignedValue<Fr> {
    ...
    if bits == 0 {
        return ctx.load_zero();
    }
    let sampled = self.sample(ctx);
    ...
}
```

Recall that a real sample goes through `sample`, which first flushes any buffered observations and then squeezes the sponge:

```
pub fn sample(&mut self, ctx: &mut Context<Fr>) → BabyBearWire {
    if let Some(val) = self.sample_buf.pop() { return val; }
    self.flush_observe_buf(ctx);
    let squeezed = self.sponge_squeeze(ctx);
    ...
}
```

`sponge_squeeze` is what flips the duplex sponge from absorb to squeeze mode: when `absorb_idx != 0` it permutes the state and resets `absorb_idx` / `sample_idx`. The zero-bit path does neither; it does not flush `observe_buf`, does not call `sponge_squeeze`, and returns a freshly loaded zero. The native out-of-circuit transcript performs the squeeze transition on every sample (the sponge tracking here is documented to "match `DuplexSponge::absorb/squeeze`"). Hence an in-circuit `sample_bits(0)` issued while the sponge is still in absorb mode (or with observations buffered) leaves the in-circuit Fiat-Shamir state one transition behind the native verifier, and every challenge drawn afterwards diverges.

Impact. Informational. `sample_bits(0)` is not reached in the current verifier with a non-trivial sponge state: `check_witness` already returns early for `bits == 0` before it would sample, and no other call site requests zero bits. The behavior cannot produce a soundness issue as the code stands; I am flagging it as a latent transcript-fidelity gap that would bite if a future call site sampled zero bits mid-transcript.

Recommendation. Make the zero-bit path perform the same sponge transition as a genuine sample, flushing the observe buffer and squeezing once (discarding the result), so the in-circuit transcript matches the native one unconditionally; alternatively assert `bits > 0` if zero is never an intended argument.

Client Response. This issue has been addressed in PR #2868 by sampling before the zero-bit early return. `sample_bits(0)` now runs the same `sample` path as a non-zero request, keeping the transcript in sync.

INFORMATIONAL · #22

signed_div_mod proof intuition does not match the code

crates/static-verifier/src/field/baby_bear/base.rs

Description. `signed_div_mod` is a crucial component of the field arithmetic emulation. It is also one of the most complex functions in the `BabyBearChip`. The documentation comment justifies why this function is mathematically sound. As noted in a previous finding, some of the claims there are not correct. Furthermore, the code does not actually implement the checks written in the documentation string.

Impact. As it stands, it is unclear why the implemented function is sound.

Recommendation. Revise the documentation comment to match the implemented checks.

Client Response. This issue has been addressed by PR #2860, which rewrites the `signed_div_mod` proof comment to match the implemented checks. Additionally, PR #2909 addresses our follow-up comment by replacing the stale `assert_zero` cross-reference with an exact-remainder explanation.

INFORMATIONAL · #23

Formal-Verification Theorems Do Not Assume a Uniform Extraction-Derived Constraint Interface

`openvm-fv VmExtensions/{Extraction,Constraints,Soundness}`

Description. The top-level SHA-2 and Keccak soundness theorems in the `openvm-fv` repository are conditional on hypotheses stating that the relevant AIR constraints and bus interactions hold, but those hypotheses are not presented through one uniform, extraction-native interface across AIRs. The raw extraction files provide a mechanical low-level surface (per-row predicates `Foo.extraction.constraint_i` and a bus predicate `Foo.extraction.constrain_interactions`), yet the theorem-facing assumptions are aggregated ad hoc per AIR:

- SHA-2 uses handwritten structures (`mainTraceConstraints`, `blockHasherConstraints`) whose fields store extracted interactions and row or group constraints;
- the Keccak permutation and `XORIN` define extracted row lists and `allHold` predicates bridged to simplified theorem-facing predicates via `rfl`;
- `KeccakfOp`'s main theorem consumes `allHold_simplified`, packaging extracted interactions with a simplified `row_constraint_list`.

This is not evidence of a false theorem; the bridges are mostly kernel-checked and many are definitional. The problem is auditability: a reviewer must inspect each AIR's bespoke aggregation layer to confirm the top-level theorem assumes exactly the extracted Rust AIR constraints and interactions, and not a strengthened, weakened, omitted, duplicated, or otherwise ad-hoc set. A soundness theorem stated over a simplified aggregate can be true while silently failing to apply to the actual extracted AIR if the aggregate diverges from the raw extraction output.

Impact. Informational (formal-verification hygiene and auditability). The risk of FV integration mistakes is increased and audit review is made materially harder. The guarantee a reader most wants, namely that a theorem's hypotheses are precisely the extracted AIR's constraints and interactions, is not directly expressible against the current interfaces.

Recommendation. Provide a uniform final theorem or corollary per verified AIR whose assumptions are entirely derived from raw extraction output: define a canonical extraction-level aggregate (for example `Foo.extraction.constrain_row_all := constraint_0 and ... and constraint_N`) and state the public soundness corollary against bounded extraction hypotheses (`constrain_interactions` plus `for all row <= air.last_row, constrain_row_all air row`). Simplified constraints and semantic bundles can remain internal proof conveniences, but the auditor-facing theorem should have extraction-native assumptions. Prefer bounded row quantification over unbounded `for all row`, since the verifier only enforces constraints over the finite trace.

Client Response. The client has acknowledged the recommendation and is evaluating it for adoption in a future update.

INFORMATIONAL · #24

Documented ptr_max_bits Precondition Is Not Asserted in SHA-2 and Keccak XORIN

extensions/sha2/circuit/src/sha2_chips/main_chip/air.rs, extensions/keccak256/circuit/src/xorin/air.rs

Description. Pointer bounding in the SHA-2 main chip and the Keccak `XORIN` chip is done by range-checking only the top limb of the pointer: `high_limb * 2^(32 – ptr_max_bits) < 2^8`. For `ptr_max_bits >= 24` this correctly enforces `pointer < 2^ptr_max_bits`. For `ptr_max_bits < 24` it only forces the top byte toward zero, i.e. it enforces the weaker bound `pointer < 2^24`, under-constraining the pointer relative to the documented precondition.

`KeccakfOpAir` asserts `ptr_max_bits >= RV32_CELL_BITS * (RV32_REGISTER_NUM_LIMBS – 1)` at construction time. `Sha2MainAir` (`eval_instruction`) and `XorinVmAir` document the same precondition but do not assert it. The shipped default (`POINTER_MAX_BITS = 29`) satisfies the precondition, so honest configurations are safe.

Impact. Informational. Honest pointers always satisfy the precondition, so there is no completeness impact, and the shipped default configuration is sound. The issue is a latent, silent soundness weakening that would only manifest if a non-default configuration set `ptr_max_bits < 24`, in which case the pointer would be under-constrained rather than bounded as documented.

Recommendation. Add the same construction-time assertion that `KeccakfOpAir` uses to `Sha2MainAir` and `XorinVmAir`, so the documented precondition is enforced rather than assumed. Consider also asserting an explicit upper bound, since `ptr_max_bits >= 31` raises a separate field-aliasing concern.

Client Response. The issue has been addressed in PR #2845 with explicit assertions in `Sha2MainAir` and `XorinVmAir`, as recommended.

INFORMATIONAL · #25

SDK App Keygen Does Not Mark Required System AIRs and verify_segments Omits the Boundary Check

crates/sdk/src/keygen/mod.rs, crates/vm/src/arch/config.rs, crates/vm/src/arch/vm.rs

Description. The architecture designates four system AIRs as required: `SystemConfig::is_required_air_id` returns true for `PROGRAM`, `CONNECTOR`, `BOUNDARY`, and `MERKLE`. The backend enforces requiredness as a proof-shape check (the verifier asserts `vk.is_required` implies `is_air_present`, via `ProofShapeVDataError::RequiredAirNoVData`), so a properly keyed verifying key rejects proofs that omit any of those AIRs.

`VirtualMachine::new_with_keygen` honors this, calling `add_required_air` exactly when `is_required_air_id(air_id)`. The SDK path does not: `AppProvingKey::keygen` calls `app_engine.keygen(&airs)` directly, and the default `StarkEngine::keygen` does `for air in airs { add_air(air) }`, never `add_required_air`. So the SDK app verifying key marks nothing required, and the backend's `is_required` enforcement does not apply.

`verify_segments` partially compensates, but asymmetrically: it explicitly checks `program_air_present`, `connector_air_present`, and `merkle_air_present`, but has no `boundary_air_present` check; the boundary AIR falls into the generic branch that only asserts empty public values. So three of the four required system AIRs are re-checked and the boundary is conspicuously omitted. Under an SDK-generated verifying key, boundary-AIR presence is enforced by neither the backend `is_required` flag (not set) nor `verify_segments` (not checked); it rests solely on memory-bus balance. A follow-up proof-of-concept confirmed that a degenerate terminate-only (no-memory) execution can omit `PersistentBoundaryAir` and pass both `engine.verify` and `verify_segments`.

Impact. Informational; no current security impact. A boundary-omitting proof can only exist for a no-memory execution: any memory access (registers are address space 1 on the same bus) leaves the memory bus unbalanced, which the GKR/LogUp zero-sum check rejects. A no-memory execution forces `initial_root = final_root = default_root`, so `exe_commit = H(program_commit, default_root, initial_pc)` matches only a trivial app with default initial memory; for any real app the degenerate proof's `exe_commit` does not match and is rejected. So no meaningful statement can be forged. The finding is a deviation from the architecture's intended proof-shape invariant: soundness here currently relies on bus balance rather than the explicit required-AIR mechanism the design specifies, which is exactly the kind of latent gap that becomes load-bearing if a future change weakens the bus argument or introduces a self-balancing system AIR.

Recommendation. Two independent halves, both warranted:

1. Root cause: make SDK app keygen mirror `new_with_keygen`, building via `MultiStarkKeygenBuilder` and calling `add_required_air` whenever `config.as_ref().is_required_air_id(air_id)`. Add a regression test asserting that `AppProvingKey::keygen(...)` marks Program, Connector, Boundary, and Merkle as required.

2. Defense in depth: add an explicit `boundary_air_present` check to `verify_segments`, matching the existing Program, Connector, and Merkle checks, so the host-level system-AIR set is symmetric.

Client Response. The observation has been acknowledged and addressed in PR #2847, with SDK app keygen now mirroring `new_with_keygen` and marking the required AIRs, and `verify_segments` explicitly checking that the AIRs are present.

KNOWN ISSUE · #26

Underconstrained pairing hint allows proof forgery

openvm/guest-libs/pairing/src/*/pairing.rs

Description. OpenVM's BN254 and BLS12-381 pairing checks in the guest-libraries implement the Novakovic-Eagon optimization. The main idea of the optimization is that, since verifying $e(P, Q) = 1$ is dominated by the final exponentiation $f^{(p^{12}-1)/r} = 1$ (where f is the output of the Miller loop and p, r are the base-field characteristic and the prime subgroup order of the curve, respectively), we can avoid this operation in favor of a much cheaper algebraic identity verified against a prover-supplied hint. The prover sends a witness (c, u) and the verifier checks the small-exponent equation $f \cdot u = c^{\lambda}$, where λ is the optimal exponent for the curve ($\lambda = 6x + 2 + q^3 - q^2 + q$ for BN254, $\lambda = q - x$ for BLS12-381) and is several orders of magnitude smaller than $(p^{12} - 1)/r$. The c^{λ} term can be folded into the Miller loop almost for free by passing c^{-1} as the embedded exponent.

The soundness of this relation rests on Theorem 3 of the paper, which states: $e(P, Q) = 1$ if and only if there exists (c, u) such that $f \cdot u = c^{\lambda}$ *and* $u^{d^i} = 1$, where d^i is the order of a specific residue subgroup of $\mathbb{F}^*_{p^{12}}$ dictated by the curve (for BN254, $d^i = 27$, i.e. the honest hint always satisfies $u^{27} = 1$). The latter constraint, however, is missing from this implementation: `Bn254::try_honest_pairing_check` performs no subgroup check on u at all, and `Bls12_381::try_honest_pairing_check` enforces only a partial check on its scaling factor s while leaving the residue witness c unconstrained.

Notably, because u is underconstrained, the algebraic identity $f \cdot u = c^{\lambda}$ collapses into a single equation in $\mathbb{F}_{p^{12}}$ with two free unknowns and is therefore trivially satisfiable for *any* (P, Q), regardless of whether $e(P, Q) = 1$ actually holds. A malicious prover simply sets $c = 1$ (so $c^{\lambda} = 1$) and solves $u = f^{-1}$, where f is the Miller-loop output computed from the (possibly invalid) input pair. The forged hint $(1, f^{-1})$ then passes the verifier's check identically to an honest hint.

As a simple proof of concept, a malicious prover can apply the following diff to the host-side phantom executor that produces the hint, causing the zkVM to emit a valid proof for *any* `Bls12_381::pairing_check`.

```
diff --git a/extensions/pairing/circuit/src/pairing_extension.rs
b/extensions/pairing/circuit/src/pairing_extension.rs
index b5de43ca4..1305ff614 100644
--- a/extensions/pairing/circuit/src/pairing_extension.rs
+++ b/extensions/pairing/circuit/src/pairing_extension.rs
@@ -251,8 +251,10 @@ pub(crate) mod phantom {
                     })
                     .collect::<eyre::Result<Vec<_>>>()?;

-                let f: Fq12 = Bls12_381::multi_miller_loop(&p, &q);
-                let (c, u) = Bls12_381::final_exp_hint(&f);
+                let c = Fq12::one();
+                let c_inv = Fq12::one();
+                let fc = Bls12_381::multi_miller_loop_embedded_exp(&p, &q, Some(c_inv));
```

```
+                let u = fc.invert().unwrap();
                 hint_stream.clear();
                 hint_stream.extend(
                     c.to_coeffs()
```

The resulting proof verifies under the unmodified verifying key, so the forgery is indistinguishable from an honest proof to any downstream verifier that consumes it.

Note: this issue is already known to the client, having been reported by a zkSecurity member via responsible disclosure prior to the audit. It is included here for completeness since it is still in scope and has not been fixed during the audit.

Impact. A malicious prover can forge successful pairing checks for false pairing equations. Any guest program relying on `openvm-pairing` pairing verification as a dependency can be made to accept invalid cryptographic proofs or commitments inside the zkVM. This breaks soundness of the proven statement.

Recommendation. Before accepting the hinted relation, enforce the missing Theorem 3 subgroup condition $u^{d^i} = 1$ on the residue witness.

Client Response. This issue has been acknowledged and fixed by rebasing from v1.6.0.